

notebookd8e32598e0

December 15, 2024

Project Title: Leveraging Explainable Machine Learning for Detecting and Diagnosing Liver Disease
| EDA and Model

0.0.1 Table of Contents:

- Introduction
- Import Libraries
- Load Dataset
- EDA
- Preprocessing
- ML Models
- DL Models

Introduction

Tabel of Contents

Did you know that timely diagnosis and treatment of liver diseases can reduce the risk of severe complications and improve patient outcomes by up to 70%?

Liver diseases, particularly Fatty Liver Disease (FLD) and Hepatitis B (HBV), are critical health issues affecting millions globally. Traditional diagnostic methods can be invasive and costly, often leading to inaccuracies in detection. This thesis proposal aims to utilize explainable machine learning techniques to improve the diagnosis of liver diseases. By analyzing patient data and using SHapley Additive exPlanations (SHAP), we can identify key indicators influencing disease progression. The goal is to develop a machine learning-based system that provides accurate and interpretable predictions, enhancing the early detection and treatment of liver diseases.

Import Libraries

Tabel of Contents

```
[1]: # Manipulation and Visualasation
import pandas as pd
import numpy as np
import random
import matplotlib.pyplot as plt
import seaborn as sns
# Libraries for Machine Learning
from sklearn.model_selection import train_test_split
```

```

from sklearn.preprocessing import MinMaxScaler, LabelEncoder
from sklearn.model_selection import cross_val_score, GridSearchCV
from sklearn.metrics import confusion_matrix, classification_report, \
    accuracy_score, f1_score, precision_score, recall_score, roc_curve, \
    roc_auc_score
from sklearn.inspection import permutation_importance

# Models
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import GradientBoostingClassifier, AdaBoostClassifier
from sklearn.naive_bayes import GaussianNB

### DL Models
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

# Save Best Model
import pickle

# ignore warning
import warnings
warnings.filterwarnings('ignore')

```

Load Dataset

Tabel of Contents

```

[2]: data = pd.read_csv("/kaggle/input/predict-liver-disease-1700-records-dataset/
    ↪Liver_disease_data.csv")
data.head()

```

```

[2]:
   Age  Gender      BMI  AlcoholConsumption  Smoking  GeneticRisk  \
0   58      0  35.857584          17.272828         0             1
1   71      1  30.732470           2.201266         0             1
2   48      0  19.971407          18.500944         0             0
3   34      1  16.615417          12.632870         0             0
4   62      1  16.065830           1.087815         0             1

   PhysicalActivity  Diabetes  Hypertension  LiverFunctionTest  Diagnosis
0          0.658940         0           0          42.734240         1
1          1.670557         1           0          67.309822         1
2          9.928308         0           0          63.738956         0

```

3	5.630129	0	0	64.555873	1
4	3.566218	1	0	77.868689	1

```
[3]: ### Shape of Dataset
print(f"The dataset has {data.shape[0]} Rows and {data.shape[1]} columns")
```

The dataset has 1700 Rows and 11 columns

```
[4]: ### Info of Dataset
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1700 entries, 0 to 1699
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Age                   1700 non-null   int64
1   Gender                1700 non-null   int64
2   BMI                   1700 non-null   float64
3   AlcoholConsumption    1700 non-null   float64
4   Smoking               1700 non-null   int64
5   GeneticRisk           1700 non-null   int64
6   PhysicalActivity       1700 non-null   float64
7   Diabetes              1700 non-null   int64
8   Hypertension          1700 non-null   int64
9   LiverFunctionTest     1700 non-null   float64
10  Diagnosis              1700 non-null   int64
dtypes: float64(4), int64(7)
memory usage: 146.2 KB
```

```
[5]: ### Descriptive Analysis
data.describe().transpose()
```

```
[5]:
```

	count	mean	std	min	25%	75%	max
Age	1700.0	50.394118	17.641915	20.000000	35.000000	66.000000	80.000000
Gender	1700.0	0.504118	0.500130	0.000000	0.000000	0.000000	0.000000
BMI	1700.0	27.699801	7.210400	15.004710	21.455414	27.699801	40.000000
AlcoholConsumption	1700.0	9.832309	5.757472	0.003731	4.841811	9.832309	20.000000
Smoking	1700.0	0.291765	0.454708	0.000000	0.000000	0.000000	0.000000
GeneticRisk	1700.0	0.521765	0.666262	0.000000	0.000000	0.000000	0.000000
PhysicalActivity	1700.0	5.000993	2.847074	0.001852	2.622121	5.000993	10.000000
Diabetes	1700.0	0.142353	0.349515	0.000000	0.000000	0.000000	0.000000
Hypertension	1700.0	0.154706	0.361730	0.000000	0.000000	0.000000	0.000000
LiverFunctionTest	1700.0	59.863867	22.996262	20.019254	40.024216	59.863867	80.000000
Diagnosis	1700.0	0.550588	0.497581	0.000000	0.000000	0.000000	0.000000

Gender	1.000000	1.000000	1.000000
BMI	27.925367	33.957668	39.992845
AlcoholConsumption	9.828195	14.871671	19.952456
Smoking	0.000000	1.000000	1.000000
GeneticRisk	0.000000	1.000000	2.000000
PhysicalActivity	5.022883	7.401642	9.994964
Diabetes	0.000000	0.000000	1.000000
Hypertension	0.000000	0.000000	1.000000
LiverFunctionTest	59.513146	79.428755	99.991413
Diagnosis	1.000000	1.000000	1.000000

```
[6]: ### Missing Values
data.isna().sum()
```

```
[6]: Age          0
Gender         0
BMI            0
AlcoholConsumption  0
Smoking        0
GeneticRisk    0
PhysicalActivity  0
Diabetes       0
Hypertension   0
LiverFunctionTest  0
Diagnosis      0
dtype: int64
```

```
[7]: ### Duplicated Values
data[data.duplicated()]
```

```
[7]: Empty DataFrame
Columns: [Age, Gender, BMI, AlcoholConsumption, Smoking, GeneticRisk,
PhysicalActivity, Diabetes, Hypertension, LiverFunctionTest, Diagnosis]
Index: []
```

EDA

Tabel of Contents

```
[8]: ### Age Distrbution
data.Age.value_counts().to_frame()
```

```
[8]:      count
Age
52      42
45      41
70      39
68      37
```

```

66      37
..      ...
50      21
37      21
61      20
62      18
65      18

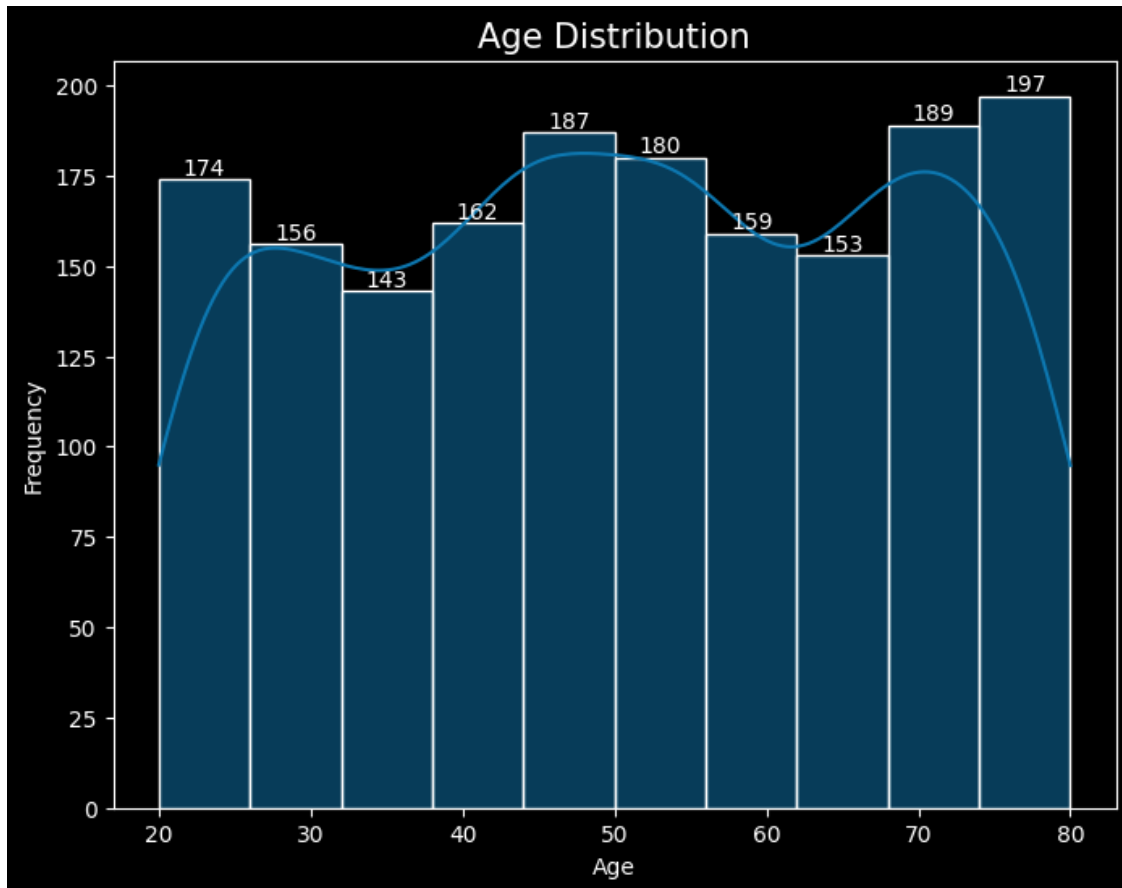
```

```
[61 rows x 1 columns]
```

```

[9]: plt.style.use('dark_background')
plt.figure(figsize=(8, 6))
ax = sns.histplot(data['Age'], bins=10, kde=True, color='#0E79B2')
for container in ax.containers:
    ax.bar_label(container)
plt.title('Age Distribution', fontsize=15)
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.show()

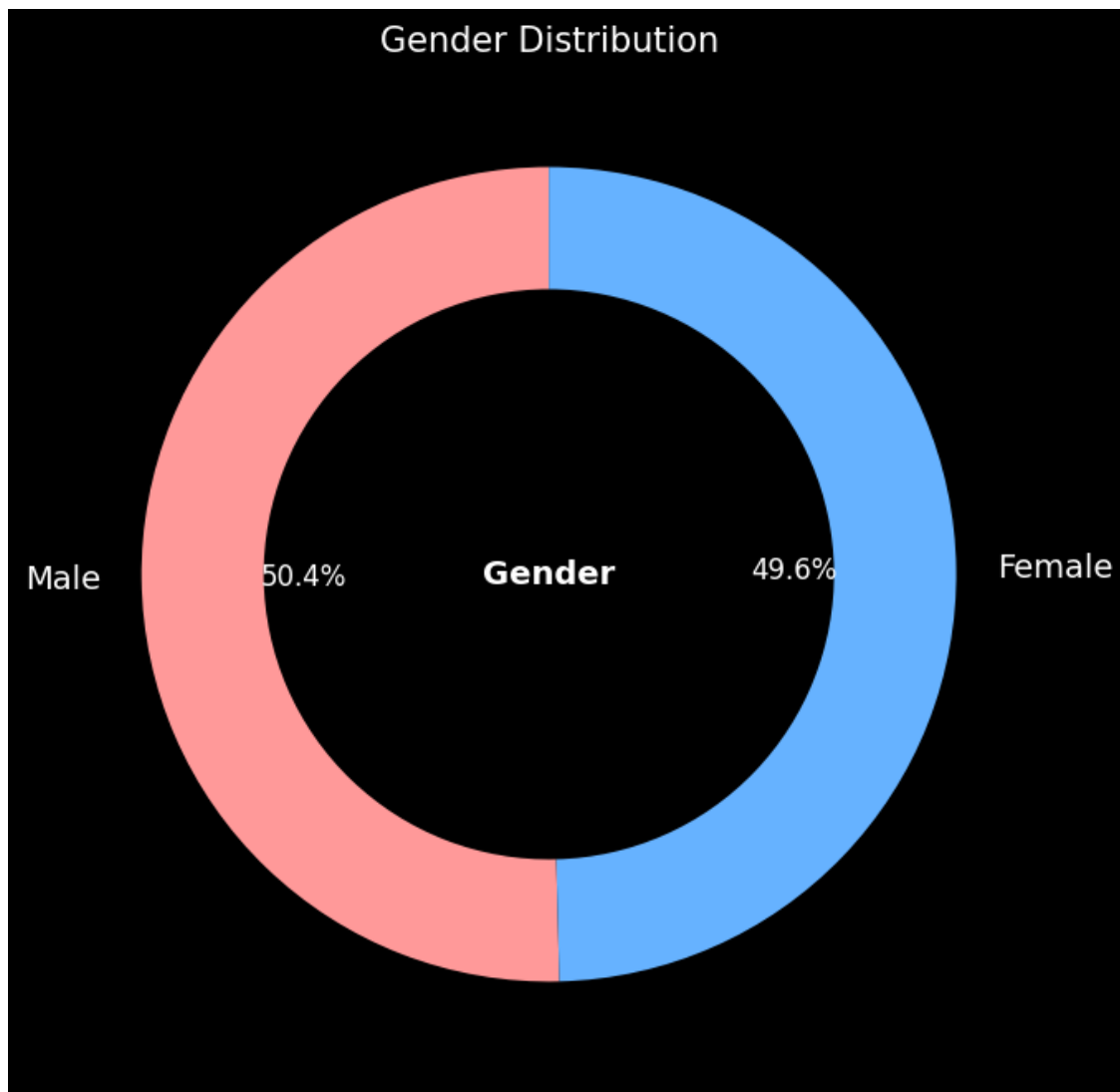
```



```
[10]: ### Gender Distrbution  
data.Gender.value_counts().to_frame()
```

```
[10]:      count  
Gender  
1      857  
0      843
```

```
[11]: colors = ['#FF9999', '#66B2FF']  
plt.figure(figsize=(8,8))  
wedges, texts, autotexts = plt.pie(  
    data.Gender.value_counts(), labels=data.Gender.value_counts().index.map({0: 'Female', 1: 'Male'}), autopct='%1.1f%%', startangle=90,  
    wedgeprops={'width': 0.3}, colors=colors  
)  
plt.text(0, 0, 'Gender', ha='center', va='center', fontsize=14,  
    fontweight='bold')  
plt.title("Gender Distribution", fontsize=15)  
plt.setp(autotexts, size=12, color="white")  
plt.setp(texts, size=14, color="white")  
plt.show()
```

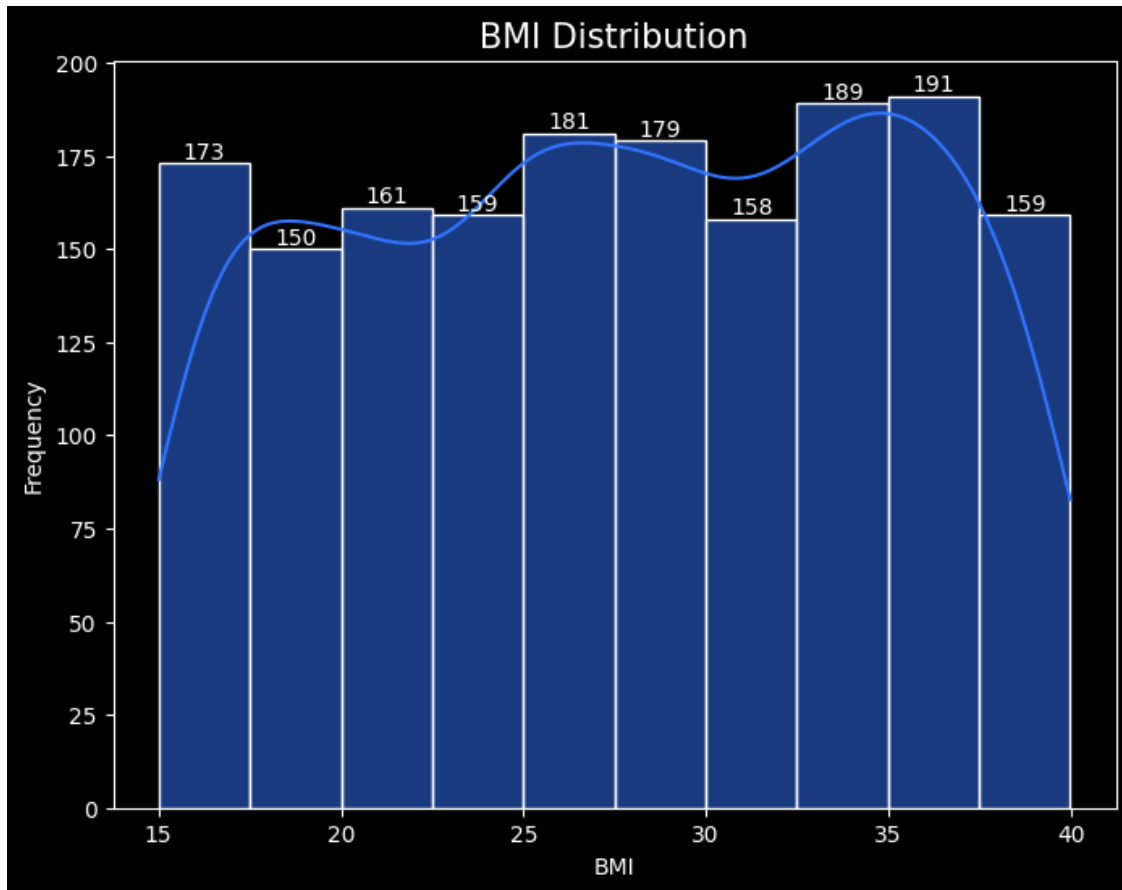


```
[12]: ### BMI Distrbution
min_bmi = data['BMI'].min()
max_bmi = data['BMI'].max()
# Print the results
print(f"Minimum BMI: {min_bmi}")
print(f"Maximum BMI: {max_bmi}")
```

Minimum BMI: 15.004709988914549
Maximum BMI: 39.99284510389785

```
[13]: plt.figure(figsize=(8, 6))
ax = sns.histplot(data['BMI'], bins=10, kde=True, color='#3375FF')
for container in ax.containers:
    ax.bar_label(container)
```

```
plt.title('BMI Distribution', fontsize=15)
plt.xlabel('BMI')
plt.ylabel('Frequency')
plt.show()
```



```
[14]: bins = [0, 18.5, 25, 30, 35, np.inf]
names = ['Underweight', 'Normal weight', 'Overweight', 'Obese', 'Extremly_
↳Obese']
data['SegmentBMI'] = pd.cut(data['BMI'],bins, labels=names)
#data.drop('BMI',axis=1,inplace=True)
data.head()
```

```
[14]:
```

	Age	Gender	BMI	AlcoholConsumption	Smoking	GeneticRisk	\
0	58	0	35.857584	17.272828	0	1	
1	71	1	30.732470	2.201266	0	1	
2	48	0	19.971407	18.500944	0	0	
3	34	1	16.615417	12.632870	0	0	
4	62	1	16.065830	1.087815	0	1	

	PhysicalActivity	Diabetes	Hypertension	LiverFunctionTest	Diagnosis	\
0	0.658940	0	0	42.734240	1	
1	1.670557	1	0	67.309822	1	
2	9.928308	0	0	63.738956	0	
3	5.630129	0	0	64.555873	1	
4	3.566218	1	0	77.868689	1	

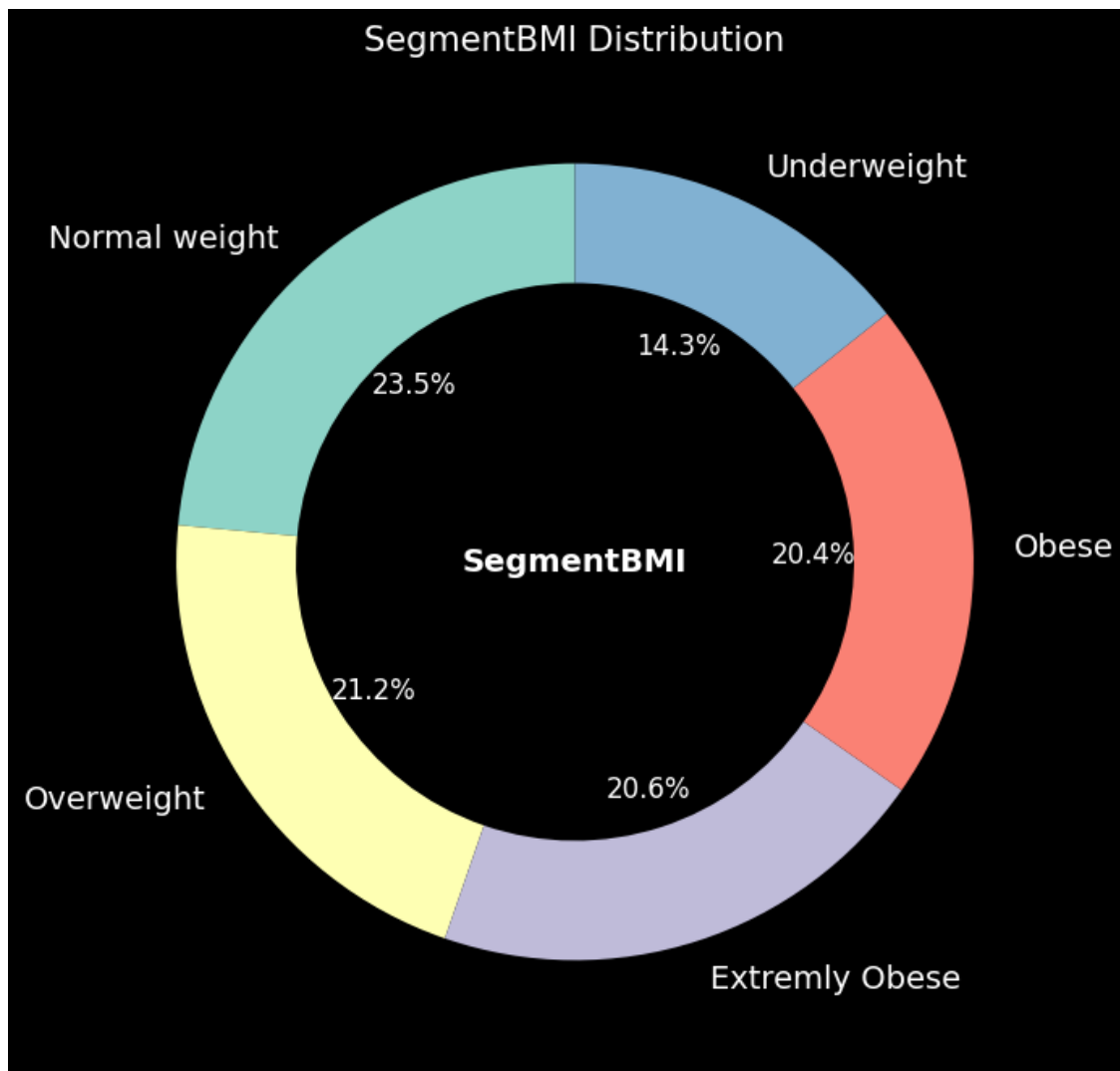
	SegmentBMI
0	Extremly Obese
1	Obese
2	Normal weight
3	Underweight
4	Underweight

```
[15]: data.SegmentBMI.value_counts().to_frame()
```

```
[15]:
```

	count
SegmentBMI	
Normal weight	400
Overweight	360
Extremly Obese	350
Obese	347
Underweight	243

```
[16]: plt.figure(figsize=(8, 8))
wedges, texts, autotexts = plt.pie(
    data.SegmentBMI.value_counts(), labels=data.SegmentBMI.value_counts().
    ↪index, autopct='%1.1f%%', startangle=90,
    wedgeprops={'width': 0.3}
)
plt.text(0, 0, 'SegmentBMI', ha='center', va='center', fontsize=14,
    ↪fontweight='bold')
plt.title("SegmentBMI Distribution", fontsize=15)
plt.setp(autotexts, size=12, color="white")
plt.setp(texts, size=14, color="white")
plt.show()
```

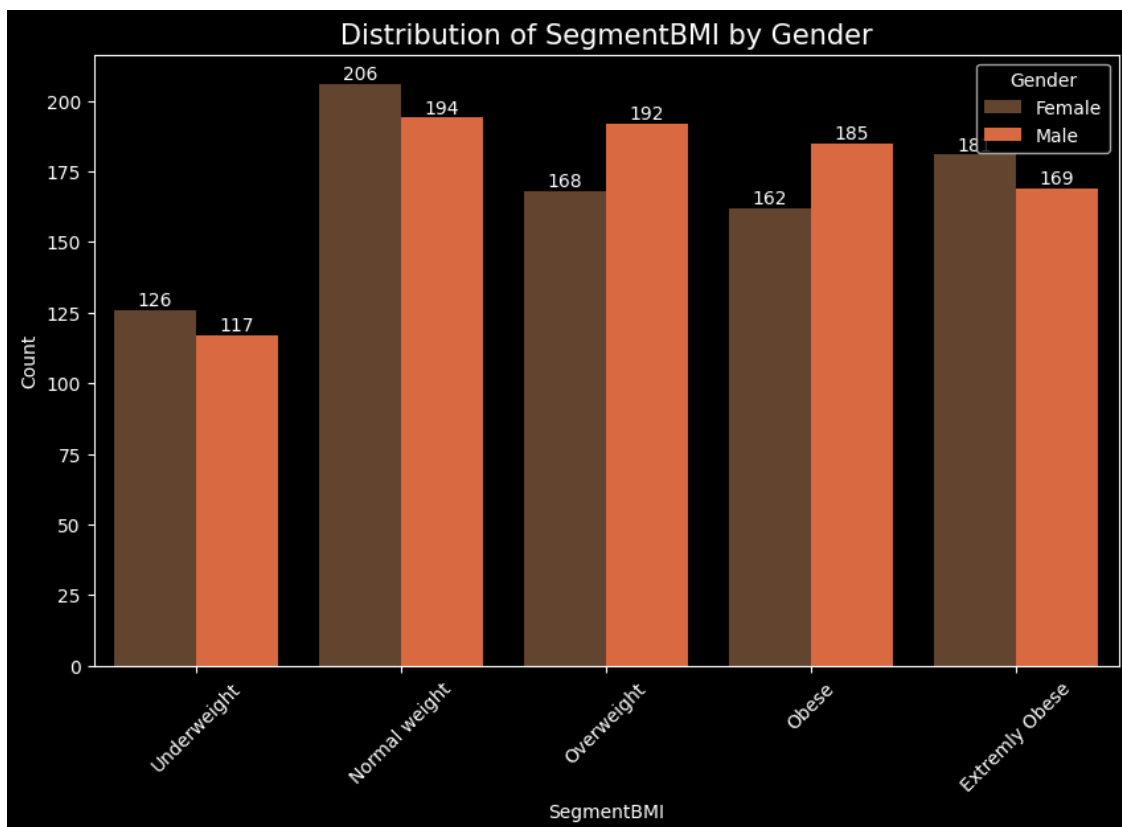


```
[17]: ### BMI VS Gender
data.groupby(['Gender', 'SegmentBMI'])['SegmentBMI'].count()
```

```
[17]: Gender  SegmentBMI
0      Underweight      126
      Normal weight      206
      Overweight        168
      Obese              162
      Extremely Obese    181
1      Underweight      117
      Normal weight      194
      Overweight        192
      Obese              185
      Extremely Obese    169
```

Name: SegmentBMI, dtype: int64

```
[18]: data2 = data.copy()
data2.Gender = data2.Gender.map({0: 'Female', 1: 'Male'})
grouped_data = data2.groupby(['Gender', 'SegmentBMI'])['SegmentBMI'].count().
    ↪reset_index(name='Count')
colors = ['#6B4226', '#F25D27']
plt.figure(figsize=(10, 6))
ax = sns.barplot(data=grouped_data, x='SegmentBMI', y='Count',
    ↪hue='Gender', palette=colors)
for container in ax.containers:
    ax.bar_label(container)
plt.title("Distribution of SegmentBMI by Gender", fontsize=15)
plt.xlabel("SegmentBMI")
plt.ylabel("Count")
plt.legend(title="Gender")
plt.xticks(rotation=45)
plt.show()
```



```
[19]: ### AlcoholConsumption Distribution
min_alcohol = data['AlcoholConsumption'].min()
```

```

max_alcohol = data['AlcoholConsumption'].max()
# Print the results
print(f"Minimum AlcoholConsumption: {min_alcohol}")
print(f"Maximum AlcoholConsumption: {max_alcohol}")

```

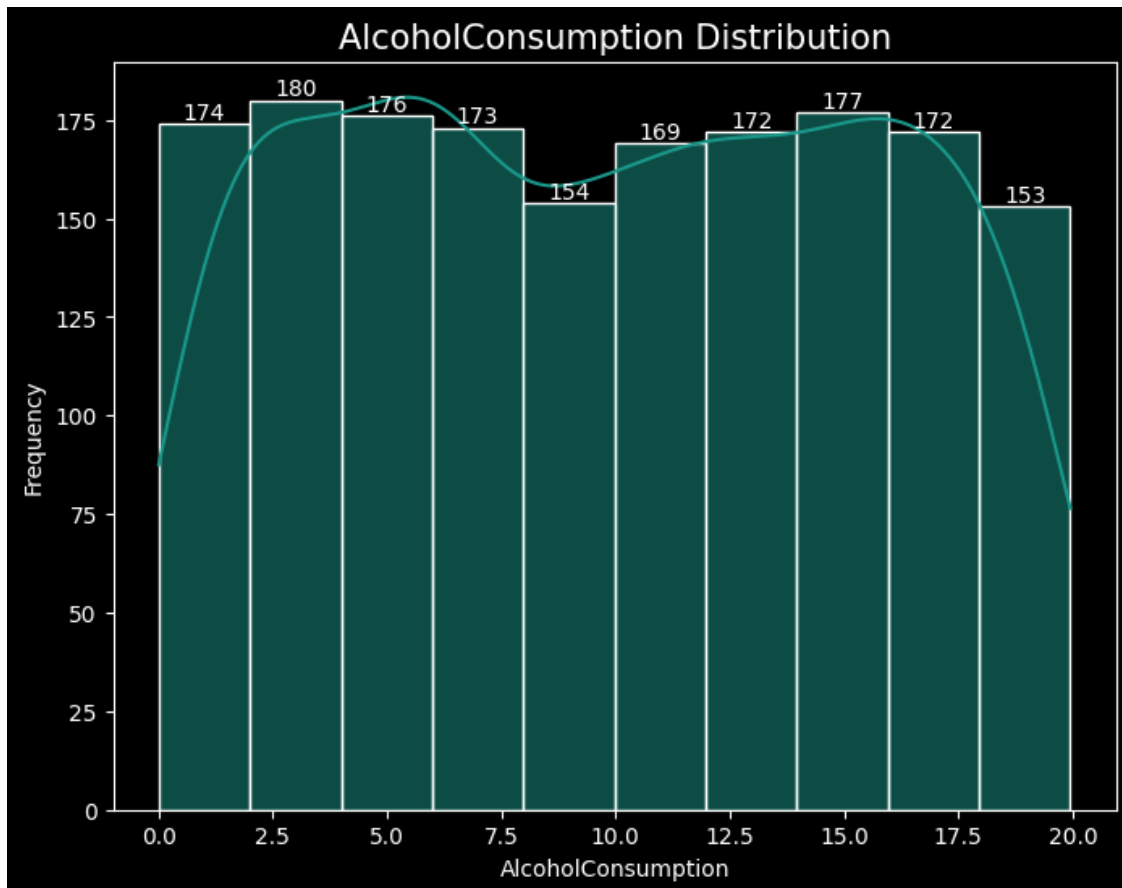
Minimum AlcoholConsumption: 0.0037305634132622

Maximum AlcoholConsumption: 19.952456445027707

```

[20]: plt.figure(figsize=(8, 6))
ax = sns.histplot(data['AlcoholConsumption'], bins=10, kde=True,
color='#1B998B')
for container in ax.containers:
    ax.bar_label(container)
plt.title('AlcoholConsumption Distribution', fontsize=15)
plt.xlabel('AlcoholConsumption')
plt.ylabel('Frequency')
plt.show()

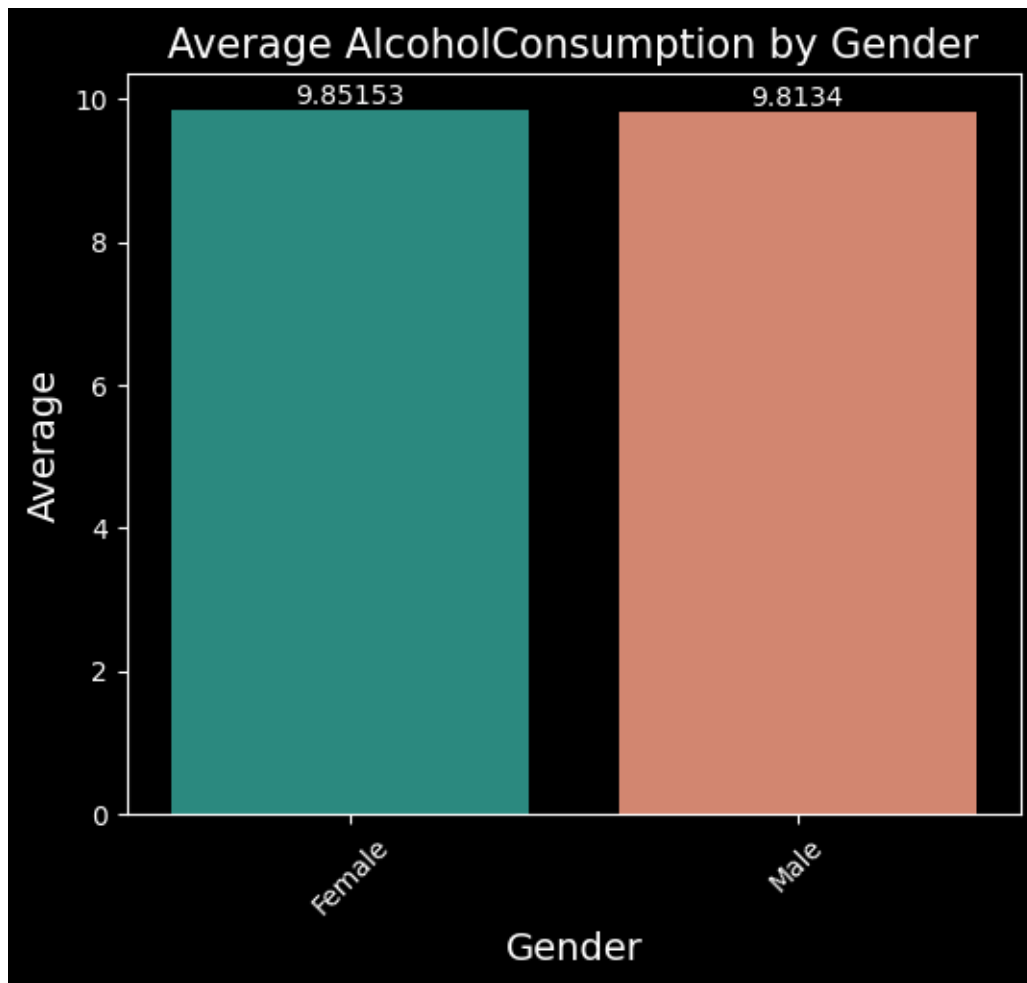
```



```
[21]: ### Average AlcoholConsumption VS Gender
pivot = pd.pivot_table(data, values="AlcoholConsumption", index="Gender",
    ↪aggfunc='mean')
pivot
```

```
[21]:      AlcoholConsumption
Gender
0          9.851531
1          9.813402
```

```
[22]: pivot1 = pivot.reset_index()
pivot1['Gender'] = pivot1['Gender'].map({0: 'Female', 1: 'Male'})
colors = ['#1B998B', '#E27D60']
plt.figure(figsize=(6, 5))
ax = sns.barplot(x=pivot1["Gender"], y=pivot1["AlcoholConsumption"],
    ↪palette=colors)
ax.set_title("Average AlcoholConsumption by Gender", fontsize=15, color="white")
ax.set_xlabel("Gender", fontsize=14, color="white")
ax.set_ylabel("Average", fontsize=14, color="white")
for container in ax.containers:
    ax.bar_label(container)
plt.xticks(rotation=45)
plt.show()
```

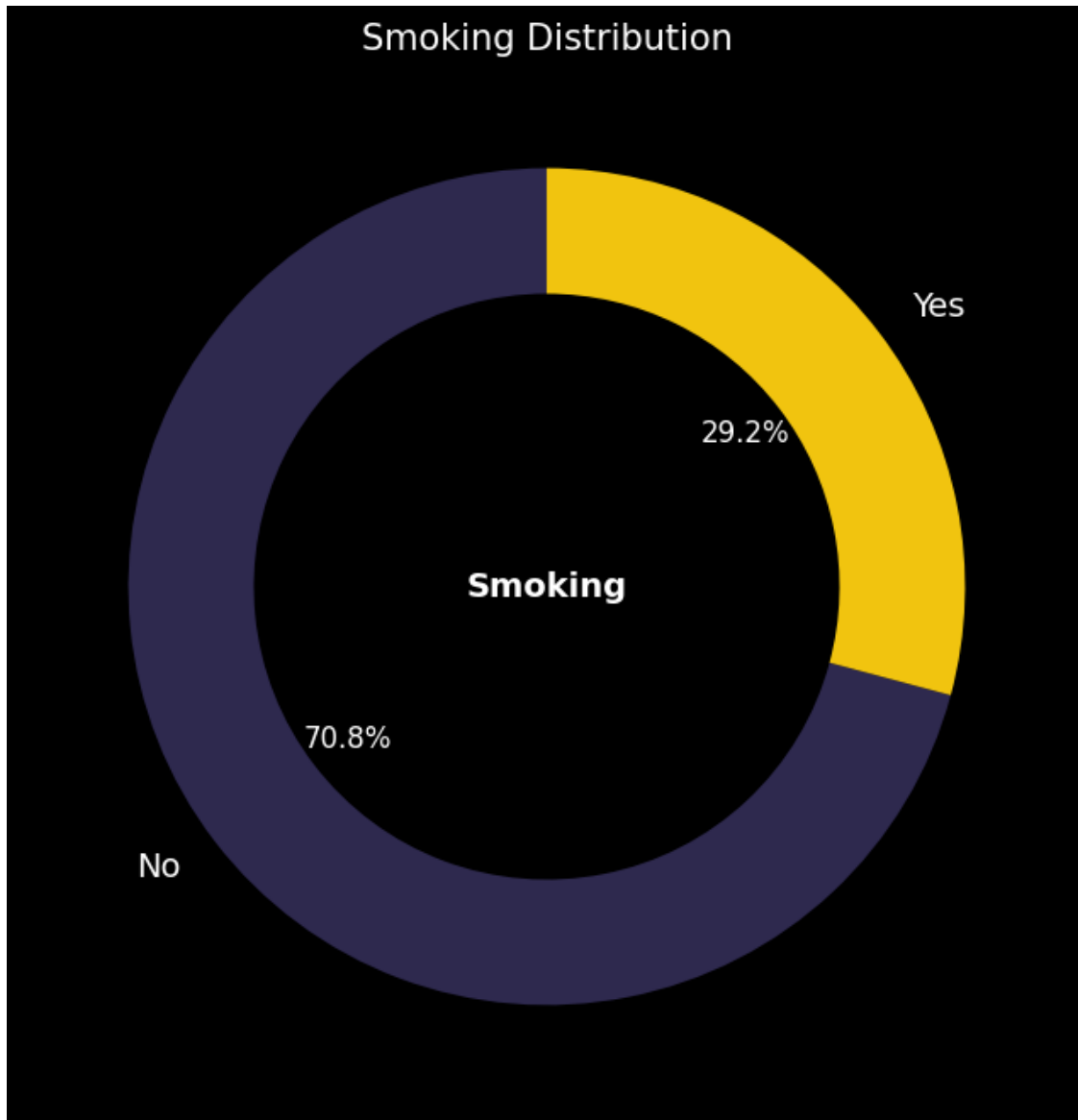


```
[23]: ### Smoking Distribution
data.Smoking.value_counts().to_frame()
```

```
[23]:      count
Smoking
0        1204
1         496
```

```
[24]: colors = ['#2E294E', '#f1c40f']
plt.figure(figsize=(8,8))
wedges, texts, autotexts = plt.pie(
    data.Smoking.value_counts(), labels=data.Smoking.value_counts().index,
    ↪map({0: 'No', 1: 'Yes'}), autopct='%1.1f%%', startangle=90,
    wedgeprops={'width': 0.3}, colors=colors
)
plt.text(0, 0, 'Smoking', ha='center', va='center', fontsize=14,
↪fontweight='bold')
```

```
plt.title("Smoking Distribution", fontsize=15)
plt.setp(autotexts, size=12, color="white")
plt.setp(texts, size=14, color="white")
plt.show()
```

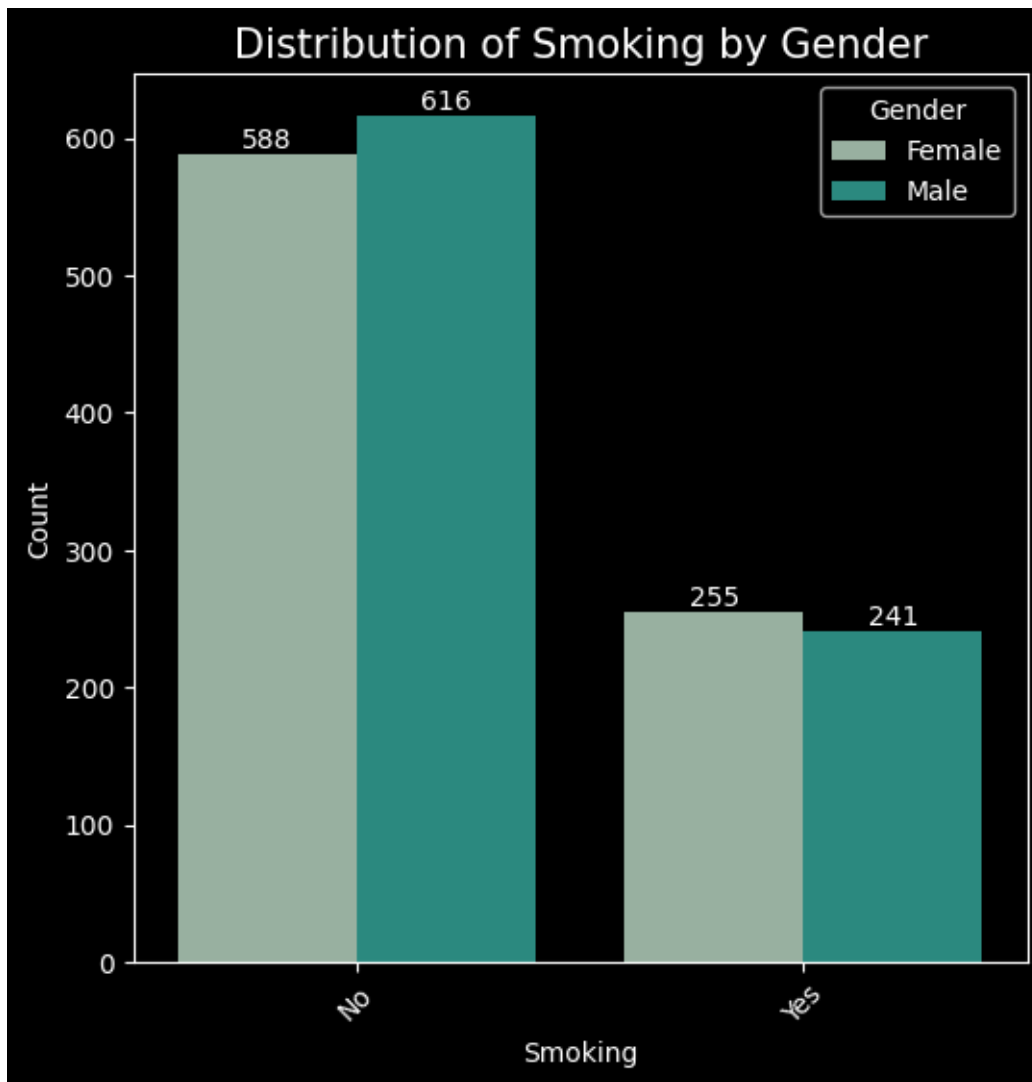


```
[25]: ### Smoking VS Gender
data.groupby(['Gender', 'Smoking'])['Smoking'].count()
```

```
[25]: Gender  Smoking
0         0         588
        1         255
```

```
1      0      616
      1      241
Name: Smoking, dtype: int64
```

```
[26]: data2 = data.copy()
data2.Gender = data2.Gender.map({0: 'Female', 1: 'Male'})
data2.Smoking = data2.Smoking.map({0: 'No', 1: 'Yes'})
grouped_data = data2.groupby(['Smoking', 'Gender'])['Smoking'].count().
    ↪reset_index(name='Count')
colors = ['#94B49F', '#1B998B']
plt.figure(figsize=(6, 6))
ax = sns.barplot(data=grouped_data, x='Smoking', y='Count',
    ↪hue='Gender', palette=colors)
for container in ax.containers:
    ax.bar_label(container)
plt.title("Distribution of Smoking by Gender", fontsize=15)
plt.xlabel("Smoking")
plt.ylabel("Count")
plt.legend(title="Gender")
plt.xticks(rotation=45)
plt.show()
```

```
[27]: ### GeneticRisk Distribution
data.GeneticRisk.value_counts().to_frame()
```

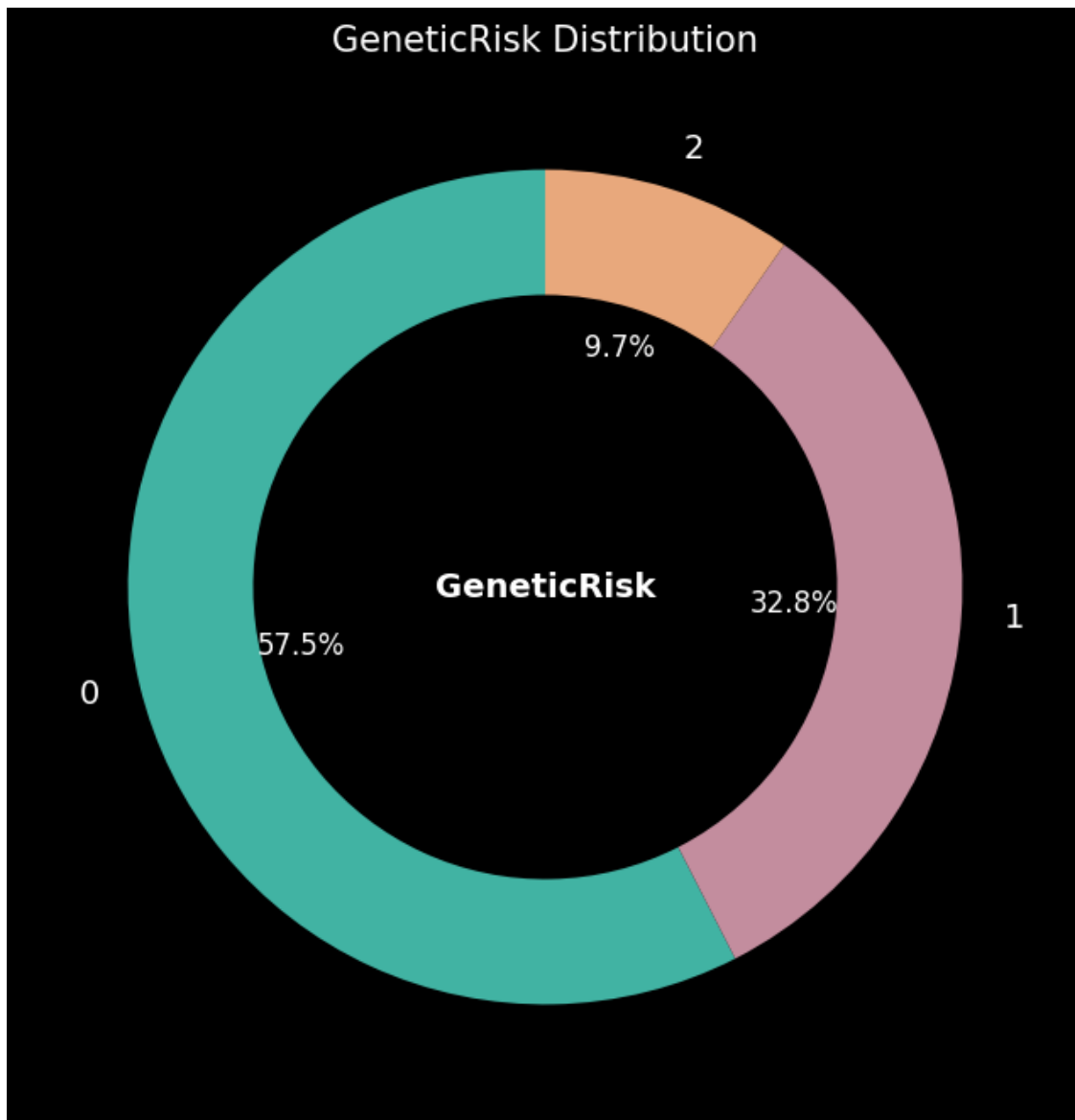
```
[27]:          count
GeneticRisk
0           978
1           557
2           165
```

```
[28]: colors = ['#41B3A3', '#C38D9E', '#E8A87C']
plt.figure(figsize=(8,8))
wedges, texts, autotexts = plt.pie(
    data.GeneticRisk.value_counts(), labels=data.GeneticRisk.value_counts().
    ↪index, autopct='%1.1f%%', startangle=90,
```

```

    wedgeprops={'width': 0.3}, colors=colors
)
plt.text(0, 0, 'GeneticRisk', ha='center', va='center', fontsize=14,
        fontweight='bold')
plt.title("GeneticRisk Distribution", fontsize=15)
plt.setp(autotexts, size=12, color="white")
plt.setp(texts, size=14, color="white")
plt.show()

```



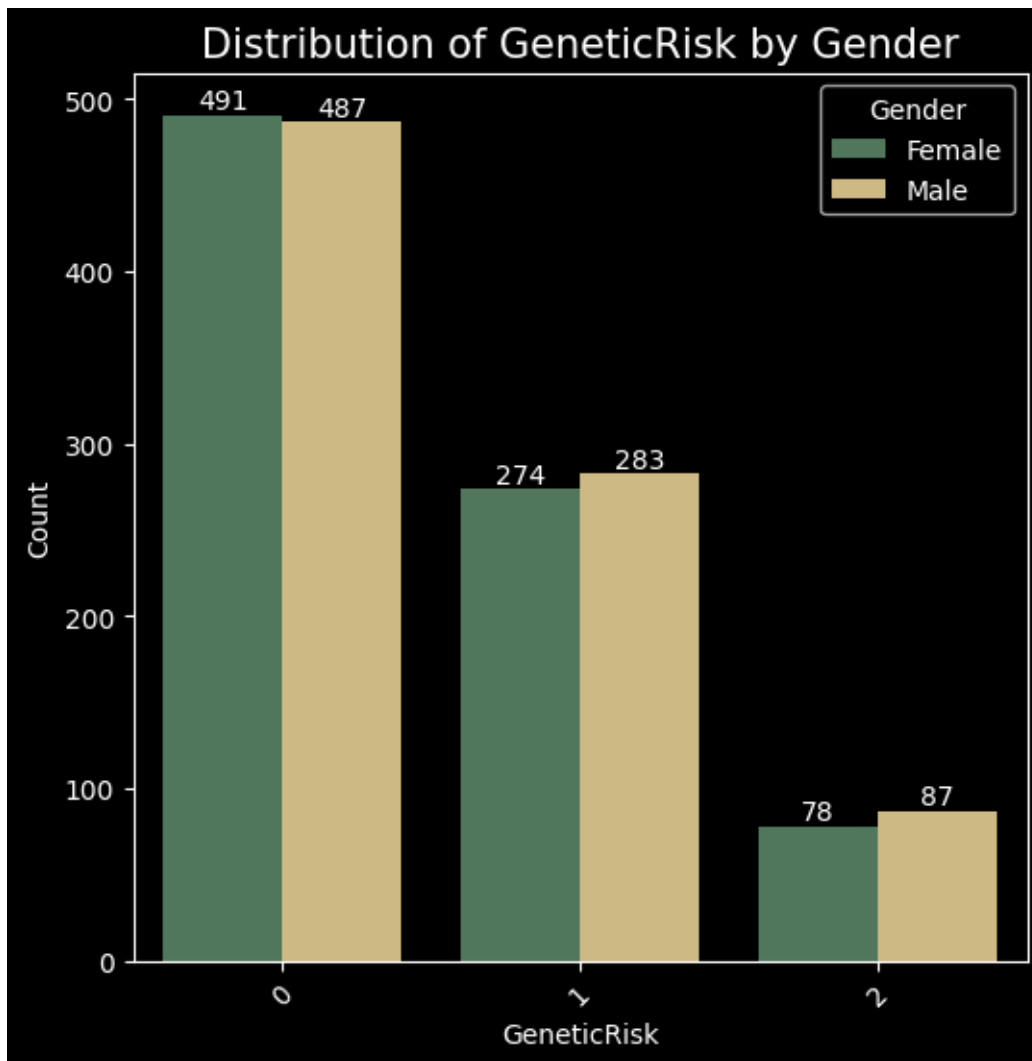
```

[29]: ### GeneticRisk VS Gender
data.groupby(['Gender', 'GeneticRisk'])['GeneticRisk'].count()

```

```
[29]: Gender  GeneticRisk
      0         0          491
          1          274
          2           78
      1         0          487
          1          283
          2           87
      Name: GeneticRisk, dtype: int64
```

```
[30]: data2 = data.copy()
data2.Gender = data2.Gender.map({0: 'Female', 1: 'Male'})
grouped_data = data2.groupby(['GeneticRisk', 'Gender'])['GeneticRisk'].count().
    ↪reset_index(name='Count')
colors = ['#4A7C59', '#D9BF77']
plt.figure(figsize=(6, 6))
ax = sns.barplot(data=grouped_data, x='GeneticRisk', y='Count',
    ↪hue='Gender', palette=colors)
for container in ax.containers:
    ax.bar_label(container)
plt.title("Distribution of GeneticRisk by Gender", fontsize=15)
plt.xlabel("GeneticRisk")
plt.ylabel("Count")
plt.legend(title="Gender")
plt.xticks(rotation=45)
plt.show()
```



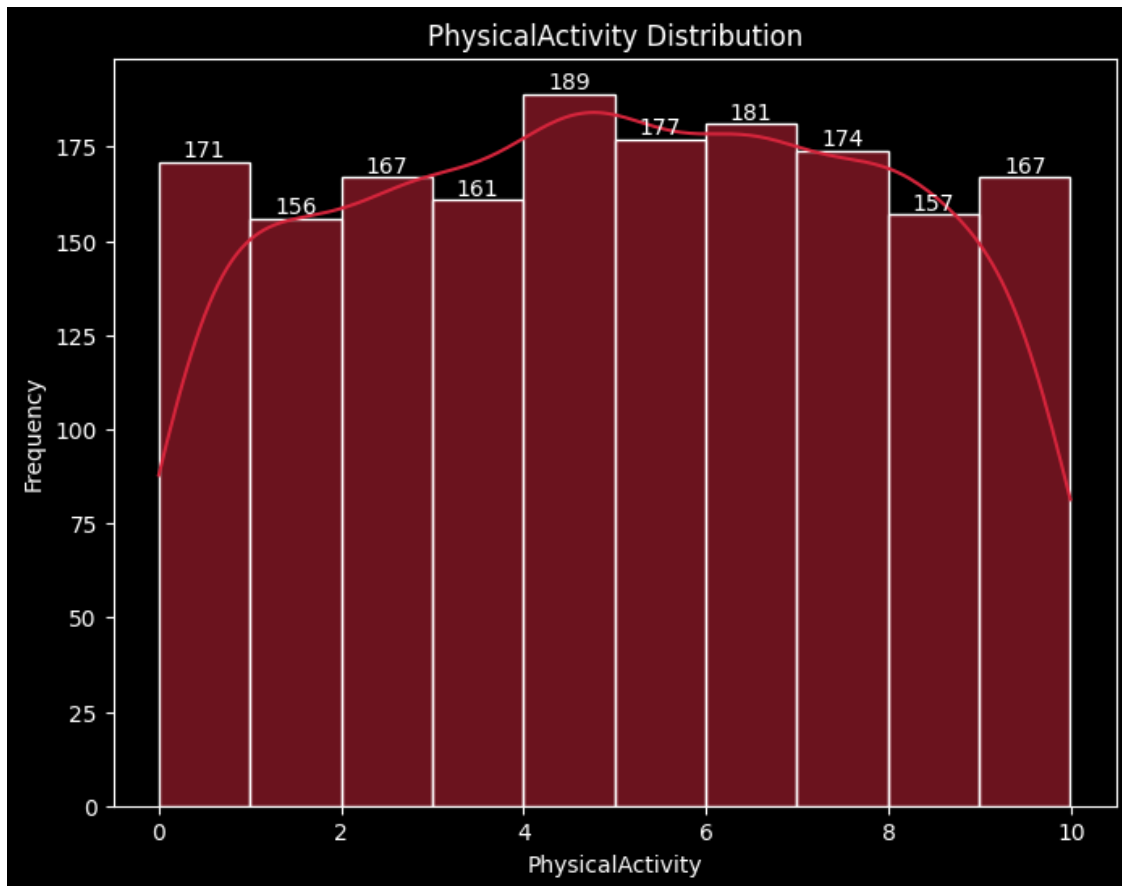
```
[31]: ### PhysicalActivity Distribution
min_phy = data['PhysicalActivity'].min()
max_phy = data['PhysicalActivity'].max()
# Print the results
print(f"Minimum PhysicalActivity: {min_phy}")
print(f"Maximum PhysicalActivity: {max_phy}")
```

Minimum PhysicalActivity: 0.001851952954428

Maximum PhysicalActivity: 9.994963504187217

```
[32]: plt.figure(figsize=(8, 6))
ax = sns.histplot(data['PhysicalActivity'], bins=10, kde=True, color='#D7263D')
for container in ax.containers:
    ax.bar_label(container)
plt.title('PhysicalActivity Distribution')
```

```
plt.xlabel('PhysicalActivity')
plt.ylabel('Frequency')
plt.show()
```



```
[33]: ### Average PhysicalActivity VS Gender
pivot = pd.pivot_table(data, values="PhysicalActivity", index="Gender",
    ↪aggfunc='mean')
pivot
```

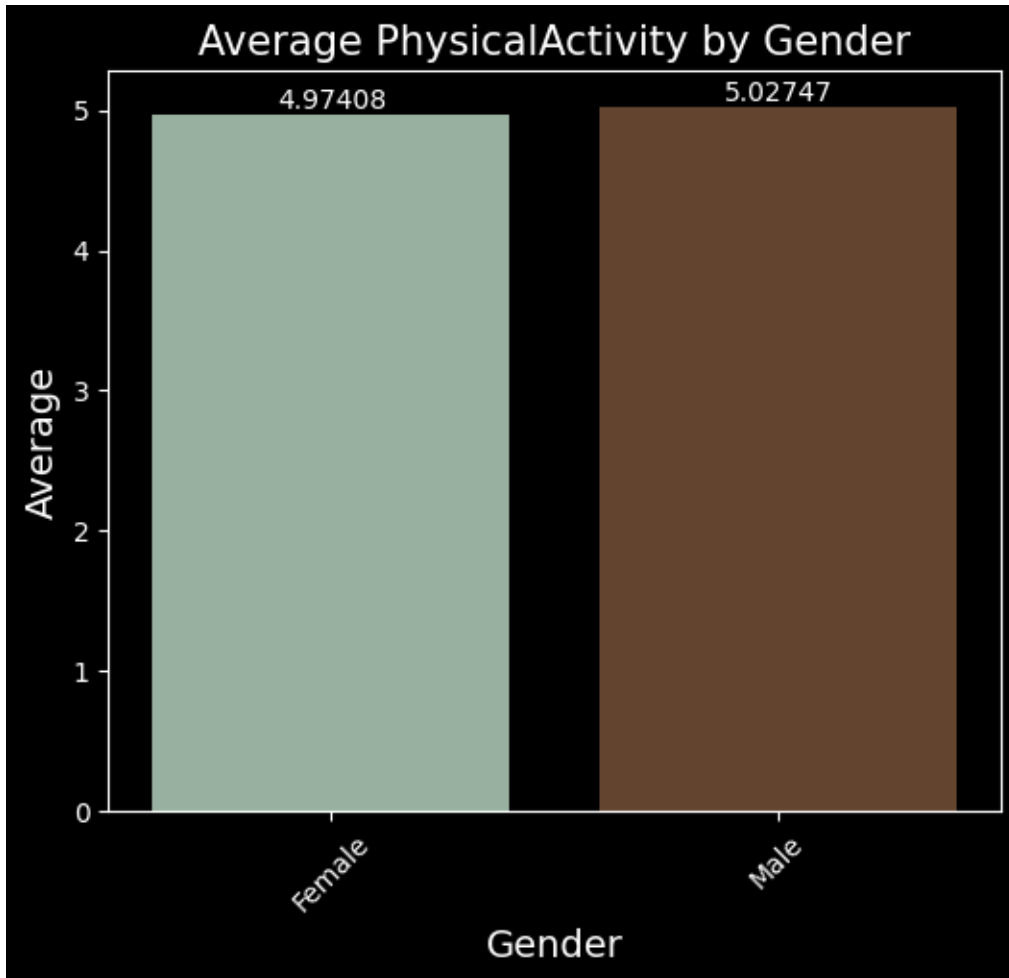
```
[33]:      PhysicalActivity
Gender
0          4.974077
1          5.027470
```

```
[34]: pivot1 = pivot.reset_index()
pivot1['Gender'] = pivot1['Gender'].map({0: 'Female', 1: 'Male'})
colors = ['#94B49F', '#6B4226']
plt.figure(figsize=(6, 5))
```

```

ax = sns.barplot(x=pivot1["Gender"], y=pivot1["PhysicalActivity"],
                 palette=colors)
ax.set_title("Average PhysicalActivity by Gender", fontsize=15, color="white")
ax.set_xlabel("Gender", fontsize=14, color="white")
ax.set_ylabel("Average", fontsize=14, color="white")
for container in ax.containers:
    ax.bar_label(container)
plt.xticks(rotation=45)
plt.show()

```



```

[35]: ### Diabetes Distribution
data.Diabetes.value_counts().to_frame()

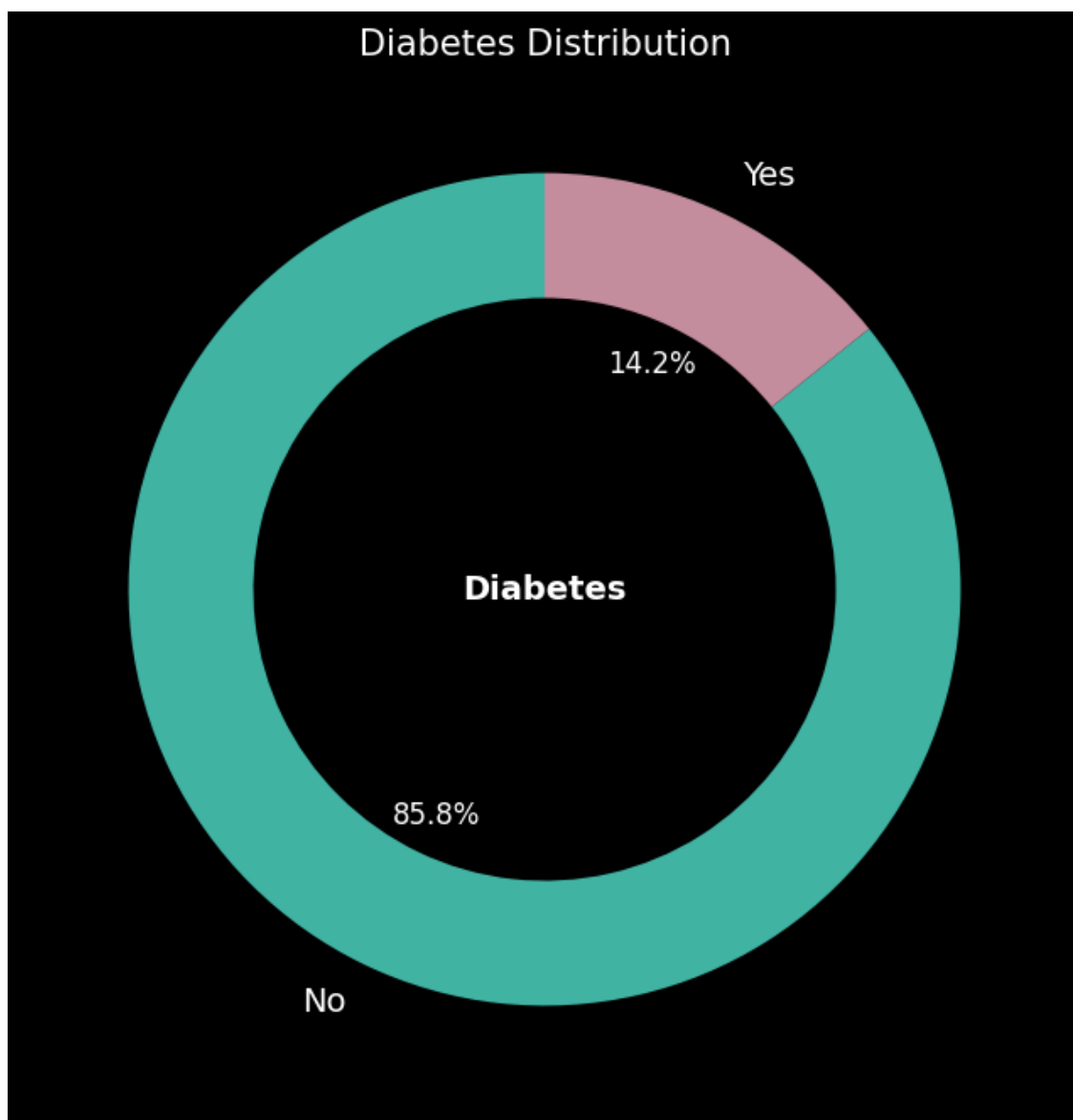
```

```

[35]:
      count
Diabetes
0        1458
1         242

```

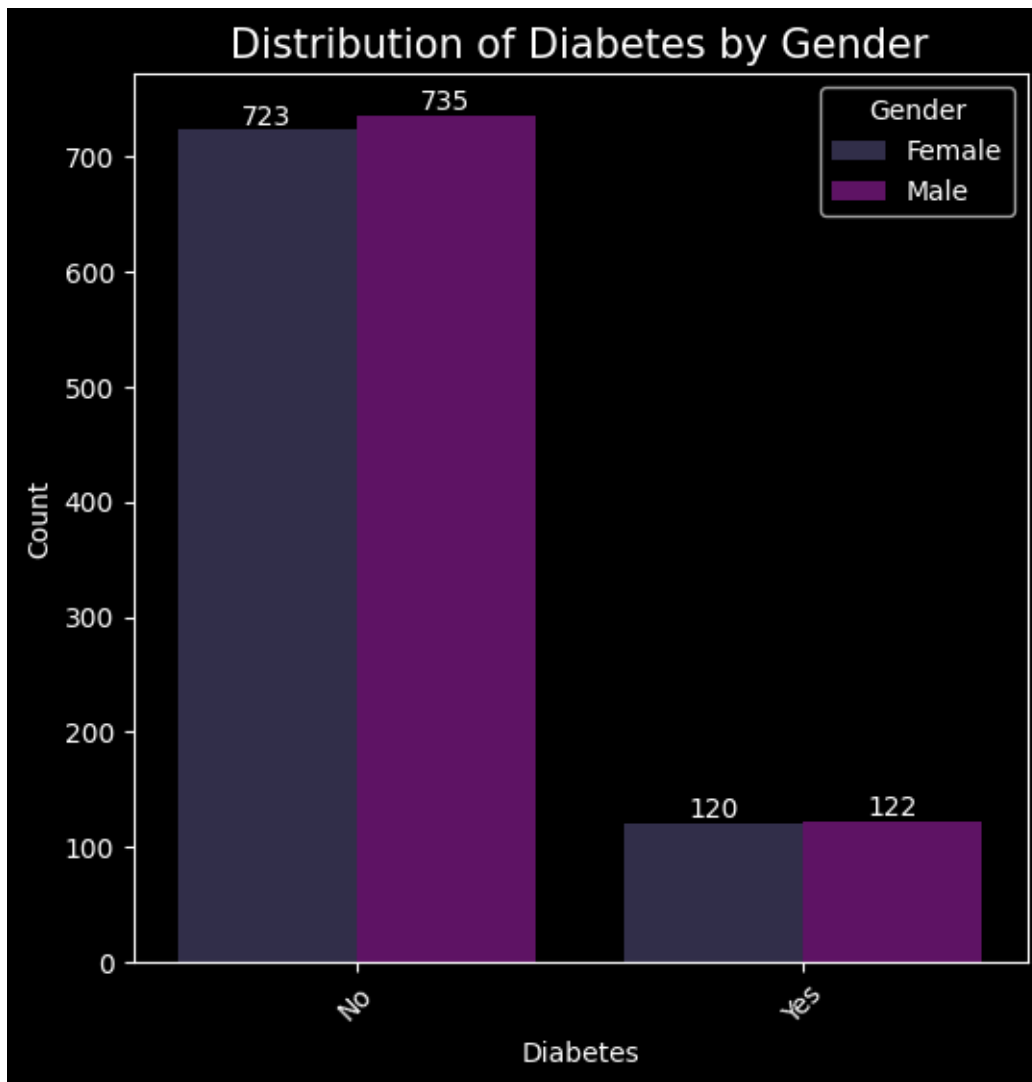
```
[36]: colors = ['#41B3A3', '#C38D9E']
plt.figure(figsize=(8,8))
wedges, texts, autotexts = plt.pie(
    data.Diabetes.value_counts(), labels=data.Diabetes.value_counts().index,
    ↪map({0: 'No', 1: 'Yes'}), autopct='%1.1f%%', startangle=90,
    wedgeprops={'width': 0.3}, colors=colors
)
plt.text(0, 0, 'Diabetes', ha='center', va='center', fontsize=14, ↪
    ↪fontweight='bold')
plt.title("Diabetes Distribution", fontsize=15)
plt.setp(autotexts, size=12, color="white")
plt.setp(texts, size=14, color="white")
plt.show()
```



```
[37]: ### Diabetes VS Gender
data.groupby(['Gender', 'Diabetes'])['Diabetes'].count()
```

```
[37]: Gender  Diabetes
0         0         723
        1         120
1         0         735
        1         122
Name: Diabetes, dtype: int64
```

```
[38]: data2 = data.copy()
data2.Gender = data2.Gender.map({0: 'Female', 1: 'Male'})
data2.Diabetes = data2.Diabetes.map({0: 'No', 1: 'Yes'})
grouped_data = data2.groupby(['Diabetes', 'Gender'])['Diabetes'].count().
    ↪reset_index(name='Count')
colors = ['#2E294E', '#6A0572']
plt.figure(figsize=(6, 6))
ax = sns.barplot(data=grouped_data, x='Diabetes', y='Count',
    ↪hue='Gender', palette=colors)
for container in ax.containers:
    ax.bar_label(container)
plt.title("Distribution of Diabetes by Gender", fontsize=15)
plt.xlabel("Diabetes")
plt.ylabel("Count")
plt.legend(title="Gender")
plt.xticks(rotation=45)
plt.show()
```

```
[39]: ### Hypertension Distribution
data.Hypertension.value_counts().to_frame()
```

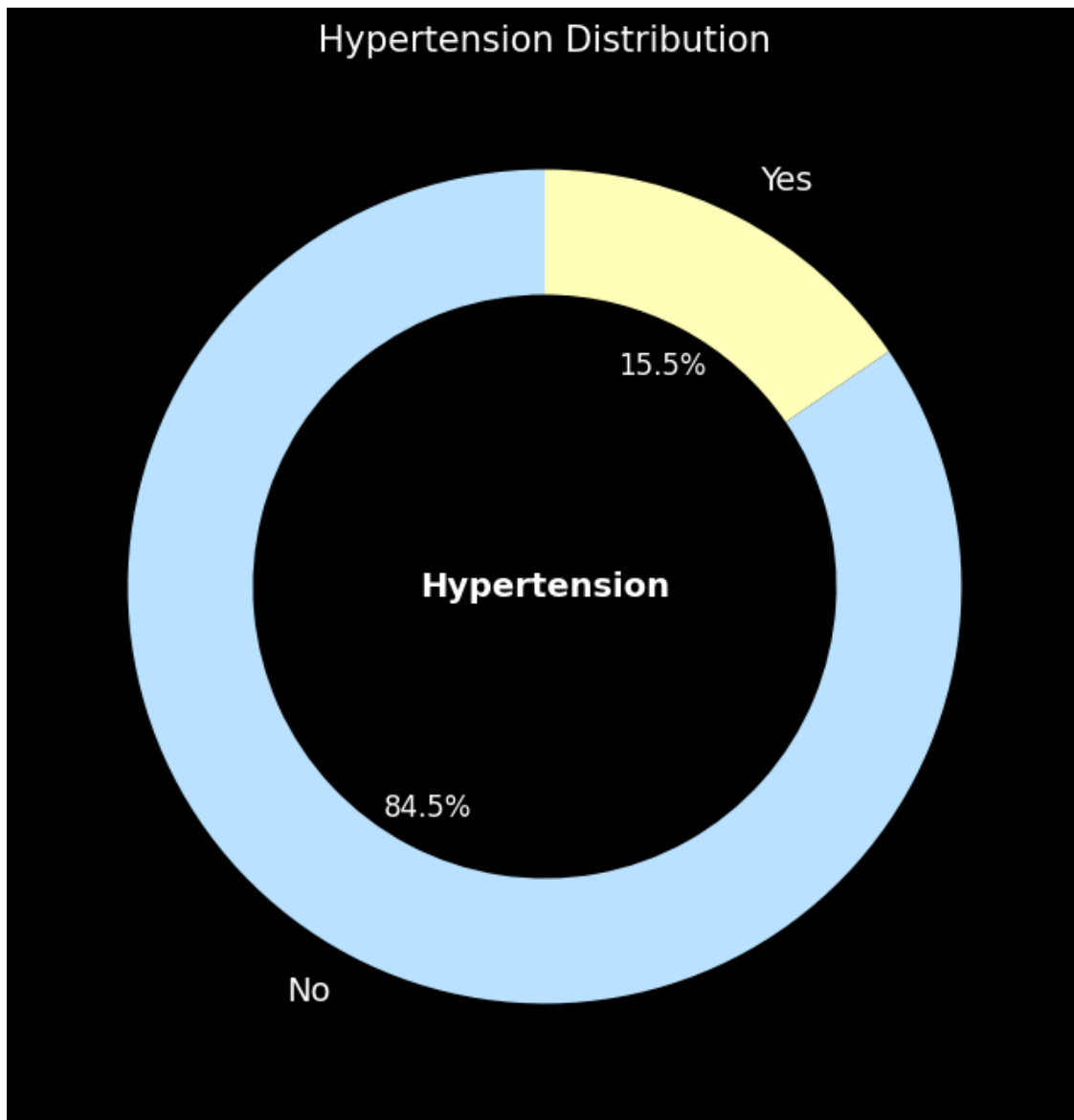
```
[39]:          count
Hypertension
0           1437
1           263
```

```
[40]: colors = ['#BAE1FF', '#FFFFBA']
plt.figure(figsize=(8,8))
wedges, texts, autotexts = plt.pie(
    data.Hypertension.value_counts(), labels=data.Hypertension.value_counts().
    index.map({0: 'No', 1: 'Yes'}), autopct='%1.1f%%', startangle=90,
    wedgeprops={'width': 0.3}, colors=colors
```

```

)
plt.text(0, 0, 'Hypertension', ha='center', va='center', fontsize=14,
        fontweight='bold')
plt.title("Hypertension Distribution", fontsize=15)
plt.setp(autotexts, size=12, color="white")
plt.setp(texts, size=14, color="white")
plt.show()

```



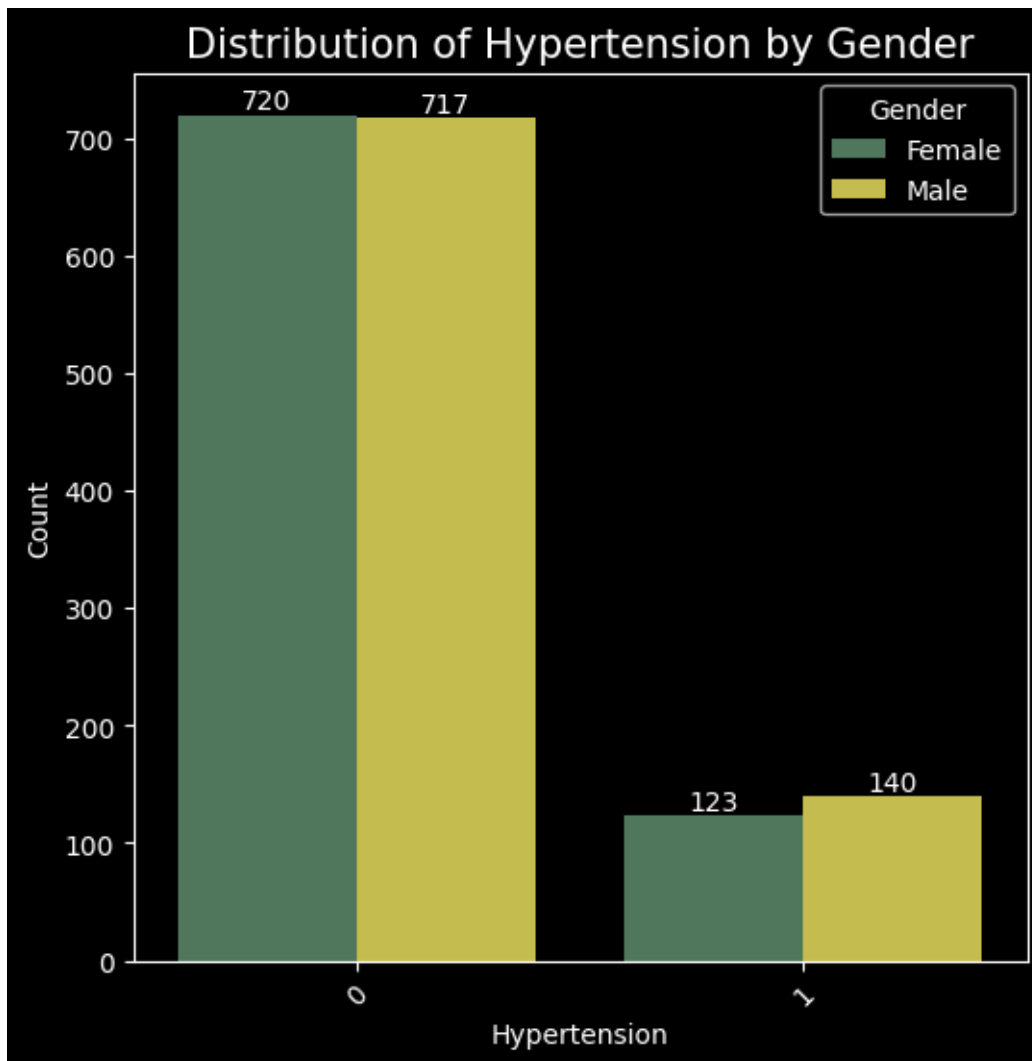
```

[41]: ### Hypertension VS Gender
data.groupby(['Gender', 'Hypertension'])['Hypertension'].count()

```

```
[41]: Gender  Hypertension
      0      0      720
      1      1      123
      1      0      717
      1      1      140
      Name: Hypertension, dtype: int64
```

```
[42]: data2 = data.copy()
data2.Gender = data2.Gender.map({0: 'Female', 1: 'Male'})
data2.Diabetes = data2.Hypertension.map({0: 'No', 1: 'Yes'})
grouped_data = data2.groupby(['Hypertension', 'Gender'])['Hypertension'].
    ↪count().reset_index(name='Count')
colors = ['#4A7C59', '#D9CC3C']
plt.figure(figsize=(6, 6))
ax = sns.barplot(data=grouped_data, x='Hypertension', y='Count',
    ↪hue='Gender', palette=colors)
for container in ax.containers:
    ax.bar_label(container)
plt.title("Distribution of Hypertension by Gender", fontsize=15)
plt.xlabel("Hypertension")
plt.ylabel("Count")
plt.legend(title="Gender")
plt.xticks(rotation=45)
plt.show()
```

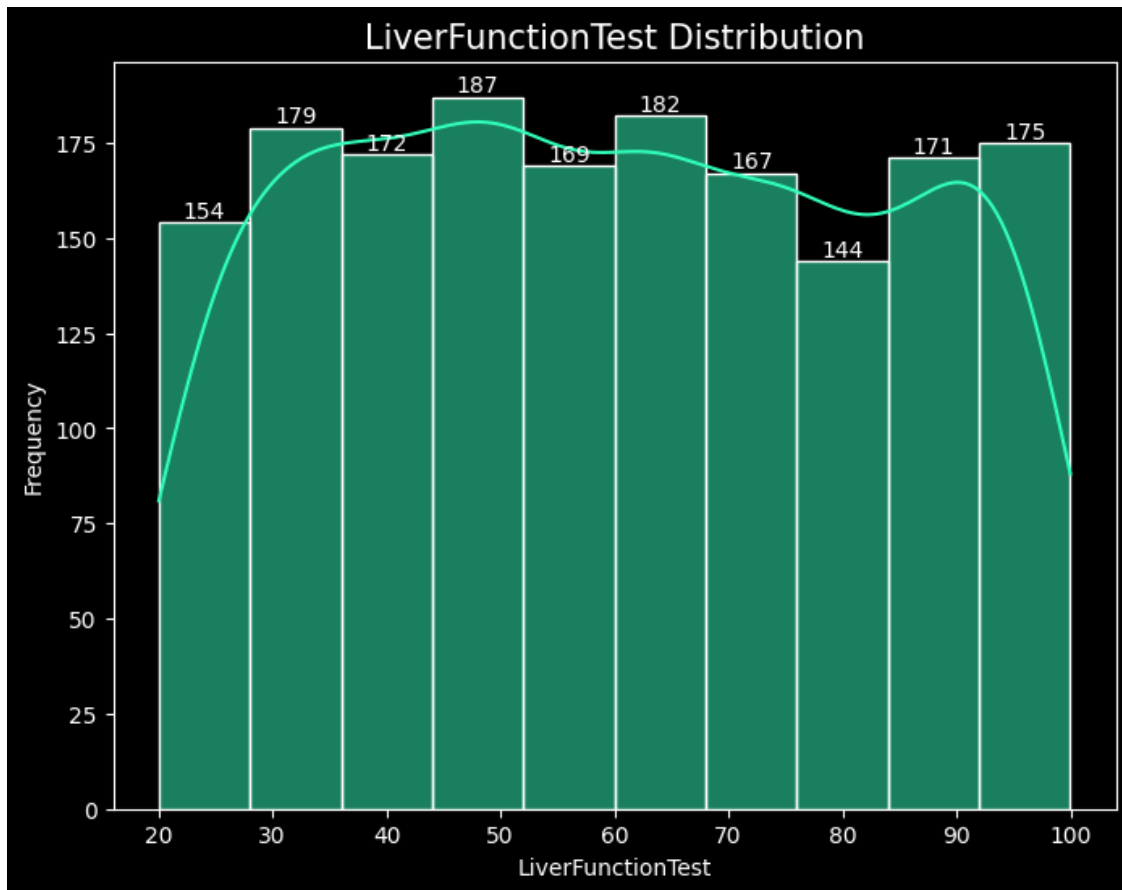


```
[43]: ### LiverFunctionTest Distribution
min_liver = data['LiverFunctionTest'].min()
max_liver = data['LiverFunctionTest'].max()
# Print the results
print(f"Minimum LiverFunctionTest: {min_liver}")
print(f"Maximum LiverFunctionTest: {max_liver}")
```

Minimum LiverFunctionTest: 20.01925357943218
Maximum LiverFunctionTest: 99.99141333640394

```
[44]: plt.figure(figsize=(8, 6))
ax = sns.histplot(data['LiverFunctionTest'], bins=10, kde=True, color='#33FFBD')
for container in ax.containers:
    ax.bar_label(container)
plt.title('LiverFunctionTest Distribution', fontsize=15)
```

```
plt.xlabel('LiverFunctionTest')
plt.ylabel('Frequency')
plt.show()
```

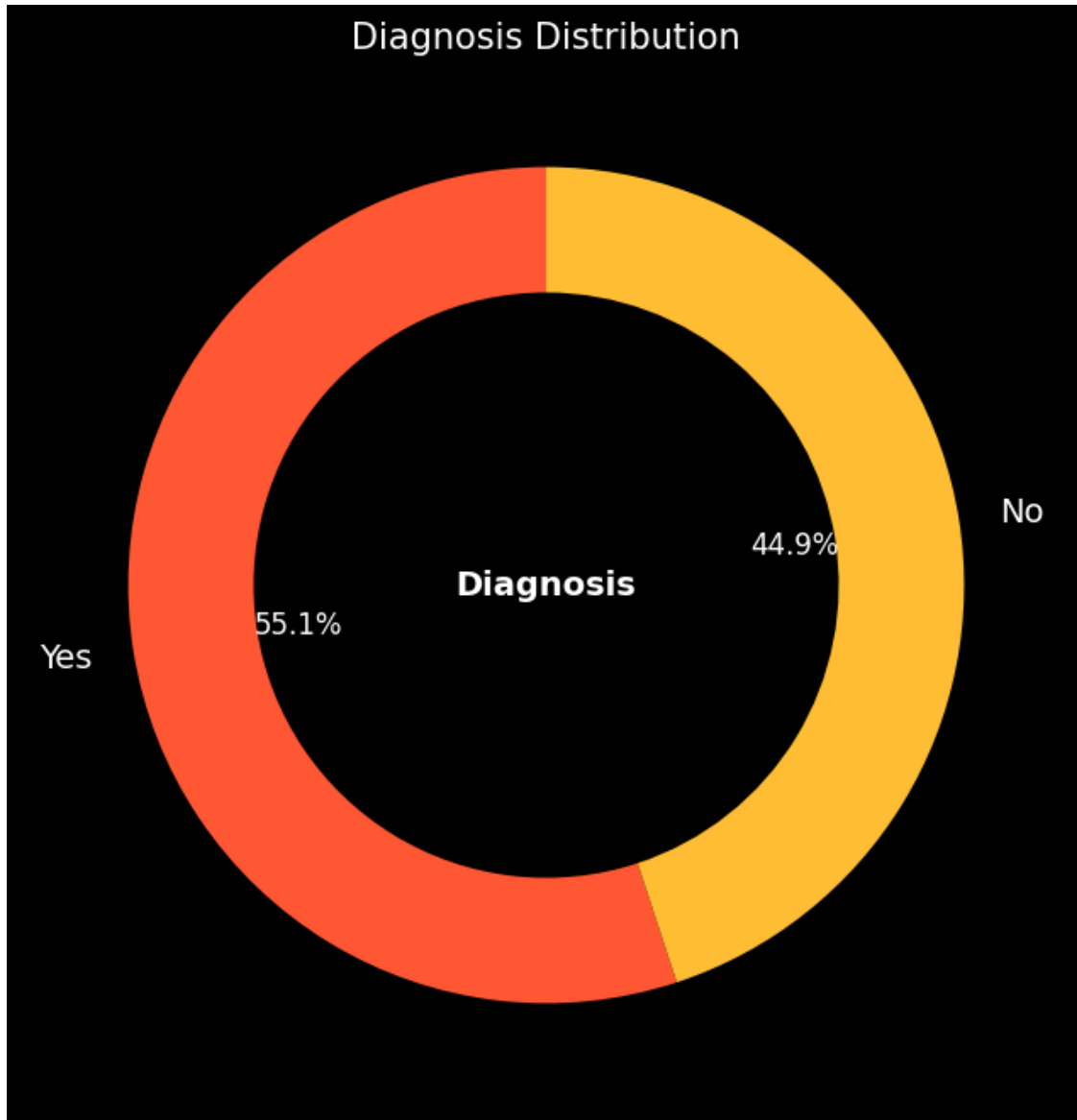


```
[45]: ### Diagnosis Distribution
data.Diagnosis.value_counts().to_frame()
```

```
[45]:      count
Diagnosis
1         936
0         764
```

```
[46]: colors = ['#FF5733', '#FFBD33']
plt.figure(figsize=(8,8))
wedges, texts, autotexts = plt.pie(
    data.Diagnosis.value_counts(), labels=data.Diagnosis.value_counts().index.
    map({0: 'No', 1: 'Yes'}), autopct='%1.1f%%', startangle=90,
    wedgeprops={'width': 0.3}, colors=colors
)
```

```
plt.text(0, 0, 'Diagnosis', ha='center', va='center', fontsize=14,
        fontweight='bold')
plt.title("Diagnosis Distribution", fontsize=15)
plt.setp(autotexts, size=12, color="white")
plt.setp(texts, size=14, color="white")
plt.show()
```



Graph Show Dataset Imbalanced

```
[47]: ### Show Columns VS Diagnosis
columns = data.select_dtypes(include=['category', 'int']).columns
columns = list(columns[1:])
```

```
columns.pop(-2)
columns
```

```
[47]: ['Gender', 'Smoking', 'GeneticRisk', 'Diabetes', 'Hypertension', 'SegmentBMI']
```

```
[48]: for col in columns:
        print(f'{col} VS Diagnosis\n')
        print(data.groupby(['Diagnosis', col])[col].count())
        print('\n*****\n')
```

Gender VS Diagnosis

Diagnosis	Gender	
0	0	459
	1	305
1	0	384
	1	552

Name: Gender, dtype: int64

Smoking VS Diagnosis

Diagnosis	Smoking	
0	0	618
	1	146
1	0	586
	1	350

Name: Smoking, dtype: int64

GeneticRisk VS Diagnosis

Diagnosis	GeneticRisk	
0	0	459
	1	278
	2	27
1	0	519
	1	279
	2	138

Name: GeneticRisk, dtype: int64

Diabetes VS Diagnosis

Diagnosis Diabetes

0	0	687
	1	77
1	0	771
	1	165

Name: Diabetes, dtype: int64

Hypertension VS Diagnosis

Diagnosis	Hypertension	
0	0	698
	1	66
1	0	739
	1	197

Name: Hypertension, dtype: int64

SegmentBMI VS Diagnosis

Diagnosis	SegmentBMI	
0	Underweight	146
	Normal weight	215
	Overweight	145
	Obese	127
	Extremly Obese	131
1	Underweight	97
	Normal weight	185
	Overweight	215
	Obese	220
	Extremly Obese	219

Name: SegmentBMI, dtype: int64

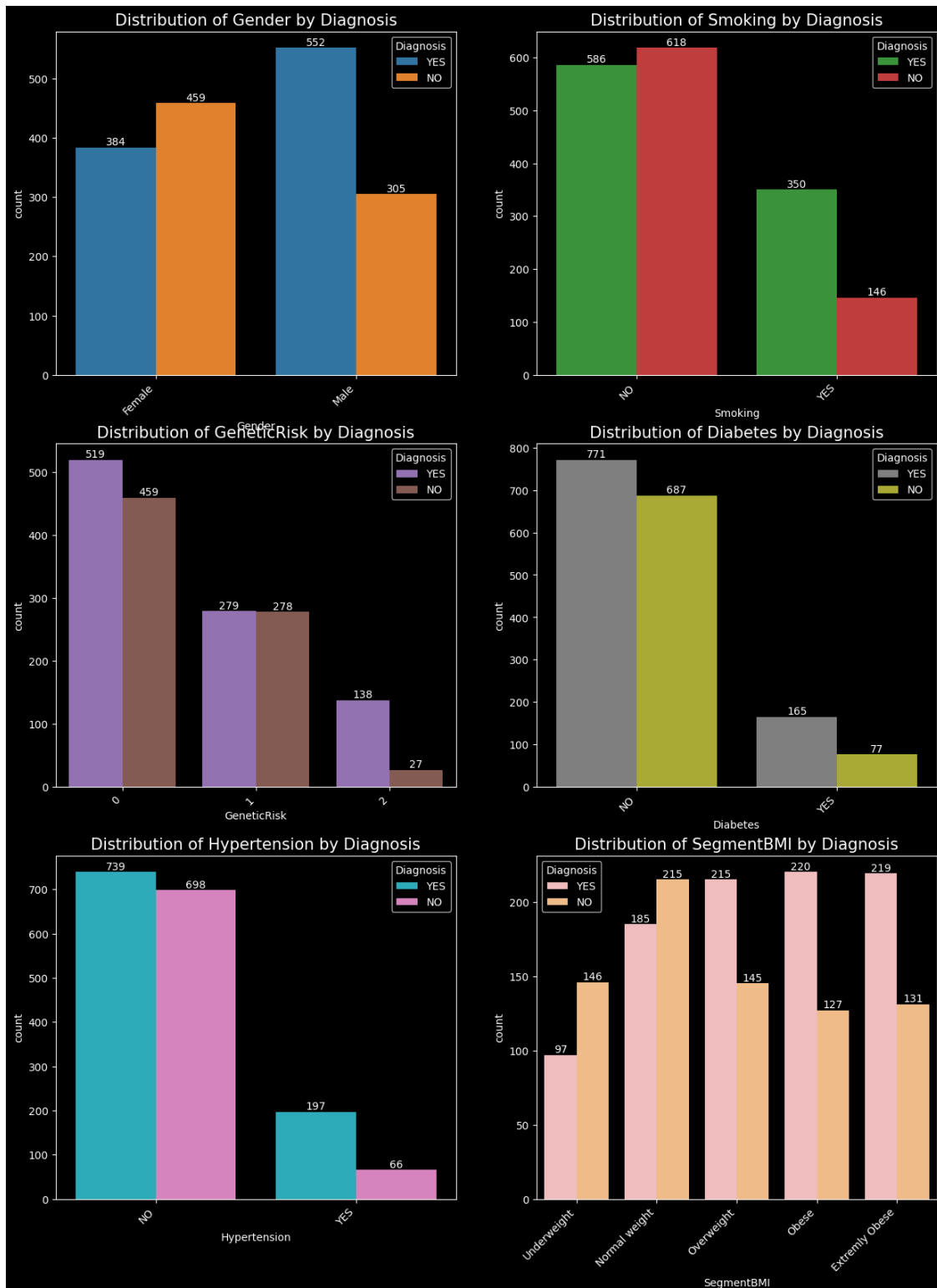
```
[49]: data1 = data[columns].copy()
data1['Gender'] = data1['Gender'].map({0: 'Female', 1: 'Male'})
data1['Smoking'] = data1['Smoking'].map({0: 'NO', 1: 'YES'})
data1['Diabetes'] = data1['Diabetes'].map({0: 'NO', 1: 'YES'})
data1['Hypertension'] = data1['Hypertension'].map({0: 'NO', 1: 'YES'})
fig, axes = plt.subplots(nrows=3, ncols=2, figsize=(15, 20))
axes = axes.flatten()
colors = [
    ['#1f77b4', '#ff7f0e'],
    ['#2ca02c', '#d62728'],
```



```

    ['#9467bd', '#8c564b'],
    ['#7f7f7f', '#bcbd22'],
    ['#17becf', '#e377c2'],
    ['#f7b5b5', '#ffbb78']
]
for i, col in enumerate(columns):
    ax = sns.countplot(x=data1[col], hue=data['Diagnosis'].map({0: 'NO', 1: 'YES'}), ax=axes[i], palette=colors[i])
    axes[i].set_title(f'Distribution of {col} by Diagnosis', fontsize=15)
    for container in ax.containers:
        ax.bar_label(container)
    ax.set_xticklabels(ax.get_xticklabels(), rotation=45, horizontalalignment='right')

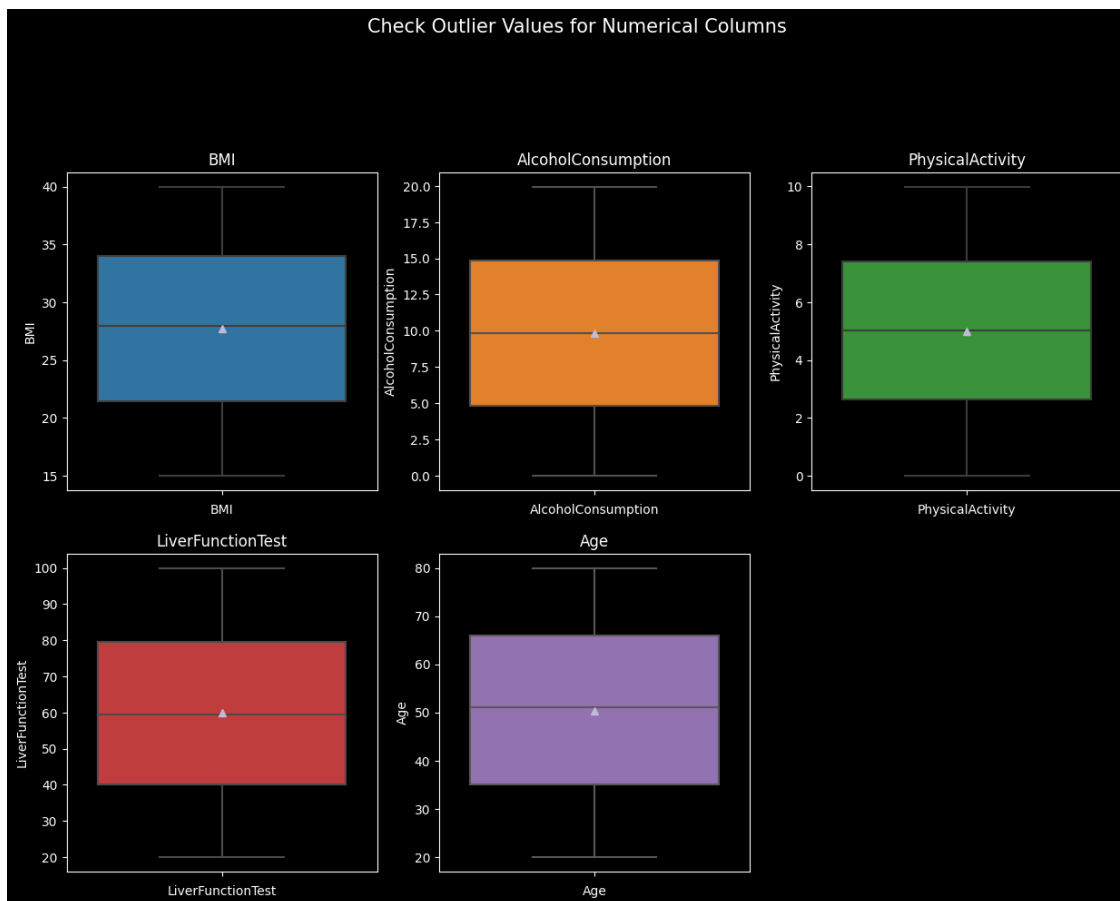
```



```
[50]: ### Check Outliers Values for Numerical Columns
columns = list(data.select_dtypes(include='float64').columns)
columns.append('Age')
columns
```

```
[50]: ['BMI', 'AlcoholConsumption', 'PhysicalActivity', 'LiverFunctionTest', 'Age']
```

```
[51]: plt.figure(figsize=(15, 10))
colors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd', '#8c564b']
for i, column in enumerate(columns):
    plt.subplot(2, 3, i + 1)
    sns.boxplot(y=data[column], color=colors[i], showmeans=True)
    plt.title(column)
    plt.xlabel(column)
plt.suptitle("Check Outlier Values for Numerical Columns", fontsize=15, y=1.05)
plt.show()
```



Preprocessing

Tabel of Contents

```
[52]: columns = data.select_dtypes(include=['category','object']).columns
columns
```

```
[52]: Index(['SegmentBMI'], dtype='object')
```

```
[53]: ### Transform Object Columns into Numbers
label = LabelEncoder()
data.SegmentBMI = label.fit_transform(data.SegmentBMI)
data.head()
```

```
[53]:
```

	Age	Gender	BMI	AlcoholConsumption	Smoking	GeneticRisk	\
0	58	0	35.857584	17.272828	0	1	
1	71	1	30.732470	2.201266	0	1	
2	48	0	19.971407	18.500944	0	0	
3	34	1	16.615417	12.632870	0	0	
4	62	1	16.065830	1.087815	0	1	

	PhysicalActivity	Diabetes	Hypertension	LiverFunctionTest	Diagnosis	\
0	0.658940	0	0	42.734240	1	
1	1.670557	1	0	67.309822	1	
2	9.928308	0	0	63.738956	0	
3	5.630129	0	0	64.555873	1	
4	3.566218	1	0	77.868689	1	

	SegmentBMI
0	0
1	2
2	1
3	4
4	4

```
[54]: ### Show Correlation
corr = data.corr()
corr
```

```
[54]:
```

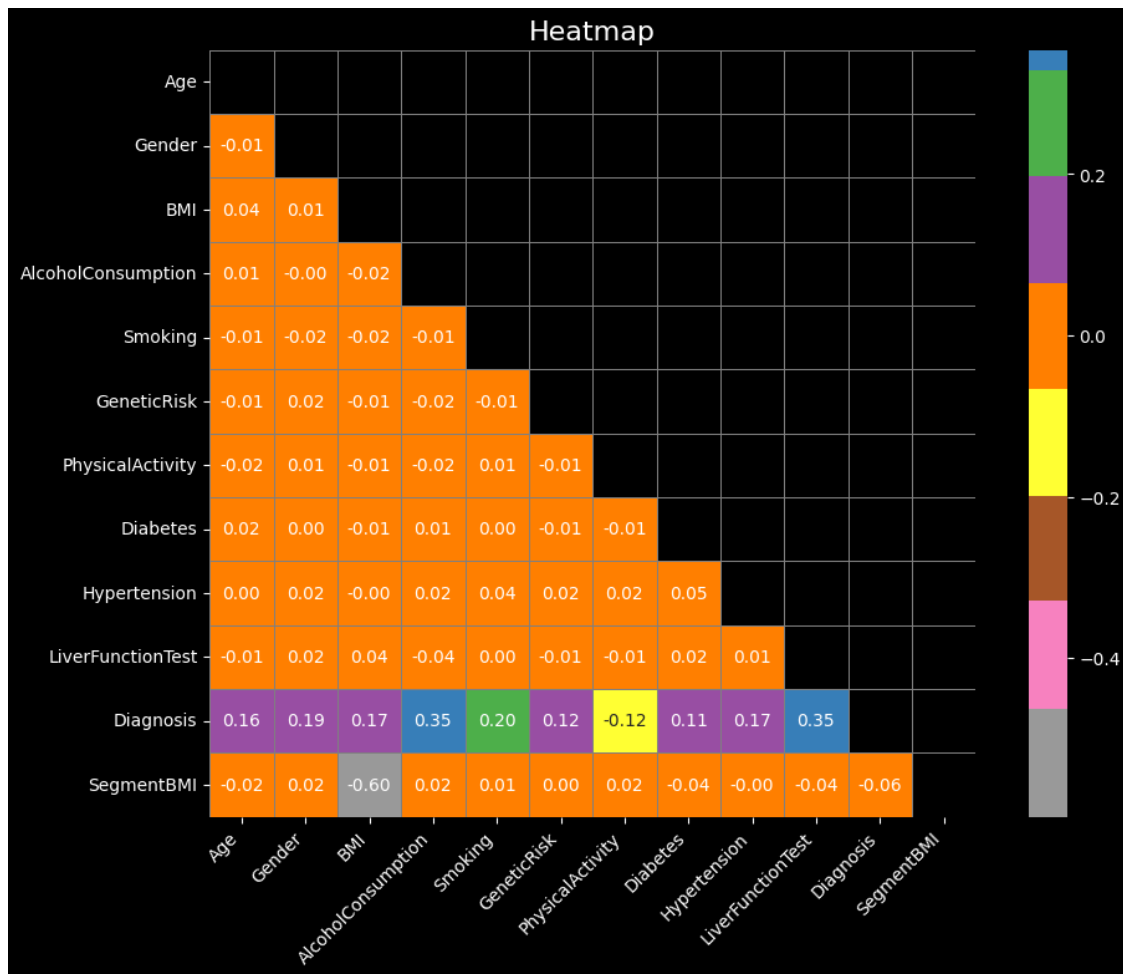
	Age	Gender	BMI	AlcoholConsumption	\
Age	1.000000	-0.009323	0.038283	0.008236	
Gender	-0.009323	1.000000	0.013458	-0.003312	
BMI	0.038283	0.013458	1.000000	-0.024749	
AlcoholConsumption	0.008236	-0.003312	-0.024749	1.000000	
Smoking	-0.013683	-0.023403	-0.018322	-0.013794	
GeneticRisk	-0.012948	0.017394	-0.007686	-0.016808	
PhysicalActivity	-0.015212	0.009379	-0.010888	-0.024232	
Diabetes	0.024209	0.000012	-0.007808	0.013490	
Hypertension	0.000678	0.024131	-0.004850	0.016640	
LiverFunctionTest	-0.013105	0.024603	0.043684	-0.043480	
Diagnosis	0.156099	0.189558	0.167655	0.349610	

SegmentBMI	-0.023162	0.019226	-0.596641	0.015214
------------	-----------	----------	-----------	----------

	Smoking	GeneticRisk	PhysicalActivity	Diabetes	\
Age	-0.013683	-0.012948	-0.015212	0.024209	
Gender	-0.023403	0.017394	0.009379	0.000012	
BMI	-0.018322	-0.007686	-0.010888	-0.007808	
AlcoholConsumption	-0.013794	-0.016808	-0.024232	0.013490	
Smoking	1.000000	-0.007374	0.007054	0.001455	
GeneticRisk	-0.007374	1.000000	-0.011594	-0.010785	
PhysicalActivity	0.007054	-0.011594	1.000000	-0.006939	
Diabetes	0.001455	-0.010785	-0.006939	1.000000	
Hypertension	0.040314	0.018990	0.018569	0.049166	
LiverFunctionTest	0.000838	-0.008061	-0.010393	0.018153	
Diagnosis	0.200071	0.118292	-0.116689	0.107480	
SegmentBMI	0.005859	0.000346	0.021407	-0.043479	

	Hypertension	LiverFunctionTest	Diagnosis	SegmentBMI
Age	0.000678	-0.013105	0.156099	-0.023162
Gender	0.024131	0.024603	0.189558	0.019226
BMI	-0.004850	0.043684	0.167655	-0.596641
AlcoholConsumption	0.016640	-0.043480	0.349610	0.015214
Smoking	0.040314	0.000838	0.200071	0.005859
GeneticRisk	0.018990	-0.008061	0.118292	0.000346
PhysicalActivity	0.018569	-0.010393	-0.116689	0.021407
Diabetes	0.049166	0.018153	0.107480	-0.043479
Hypertension	1.000000	0.007847	0.170683	-0.004467
LiverFunctionTest	0.007847	1.000000	0.354003	-0.043639
Diagnosis	0.170683	0.354003	1.000000	-0.064997
SegmentBMI	-0.004467	-0.043639	-0.064997	1.000000

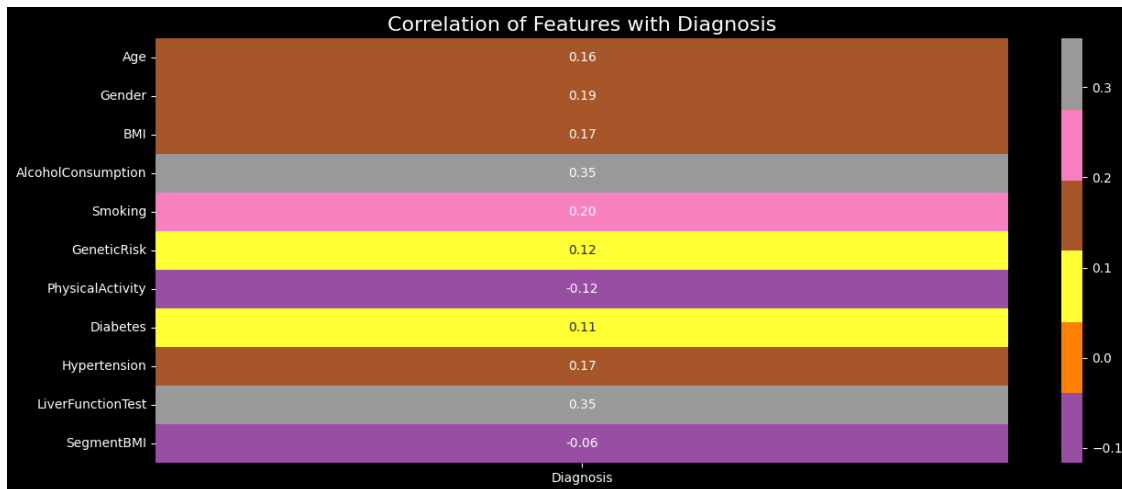
```
[55]: mask = np.triu(np.ones_like(corr, dtype=bool))
plt.figure(figsize=(11, 8))
sns.heatmap(corr, mask=mask, annot=True, fmt=".2f", cmap='Set1_r',
            cbar=True, square=True, linewidths=0.5,
            linecolor='gray', center=0)
plt.title('Heatmap', fontsize=16)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.show()
```



```
[56]: diag_corr = corr['Diagnosis'].drop('Diagnosis')
      diag_corr
```

```
[56]: Age          0.156099
      Gender       0.189558
      BMI          0.167655
      AlcoholConsumption  0.349610
      Smoking      0.200071
      GeneticRisk   0.118292
      PhysicalActivity -0.116689
      Diabetes      0.107480
      Hypertension  0.170683
      LiverFunctionTest 0.354003
      SegmentBMI    -0.064997
      Name: Diagnosis, dtype: float64
```

```
[57]: plt.figure(figsize=(15, 6))
diag_corr_df = pd.DataFrame(diag_corr)
sns.heatmap(diag_corr_df, annot=True, fmt=".2f", cmap='Set1',
            linecolor='gray', center=0)
plt.title('Correlation of Features with Diagnosis', fontsize=16)
plt.show()
```



```
[58]: ### Split
X = data.drop(['Diagnosis', 'SegmentBMI'], axis=1)
y = data['Diagnosis']
keys = X.columns
X.head()
```

```
[58]:
```

	Age	Gender	BMI	AlcoholConsumption	Smoking	GeneticRisk	\
0	58	0	35.857584	17.272828	0	1	
1	71	1	30.732470	2.201266	0	1	
2	48	0	19.971407	18.500944	0	0	
3	34	1	16.615417	12.632870	0	0	
4	62	1	16.065830	1.087815	0	1	

	PhysicalActivity	Diabetes	Hypertension	LiverFunctionTest
0	0.658940	0	0	42.734240
1	1.670557	1	0	67.309822
2	9.928308	0	0	63.738956
3	5.630129	0	0	64.555873
4	3.566218	1	0	77.868689

```
[59]: y.head()
```

```
[59]: 0    1
      1    1
      2    0
      3    1
      4    1
      Name: Diagnosis, dtype: int64
```

```
[60]: ### Scaling
scale = MinMaxScaler()
X = scale.fit_transform(X)
X = pd.DataFrame(X, columns=keys)
X.head()
```

```
[60]:      Age  Gender      BMI  AlcoholConsumption  Smoking  GeneticRisk  \
0  0.633333    0.0  0.834511          0.865674        0.0          0.5
1  0.850000    1.0  0.629409          0.110159        0.0          0.5
2  0.466667    0.0  0.198762          0.927238        0.0          0.0
3  0.233333    1.0  0.064459          0.633080        0.0          0.0
4  0.700000    1.0  0.042465          0.054344        0.0          0.5

      PhysicalActivity  Diabetes  Hypertension  LiverFunctionTest
0          0.065754        0.0          0.0          0.284036
1          0.166986        1.0          0.0          0.591338
2          0.993330        0.0          0.0          0.546687
3          0.563216        0.0          0.0          0.556902
4          0.356682        1.0          0.0          0.723370
```

```
[61]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20,
↳ random_state=42)
print('X_train shape is ', X_train.shape)
print('X_test shape is ', X_test.shape)
print('y_train shape is ', y_train.shape)
print('y_test shape is ', y_test.shape)
```

```
X_train shape is (1360, 10)
X_test shape is (340, 10)
y_train shape is (1360,)
y_test shape is (340,)
```

ML Models

Tabel of Contents

```
[62]: models = {
      'RandomForest': RandomForestClassifier(random_state=42),
      'XGBoost': XGBClassifier(),
      'LGBM': LGBMClassifier(verbose=-1),
      'DecisionTree': DecisionTreeClassifier(random_state=42),
      'KNN': KNeighborsClassifier(),
```



```

    'SVC': SVC(probability=True, random_state=42, class_weight='balanced'),
    'LogisticRegression': LogisticRegression(random_state=42,
    ↪class_weight='balanced'),
    'CatBoost': CatBoostClassifier(logging_level='Silent'),
    'GradientBoosting': GradientBoostingClassifier(random_state=42),
    'AdaBoost': AdaBoostClassifier(random_state=42),
    'NaiveBayes': GaussianNB()
}
### Validation Score
cv_results = {}
for model_name, model in models.items():
    print(f"Cross-validating {model_name}...")
    cv_scores = cross_val_score(model, X_train, y_train, cv=5,
    ↪scoring='accuracy', n_jobs=-1, verbose=0)
    cv_results[model_name] = cv_scores.mean()
    print(f"Mean accuracy for {model_name}: {cv_scores.mean():.4f}")

```

```

Cross-validating RandomForest...
Mean accuracy for RandomForest: 0.8831
Cross-validating XGBoost...
Mean accuracy for XGBoost: 0.8904
Cross-validating LGBM...
Mean accuracy for LGBM: 0.8868
Cross-validating DecisionTree...
Mean accuracy for DecisionTree: 0.8279
Cross-validating KNN...
Mean accuracy for KNN: 0.7941
Cross-validating SVC...
Mean accuracy for SVC: 0.8360
Cross-validating LogisticRegression...
Mean accuracy for LogisticRegression: 0.8221
Cross-validating CatBoost...
Mean accuracy for CatBoost: 0.9140
Cross-validating GradientBoosting...
Mean accuracy for GradientBoosting: 0.8949
Cross-validating AdaBoost...
Mean accuracy for AdaBoost: 0.8904
Cross-validating NaiveBayes...
Mean accuracy for NaiveBayes: 0.8015

```

```

[63]: cv_results_df = pd.DataFrame(list(cv_results.items()), columns=['Model', 'Mean_
    ↪Accuracy'])
cv_results_df = cv_results_df.sort_values(by='Mean Accuracy', ascending=False)
cv_results_df

```

```

[63]:
      Model  Mean Accuracy
7      CatBoost      0.913971

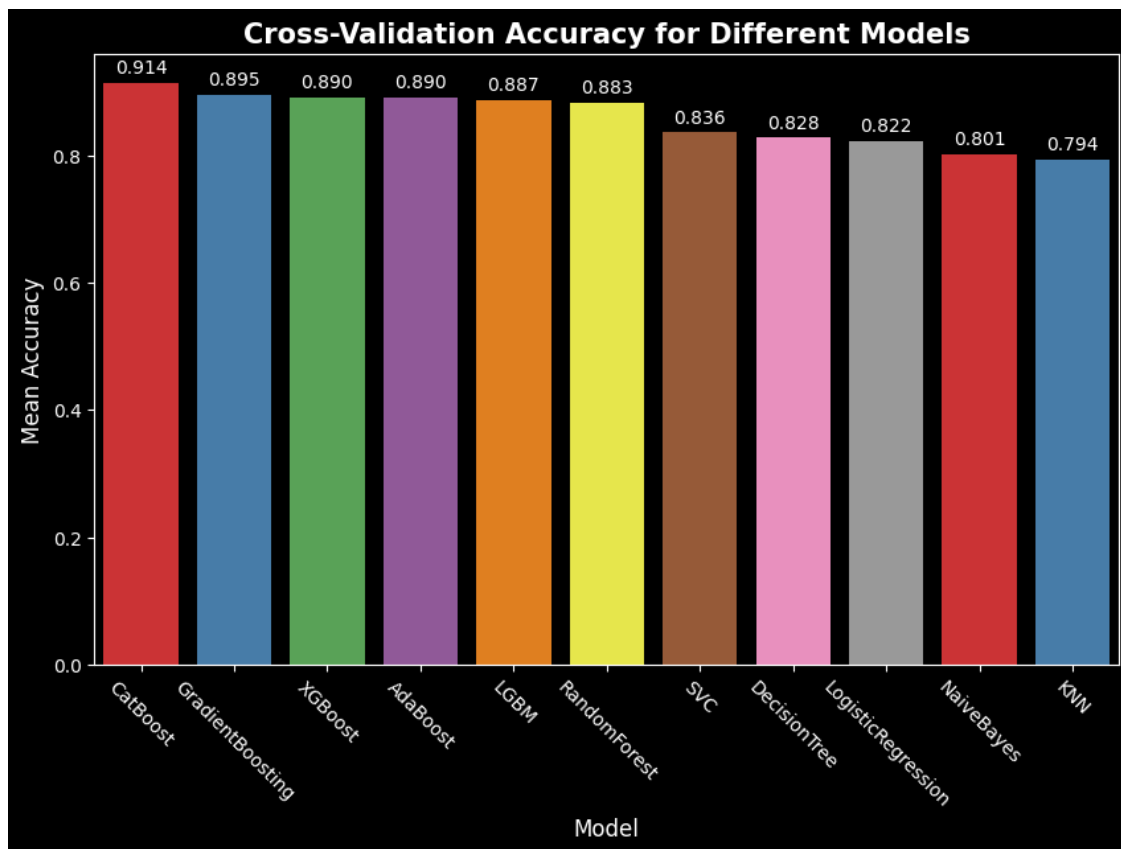
```

8	GradientBoosting	0.894853
1	XGBoost	0.890441
9	AdaBoost	0.890441
2	LGBM	0.886765
0	RandomForest	0.883088
5	SVC	0.836029
3	DecisionTree	0.827941
6	LogisticRegression	0.822059
10	NaiveBayes	0.801471
4	KNN	0.794118

```
[64]: best_model_name = cv_results_df.iloc[0]['Model']
      print(f"\nBest model from CV: {best_model_name}")
```

Best model from CV: CatBoost

```
[65]: plt.figure(figsize=(10, 6))
      ax = sns.barplot(data=cv_results_df, x='Model', y='Mean Accuracy',
      ↪palette='Set1')
      ax.set_title('Cross-Validation Accuracy for Different Models', fontsize=15,
      ↪weight='bold', loc='center')
      ax.set_xlabel('Model', fontsize=12)
      ax.set_ylabel('Mean Accuracy', fontsize=12)
      plt.xticks(rotation=-45)
      for container in ax.containers:
          ax.bar_label(container, labels=[f'{v.get_height():.3f}' for v in
          ↪container], padding=3)
      plt.show()
```



```
[66]: param_grids = {
    'RandomForest': {
        'n_estimators': [50, 100, 200],
        'max_depth': [None, 10, 20, 30, 40],
        'min_samples_split': [2, 5, 10]
    },
    'XGBoost': {
        'max_depth': [3, 5, 7, 10, 20, 30, 40],
        'n_estimators': [50, 100, 200],
        'subsample': [0.6, 0.8, 1],
        'learning_rate': [0.05, 0.1, 0.5, 1]
    },
    'LGBM': {
        'n_estimators': [50, 100, 200],
        'max_depth': [None, 5, 10, 20, 30, 40],
        'learning_rate': [0.05, 0.1, 0.5, 1]
    },
    'DecisionTree': {
        'max_depth': [None, 10, 20, 30, 40],
        'min_samples_split': [2, 5, 10]
    }
}
```

```

},
'KNN': {
    'n_neighbors': [3, 5, 7],
    'weights': ['uniform', 'distance']
},
'SVC': {
    'C': [0.1, 1, 10],
    'kernel': ['linear', 'rbf', 'poly'],
    'gamma': ['scale', 'auto']
},
'LogisticRegression': {
    'C': [0.01, 0.1, 1, 10],
    'penalty': ['l1', 'l2'],
    'solver': ['liblinear', 'saga']
},
'CatBoost': {
    'iterations': [100, 200],
    'learning_rate': [0.05, 0.1, .5, 1],
    'depth': [6, 10, 15, 20, 25, 30, 40]
},
'GradientBoosting': {
    'n_estimators': [50, 100, 200],
    'learning_rate': [0.05, 0.1, .5, 1],
    'max_depth': [None, 10, 20, 30, 40]
},
'AdaBoost': {
    'n_estimators': [50, 100, 200],
    'learning_rate': [0.05, 0.1, .5, 1]
},
'NaiveBayes': {}
}

best_results = []
for model_name, model in models.items():
    print(f"\nPerforming GridSearchCV for {model_name}...")
    grid_search = GridSearchCV(estimator=model,
    ↪ param_grid=param_grids[model_name], cv=5, scoring='accuracy', verbose=0,
    ↪ n_jobs=-1)
    grid_search.fit(X_train, y_train)
    best_params = grid_search.best_params_
    best_score = grid_search.best_score_
    best_results.append({'Model': model_name, 'Best Score': best_score, 'Best_
    ↪ Params': best_params})
    print(f"\nBest parameters for {model_name}: {best_params}")
    print(f"Best cross-validation score for {model_name}: {best_score:.4f}")

```

Performing GridSearchCV for RandomForest...

Best parameters for RandomForest: {'max_depth': None, 'min_samples_split': 5, 'n_estimators': 200}

Best cross-validation score for RandomForest: 0.8860

Performing GridSearchCV for XGBoost...

Best parameters for XGBoost: {'learning_rate': 0.05, 'max_depth': 3, 'n_estimators': 200, 'subsample': 0.8}

Best cross-validation score for XGBoost: 0.9059

Performing GridSearchCV for LGBM...

Best parameters for LGBM: {'learning_rate': 0.05, 'max_depth': 5, 'n_estimators': 200}

Best cross-validation score for LGBM: 0.8956

Performing GridSearchCV for DecisionTree...

Best parameters for DecisionTree: {'max_depth': None, 'min_samples_split': 10}

Best cross-validation score for DecisionTree: 0.8382

Performing GridSearchCV for KNN...

Best parameters for KNN: {'n_neighbors': 7, 'weights': 'distance'}

Best cross-validation score for KNN: 0.8000

Performing GridSearchCV for SVC...

Best parameters for SVC: {'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}

Best cross-validation score for SVC: 0.8360

Performing GridSearchCV for LogisticRegression...

Best parameters for LogisticRegression: {'C': 1, 'penalty': 'l1', 'solver': 'liblinear'}

Best cross-validation score for LogisticRegression: 0.8228

Performing GridSearchCV for CatBoost...

Best parameters for CatBoost: {'depth': 10, 'iterations': 100, 'learning_rate': 0.1}

Best cross-validation score for CatBoost: 0.9235

Performing GridSearchCV for GradientBoosting...

Best parameters for GradientBoosting: {'learning_rate': 1, 'max_depth': 10,

```
'n_estimators': 50}
Best cross-validation score for GradientBoosting: 0.8846
```

Performing GridSearchCV for AdaBoost...

```
Best parameters for AdaBoost: {'learning_rate': 0.1, 'n_estimators': 200}
Best cross-validation score for AdaBoost: 0.9118
```

Performing GridSearchCV for NaiveBayes...

```
Best parameters for NaiveBayes: {}
Best cross-validation score for NaiveBayes: 0.8015
```

```
[67]: best_results_df = pd.DataFrame(best_results)
      best_results_df = best_results_df.sort_values(by='Best Score', ascending=False)
      best_results_df
```

```
[67]:
```

	Model	Best Score \
7	CatBoost	0.923529
9	AdaBoost	0.911765
1	XGBoost	0.905882
2	LGBM	0.895588
0	RandomForest	0.886029
8	GradientBoosting	0.884559
3	DecisionTree	0.838235
5	SVC	0.836029
6	LogisticRegression	0.822794
10	NaiveBayes	0.801471
4	KNN	0.800000

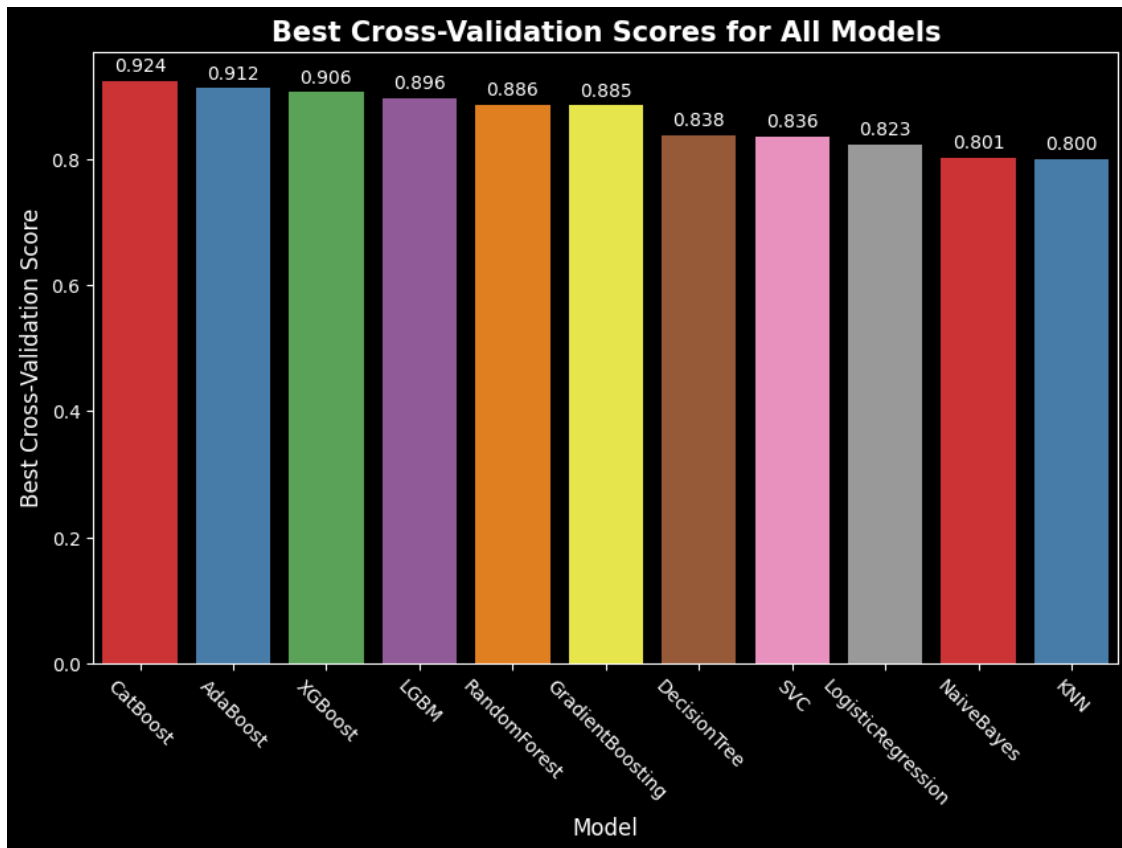
	Best Params
7	{'depth': 10, 'iterations': 100, 'learning_rat...
9	{'learning_rate': 0.1, 'n_estimators': 200}
1	{'learning_rate': 0.05, 'max_depth': 3, 'n_est...
2	{'learning_rate': 0.05, 'max_depth': 5, 'n_est...
0	{'max_depth': None, 'min_samples_split': 5, 'n...
8	{'learning_rate': 1, 'max_depth': 10, 'n_estim...
3	{'max_depth': None, 'min_samples_split': 10}
5	{'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}
6	{'C': 1, 'penalty': 'l1', 'solver': 'liblinear'}
10	{}
4	{'n_neighbors': 7, 'weights': 'distance'}

```
[68]: best_model_name, best_model_param = best_results_df.
      ↪iloc[0]['Model'], best_results_df.iloc[0]['Best Params']
      print(f"\nBest model : {best_model_name}")
      print(f"\nBest Parameters : {best_model_param}")
```

Best model : CatBoost

Best Parameters : {'depth': 10, 'iterations': 100, 'learning_rate': 0.1}

```
[69]: plt.figure(figsize=(10, 6))
      ax = sns.barplot(data=best_results_df, x='Model', y='Best Score',
      ↪palette='Set1')
      ax.set_title('Best Cross-Validation Scores for All Models', fontsize=15,
      ↪weight='bold', loc='center')
      ax.set_xlabel('Model', fontsize=12)
      ax.set_ylabel('Best Cross-Validation Score', fontsize=12)
      for container in ax.containers:
          ax.bar_label(container, labels=[f'{v.get_height():.3f}' for v in
          ↪container], padding=3)
      plt.xticks(rotation=-45)
      plt.show()
```



```
[70]: def plot_feature_importance(model, model_name, X_train, y_train, feature_names):
      """
```

Plot feature importance for a given model using colors from Set2 palette and display bar values. Rotate feature names on the y-axis for better readability.

```
Parameters:
    model: Trained model
    model_name: Name of the model as a string
    X_train: Training features
    y_train: Training labels
    feature_names: List of feature names (optional)
"""
# Determine feature importance method
if hasattr(model, "feature_importances_"):
    # Tree-based models
    importances = model.feature_importances_
elif model_name in ['LogisticRegression', 'SVC'] and hasattr(model,
↳"coef_"):
    # Linear models
    importances = np.abs(model.coef_[0])
else:
    # General models using permutation importance
    result = permutation_importance(model, X_train, y_train, n_repeats=10,
↳random_state=42)
    importances = result.importances_mean

# Sort importance and indices (reverse for largest on top)
sorted_idx = importances.argsort()[::-1] # Reverse sorting
sorted_importances = importances[sorted_idx]
sorted_features = [feature_names[i] for i in sorted_idx] if feature_names
↳else sorted_idx

# Plot feature importance using Seaborn barh
plt.figure(figsize=(10, 6))
ax = sns.barplot(y=sorted_features, x=sorted_importances, palette="Set2",
↳orient="h")

# Set axis labels and title with custom font sizes
ax.set_xlabel("Feature Importance", fontsize=12)
ax.set_title(f"Feature Importance for {model_name}", fontsize=15)

# Add bar values using ax.bar_label for each container (specific to
↳horizontal bars)
for container in ax.containers:
    labels = [f'{v.get_width():.3f}' for v in container]
    ax.bar_label(container, labels=labels, padding=3)
```



```

# Adjust y-axis labels
ax.set_yticklabels(ax.get_yticklabels(), rotation=0, ha="right",
↳ fontsize=10)

# Increase spacing between ticks
plt.subplots_adjust(left=0.3)

plt.tight_layout()
plt.show()

```

```

[71]: ##### Training Models With Best Parameters
metrics_dict = {}
# Function to train and evaluate the model
def train_and_evaluate_model(model, model_name, X_train, y_train, X_test,
↳ y_test, feature_names=keys.tolist()):
    # Train the model
    model.fit(X_train, y_train)
    # Predictions for training and test sets
    y_train_pred = model.predict(X_train)
    y_test_pred = model.predict(X_test)
    # Training metrics
    train_accuracy = accuracy_score(y_train, y_train_pred)
    train_precision = precision_score(y_train, y_train_pred, zero_division=1)
    train_recall = recall_score(y_train, y_train_pred, zero_division=1)
    train_f1 = f1_score(y_train, y_train_pred, zero_division=1)
    # Test metrics
    test_accuracy = accuracy_score(y_test, y_test_pred)
    test_precision = precision_score(y_test, y_test_pred, zero_division=1)
    test_recall = recall_score(y_test, y_test_pred, zero_division=1)
    test_f1 = f1_score(y_test, y_test_pred, zero_division=1)
    # Print evaluation metrics
    print(f"{model_name} - Training Accuracy: {train_accuracy:.3f}")
    print(f"{model_name} - Test Accuracy: {test_accuracy:.3f}")
    print(f"{model_name} - Training Precision: {train_precision:.3f}")
    print(f"{model_name} - Test Precision: {test_precision:.3f}")
    print(f"{model_name} - Training Recall: {train_recall:.3f}")
    print(f"{model_name} - Test Recall: {test_recall:.3f}")
    print(f"{model_name} - Training F1 Score: {train_f1:.3f}")
    print(f"{model_name} - Test F1 Score: {test_f1:.3f}")
    # Plot ROC curve if available
    y_test_prob = model.predict_proba(X_test)[: , 1]
    fpr, tpr, _ = roc_curve(y_test, y_test_prob)
    roc_auc = roc_auc_score(y_test, y_test_prob)
    print(f'{model_name} - ROC AUC Score: {roc_auc:.3f}')
    plt.figure(figsize=(10, 6))
    plt.plot(fpr, tpr, color='blue', label=f'ROC Curve (AUC = {roc_auc:.3f})')
    plt.plot([0, 1], [0, 1], color='red', linestyle='--', label='Random Chance')

```

```

plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title(f'ROC Curve for {model_name}')
plt.legend(loc='lower right')
plt.show()
# Confusion matrix and classification report
cm = confusion_matrix(y_test, y_test_pred)
print(f'{model_name} - Confusion Matrix:\n', cm)
# Plot confusion matrix as a heatmap
plt.figure(figsize=(8, 5))
sns.heatmap(cm, annot=True, fmt='g', cmap="Blues", xticklabels=['NO', '
↪YES'], yticklabels=['NO', 'YES'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title(f'Confusion Matrix for {model_name}')
plt.show()
# Print the classification report and confusion matrix
classification_report_str = classification_report(y_test, y_test_pred,
↪target_names=['NO', 'YES'])
print(f'{model_name} - Classification Report:\n', classification_report_str)
# Store metrics in the dictionary
metrics_dict[model_name] = {
    'train_accuracy': train_accuracy,
    'train_precision': train_precision,
    'train_recall': train_recall,
    'train_f1': train_f1,
    'test_accuracy': test_accuracy,
    'test_precision': test_precision,
    'test_recall': test_recall,
    'test_f1': test_f1,
    'roc_auc': roc_auc
}
# Plot feature importance
plot_feature_importance(model, model_name, X_train, y_train, feature_names)
return model

```

```

[72]: ### RandomForest
random_forest_params = best_results_df.loc[best_results_df['Model'] ==
↪'RandomForest', 'Best Params'].values[0]
models['RandomForest'].set_params(**random_forest_params)
RandomForest = train_and_evaluate_model(models['RandomForest'], 'RandomForest',
↪X_train, y_train, X_test, y_test)

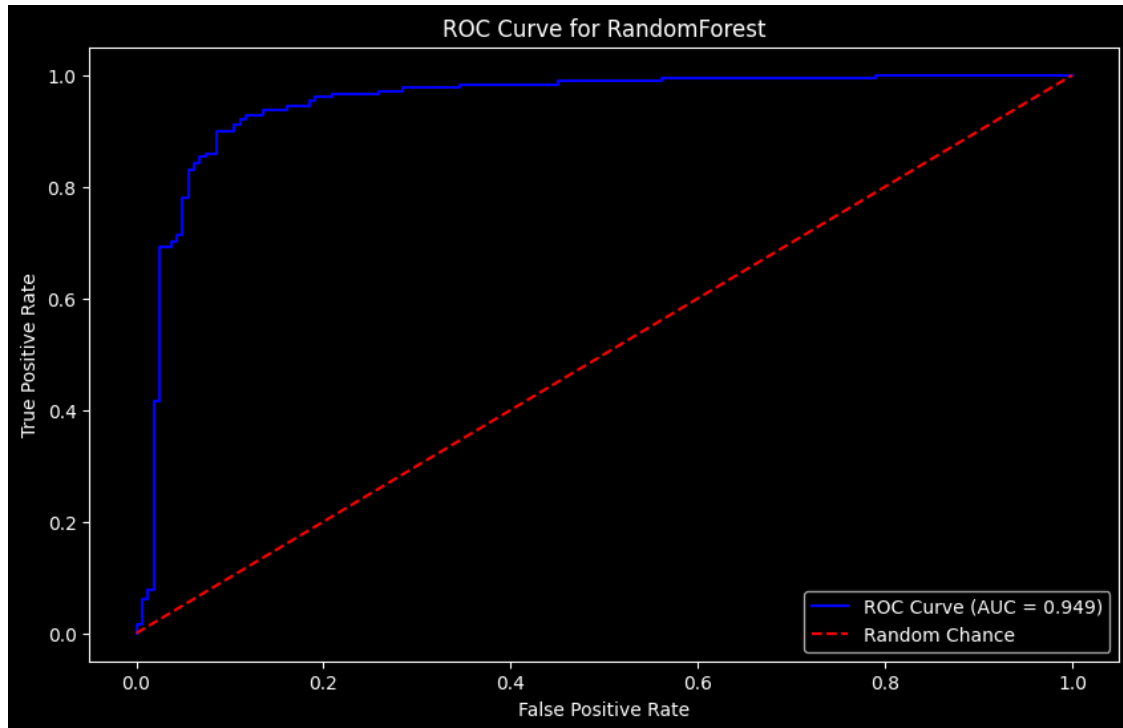
```

```

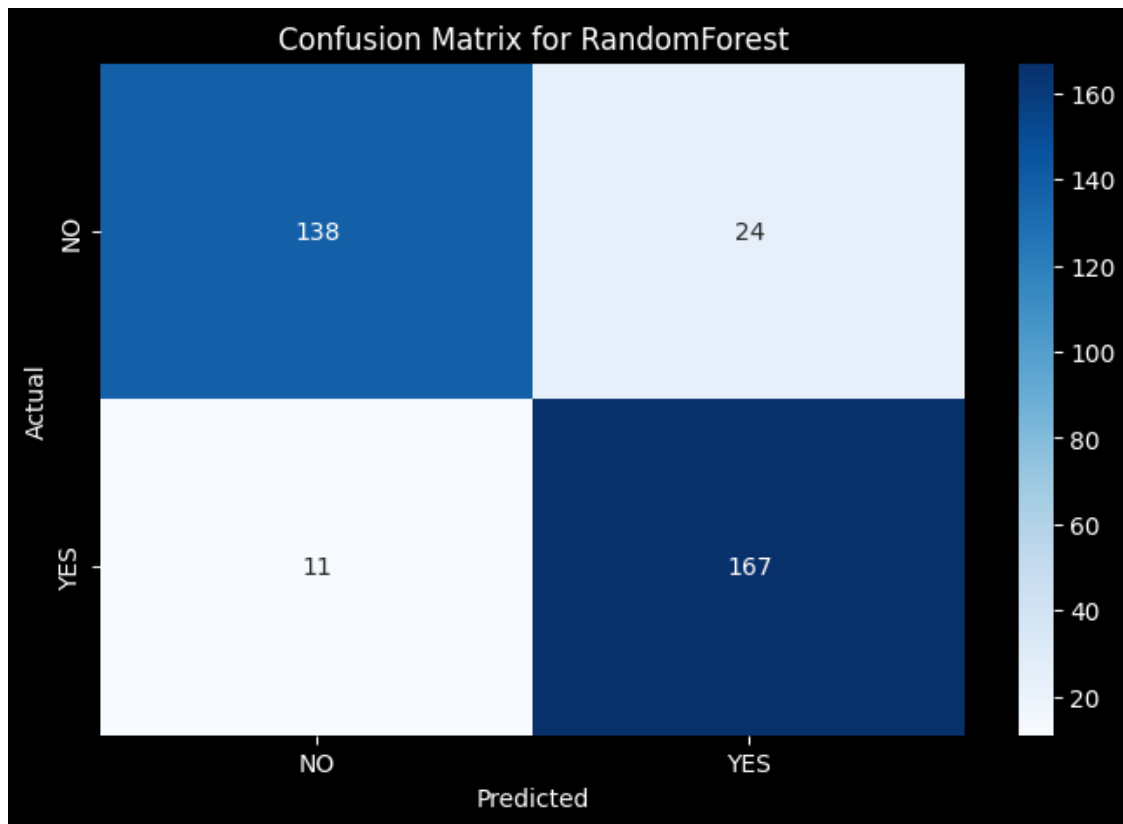
RandomForest - Training Accuracy: 0.992
RandomForest - Test Accuracy: 0.897
RandomForest - Training Precision: 0.988
RandomForest - Test Precision: 0.874

```

RandomForest - Training Recall: 0.997
RandomForest - Test Recall: 0.938
RandomForest - Training F1 Score: 0.993
RandomForest - Test F1 Score: 0.905
RandomForest - ROC AUC Score: 0.949



RandomForest - Confusion Matrix:
[[138 24]
[11 167]]

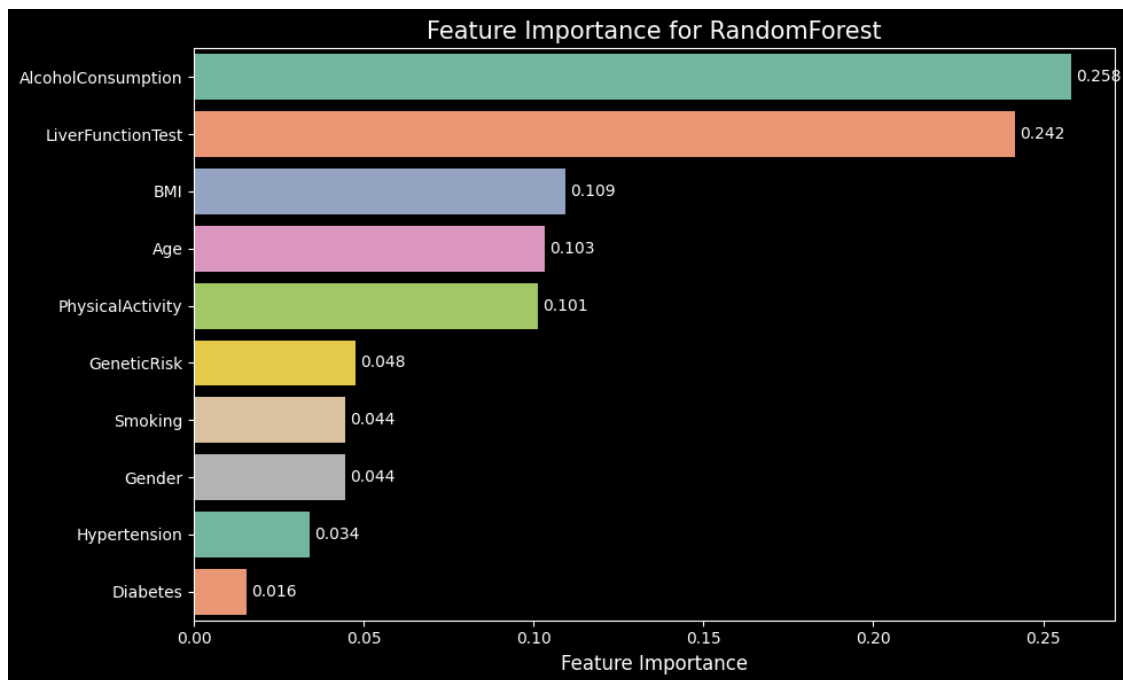


```

RandomForest - Classification Report:
              precision    recall  f1-score   support

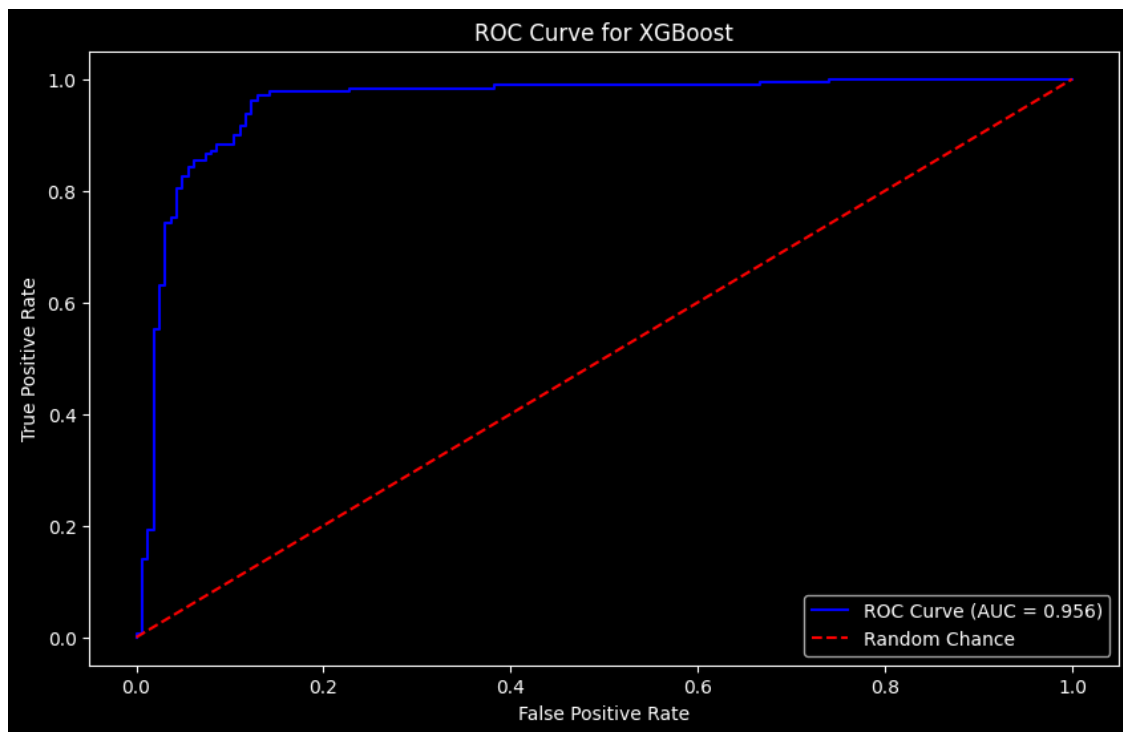
      NO         0.93        0.85        0.89         162
      YES         0.87        0.94        0.91         178

 accuracy              0.90         340
 macro avg              0.90         340
 weighted avg           0.90         340
  
```



```
[73]: # XGBoost
xgboost_params = best_results_df.loc[best_results_df['Model'] == 'XGBoost',
↳ 'Best Params'].values[0]
models['XGBoost'].set_params(**xgboost_params)
XGBoost = train_and_evaluate_model(models['XGBoost'], 'XGBoost', X_train,
↳ y_train, X_test, y_test)
```

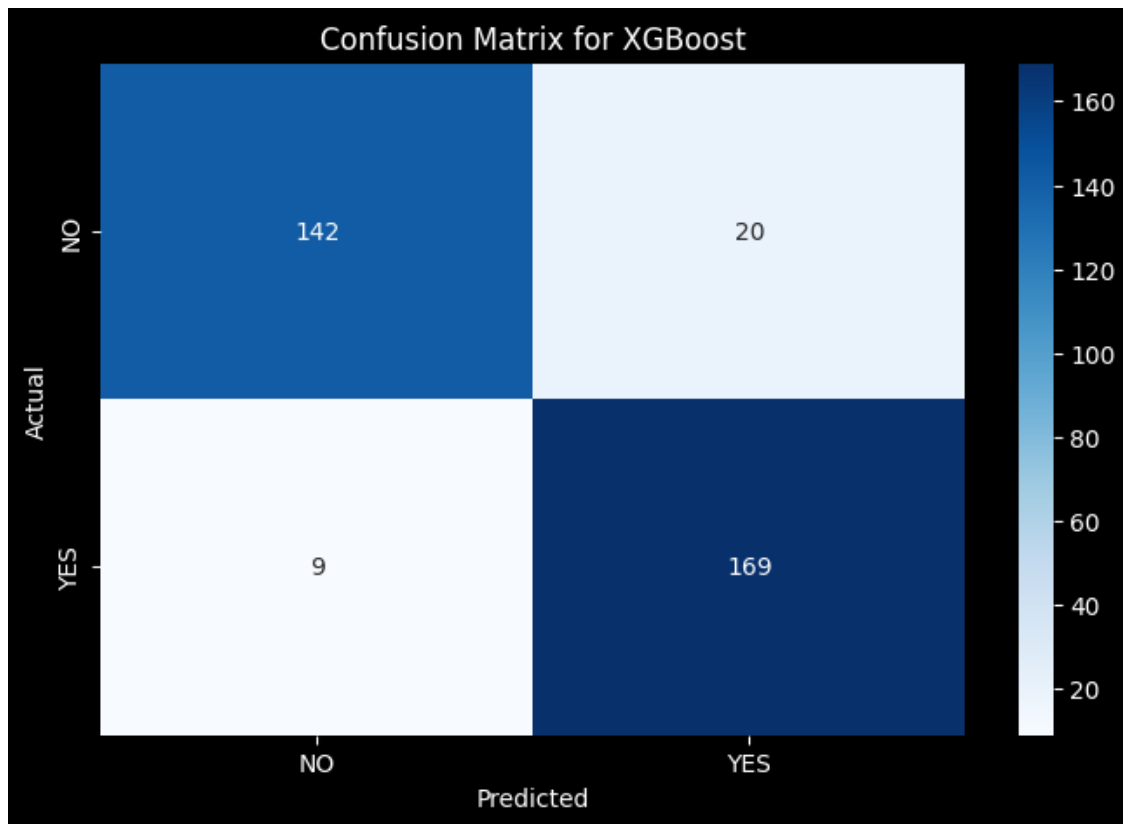
```
XGBoost - Training Accuracy: 0.954
XGBoost - Test Accuracy: 0.915
XGBoost - Training Precision: 0.953
XGBoost - Test Precision: 0.894
XGBoost - Training Recall: 0.966
XGBoost - Test Recall: 0.949
XGBoost - Training F1 Score: 0.959
XGBoost - Test F1 Score: 0.921
XGBoost - ROC AUC Score: 0.956
```



XGBoost - Confusion Matrix:

```
[[142  20]
```

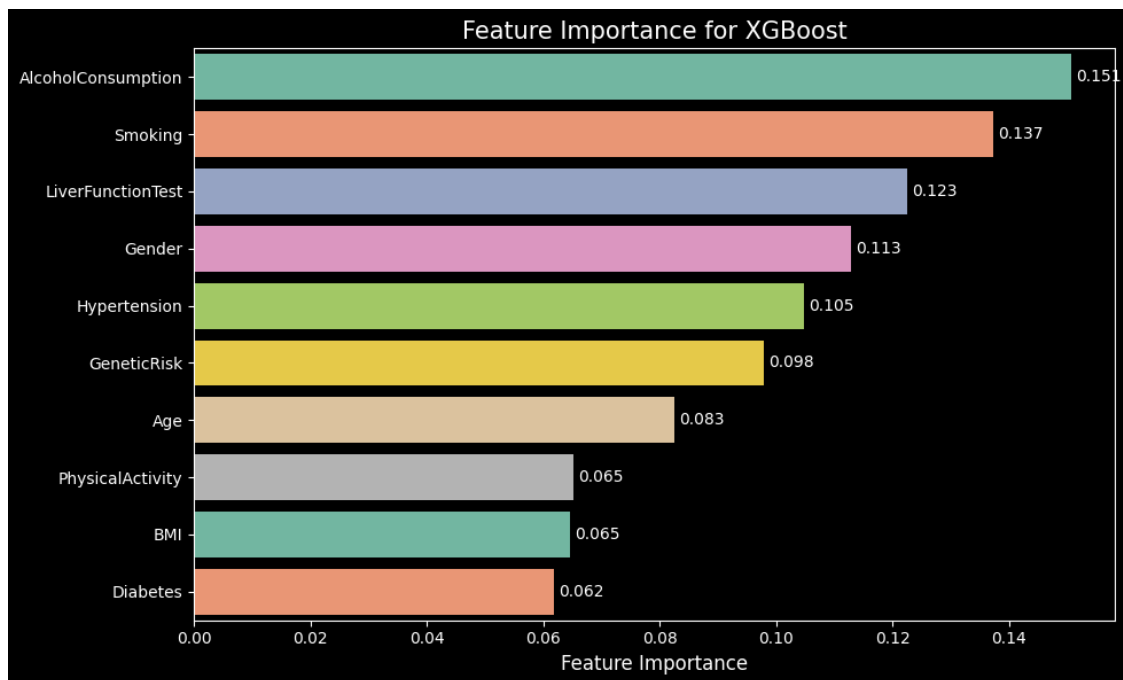
```
[  9 169]]
```



```
XGBoost - Classification Report:
              precision    recall  f1-score   support

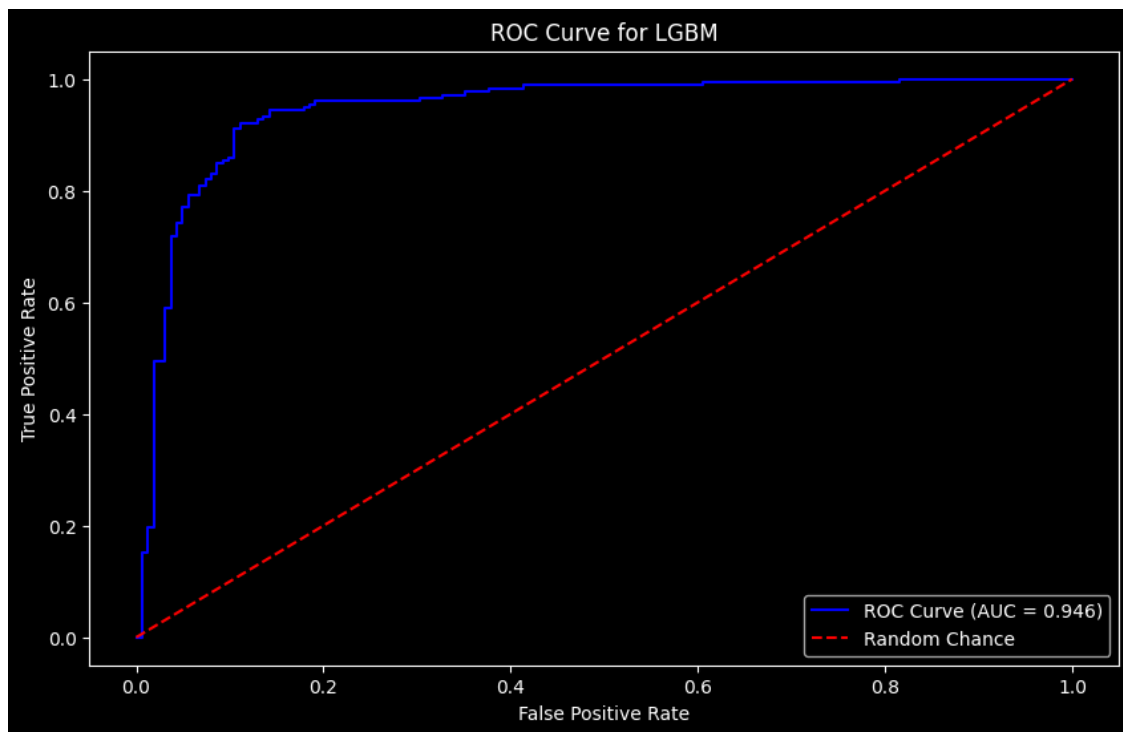
      NO         0.94         0.88         0.91         162
      YES         0.89         0.95         0.92         178

 accuracy              0.91         340
 macro avg           0.92         0.91         0.91         340
 weighted avg        0.92         0.91         0.91         340
```



```
[74]: # LGBM
lgbm_params = best_results_df.loc[best_results_df['Model'] == 'LGBM', 'Best_
↳Params'].values[0]
models['LGBM'].set_params(**lgbm_params)
LGBM = train_and_evaluate_model(models['LGBM'], 'LGBM', X_train, y_train,
↳X_test, y_test)
```

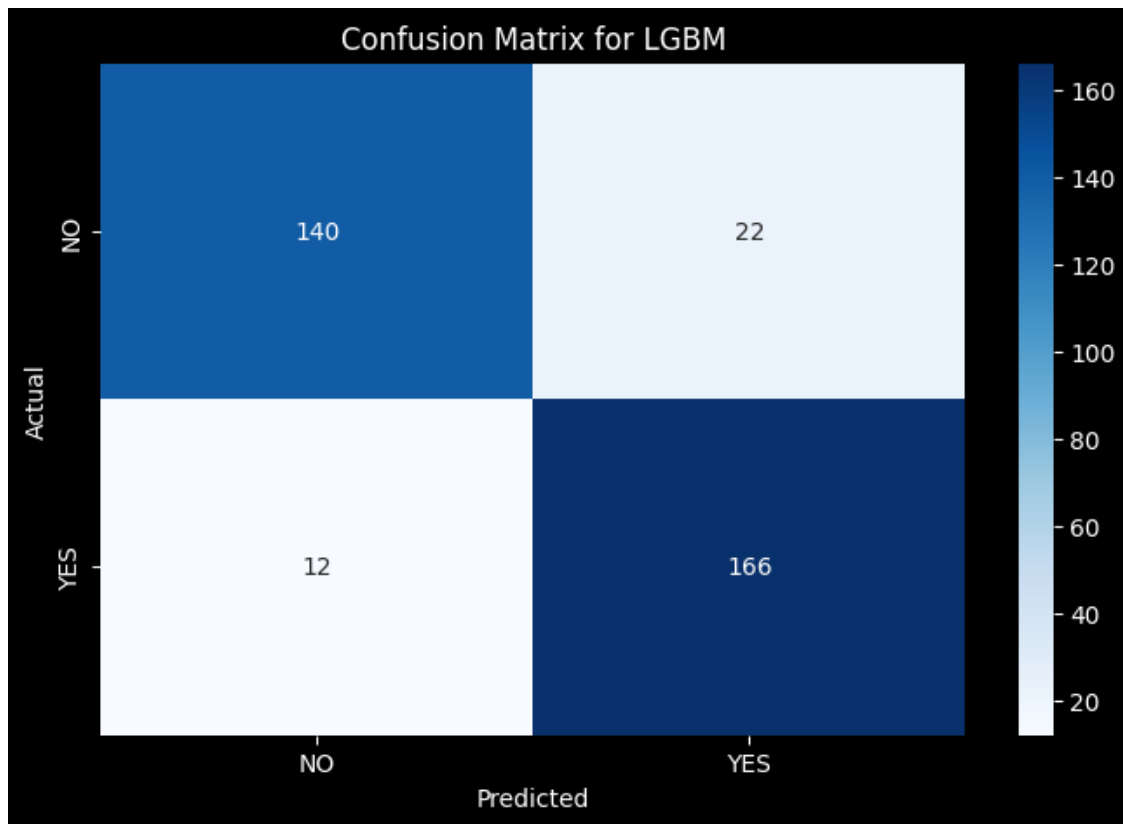
```
LGBM - Training Accuracy: 0.985
LGBM - Test Accuracy: 0.900
LGBM - Training Precision: 0.989
LGBM - Test Precision: 0.883
LGBM - Training Recall: 0.984
LGBM - Test Recall: 0.933
LGBM - Training F1 Score: 0.987
LGBM - Test F1 Score: 0.907
LGBM - ROC AUC Score: 0.946
```

LGBM - Confusion Matrix:

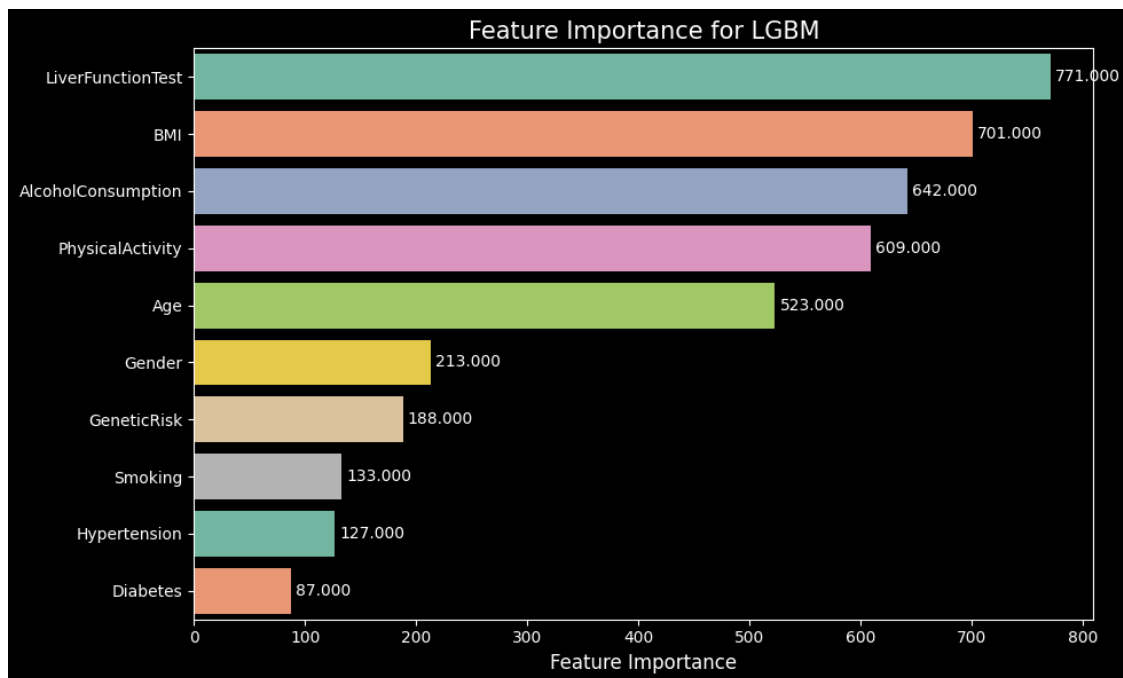
```
[[140  22]
```

```
[ 12 166]]
```



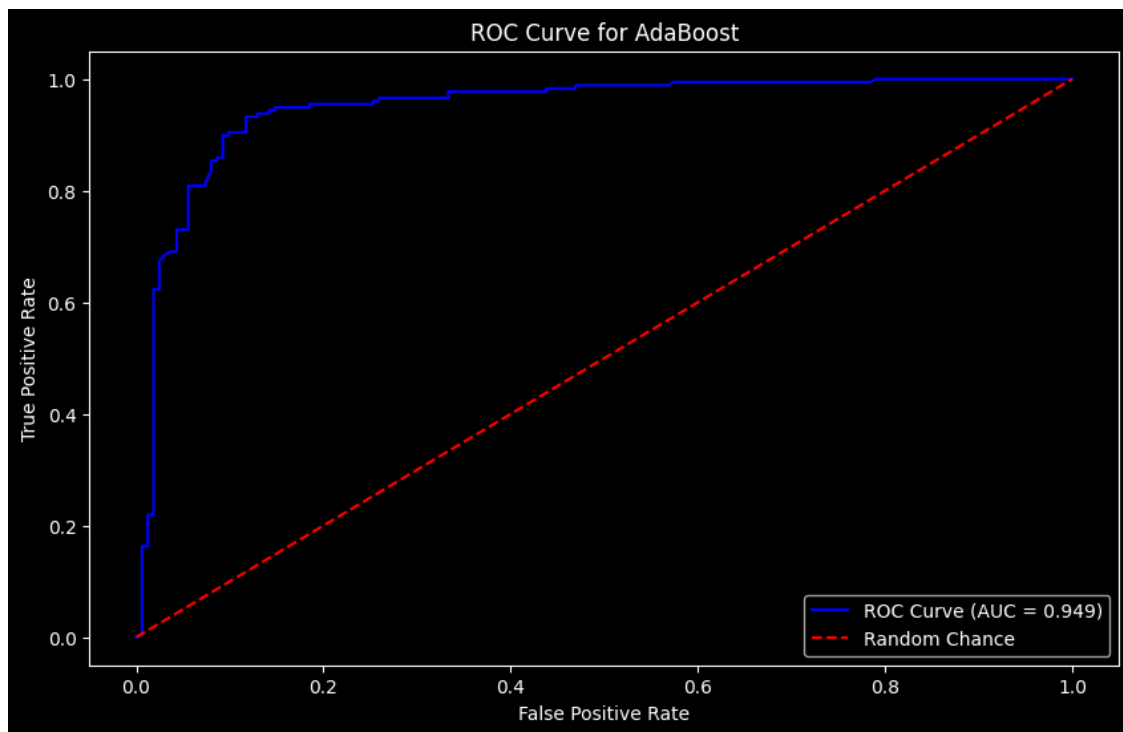
LGBM - Classification Report:

	precision	recall	f1-score	support
NO	0.92	0.86	0.89	162
YES	0.88	0.93	0.91	178
accuracy			0.90	340
macro avg	0.90	0.90	0.90	340
weighted avg	0.90	0.90	0.90	340



```
[75]: # AdaBoost
adaboost_params = best_results_df.loc[best_results_df['Model'] == 'AdaBoost',
↳ 'Best Params'].values[0]
models['AdaBoost'].set_params(**adaboost_params)
AdaBoost = train_and_evaluate_model(models['AdaBoost'], 'AdaBoost', X_train,
↳ y_train, X_test, y_test)
```

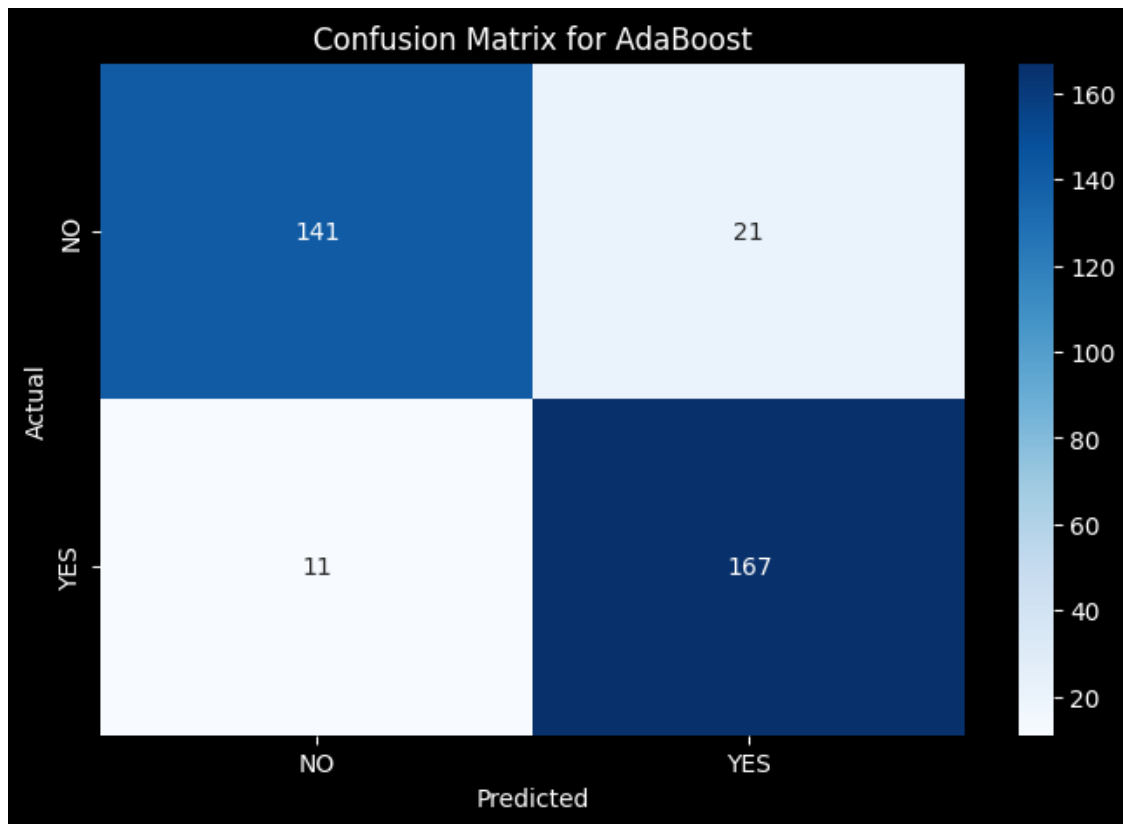
```
AdaBoost - Training Accuracy: 0.925
AdaBoost - Test Accuracy: 0.906
AdaBoost - Training Precision: 0.928
AdaBoost - Test Precision: 0.888
AdaBoost - Training Recall: 0.938
AdaBoost - Test Recall: 0.938
AdaBoost - Training F1 Score: 0.933
AdaBoost - Test F1 Score: 0.913
AdaBoost - ROC AUC Score: 0.949
```



AdaBoost - Confusion Matrix:

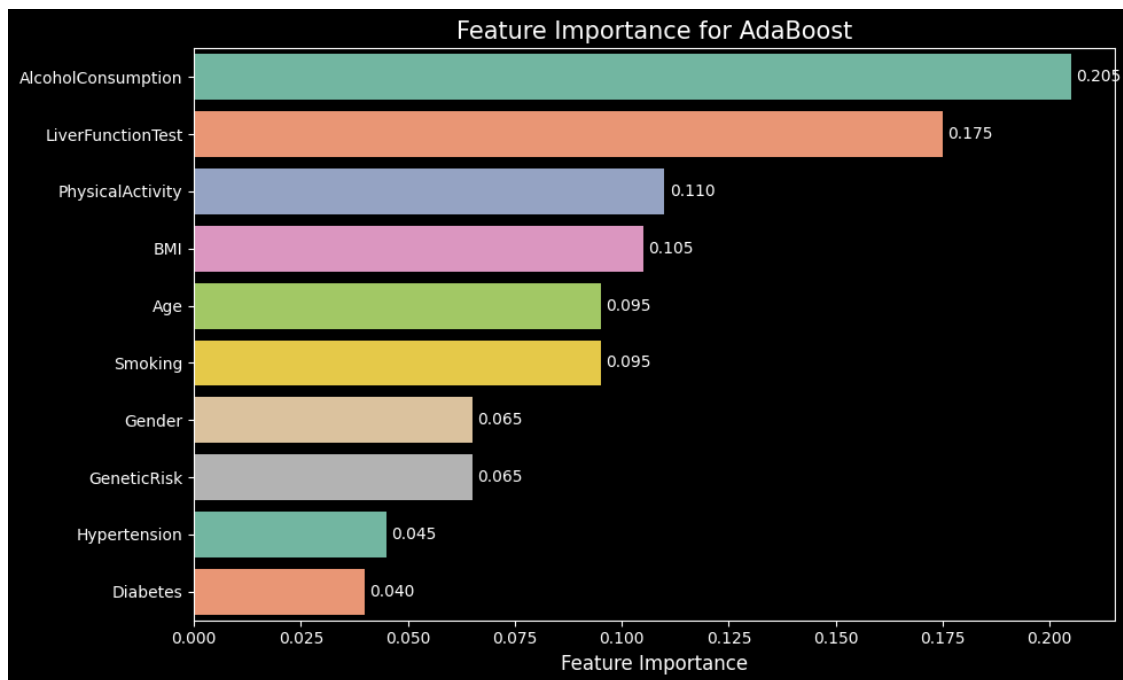
```
[[141  21]
```

```
[ 11 167]]
```



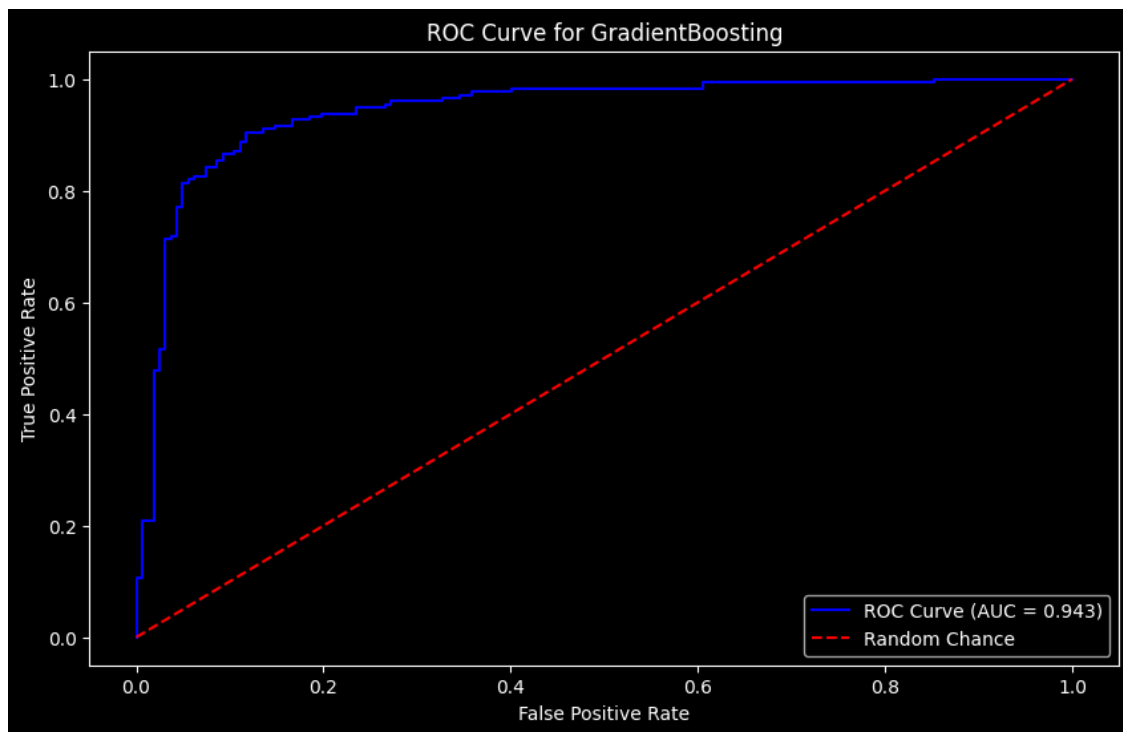
AdaBoost - Classification Report:

	precision	recall	f1-score	support
NO	0.93	0.87	0.90	162
YES	0.89	0.94	0.91	178
accuracy			0.91	340
macro avg	0.91	0.90	0.91	340
weighted avg	0.91	0.91	0.91	340



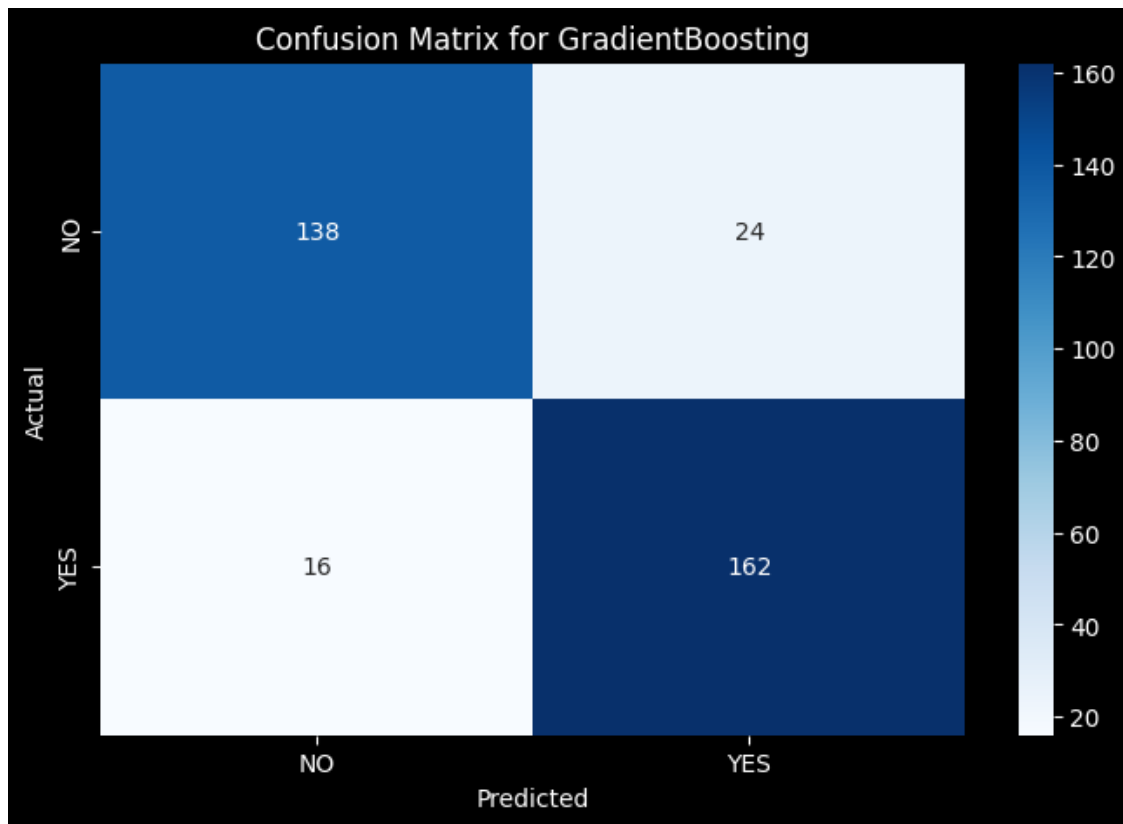
```
[76]: # Gradient Boosting
gradient_boosting_params = best_results_df.loc[best_results_df['Model'] ==
↳ 'GradientBoosting', 'Best Params'].values[0]
models['GradientBoosting'].set_params(**gradient_boosting_params)
GradientBoosting = train_and_evaluate_model(models['GradientBoosting'],
↳ 'GradientBoosting', X_train, y_train, X_test, y_test)
```

```
GradientBoosting - Training Accuracy: 1.000
GradientBoosting - Test Accuracy: 0.882
GradientBoosting - Training Precision: 1.000
GradientBoosting - Test Precision: 0.871
GradientBoosting - Training Recall: 1.000
GradientBoosting - Test Recall: 0.910
GradientBoosting - Training F1 Score: 1.000
GradientBoosting - Test F1 Score: 0.890
GradientBoosting - ROC AUC Score: 0.943
```



GradientBoosting - Confusion Matrix:

```
[[138  24]
 [ 16 162]]
```

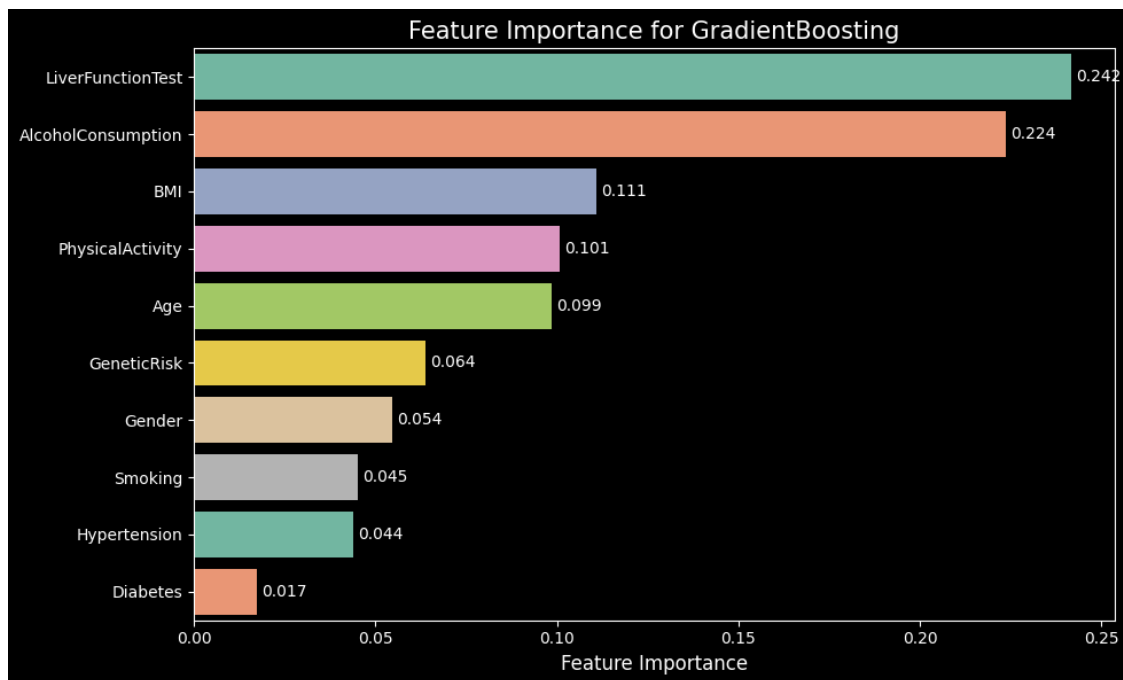


```

GradientBoosting - Classification Report:
              precision    recall  f1-score   support

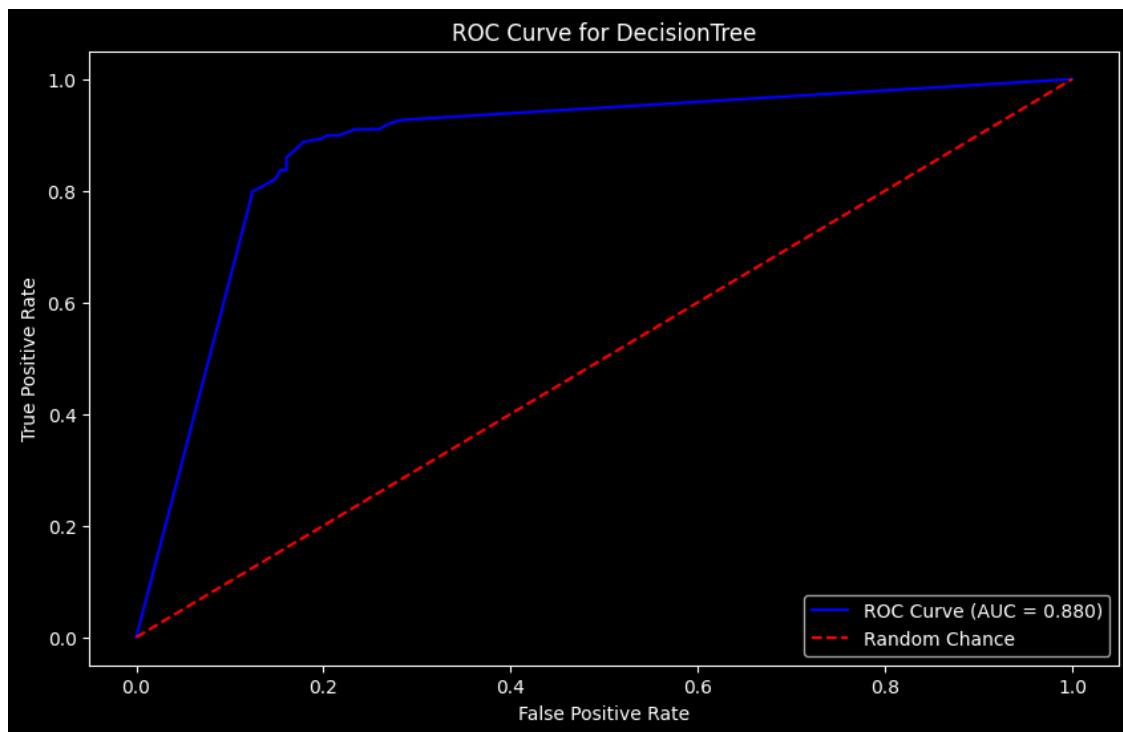
      NO         0.90      0.85      0.87        162
      YES         0.87      0.91      0.89        178

 accuracy              0.88        340
 macro avg           0.88      0.88      0.88        340
 weighted avg        0.88      0.88      0.88        340
  
```

```
[77]: # DecisionTree
decision_tree_params = best_results_df.loc[best_results_df['Model'] == 'DecisionTree', 'Best Params'].values[0]
models['DecisionTree'].set_params(**decision_tree_params)
DecisionTree = train_and_evaluate_model(models['DecisionTree'], 'DecisionTree', X_train, y_train, X_test, y_test)
```

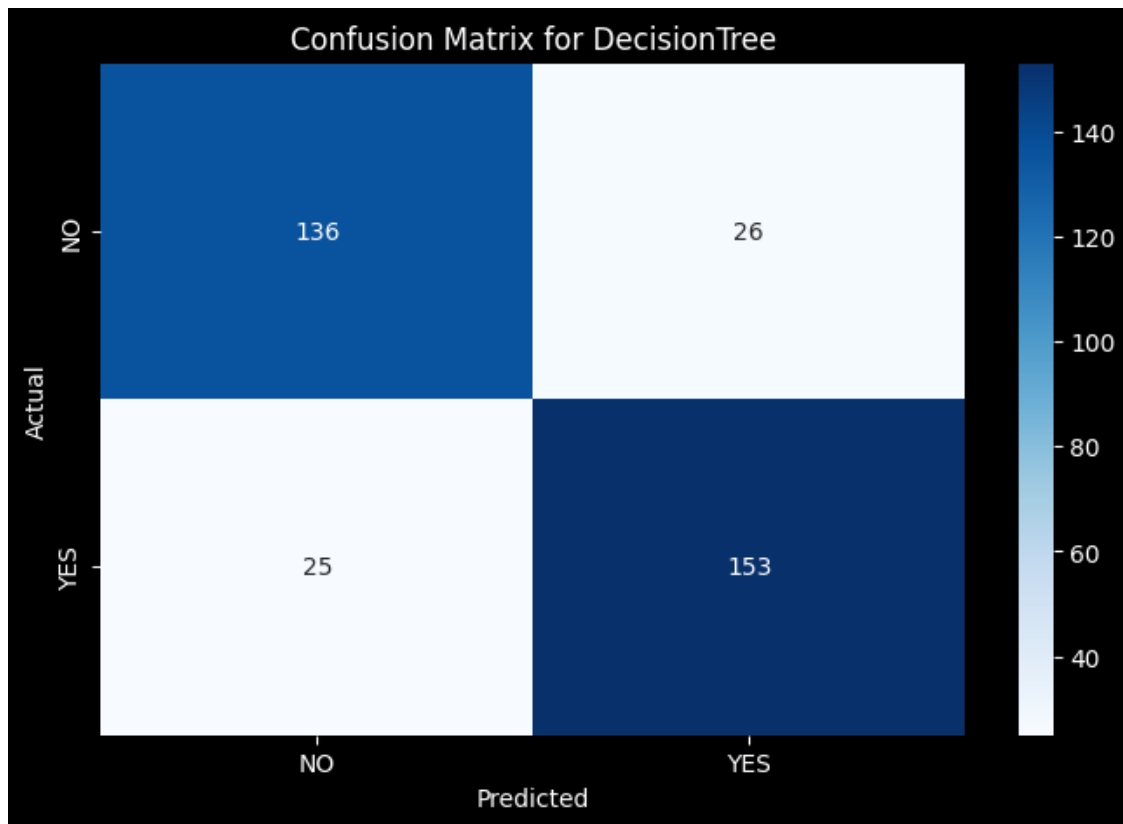
```
DecisionTree - Training Accuracy: 0.960
DecisionTree - Test Accuracy: 0.850
DecisionTree - Training Precision: 0.978
DecisionTree - Test Precision: 0.855
DecisionTree - Training Recall: 0.950
DecisionTree - Test Recall: 0.860
DecisionTree - Training F1 Score: 0.964
DecisionTree - Test F1 Score: 0.857
DecisionTree - ROC AUC Score: 0.880
```



DecisionTree - Confusion Matrix:

```
[[136 26]
```

```
[ 25 153]]
```



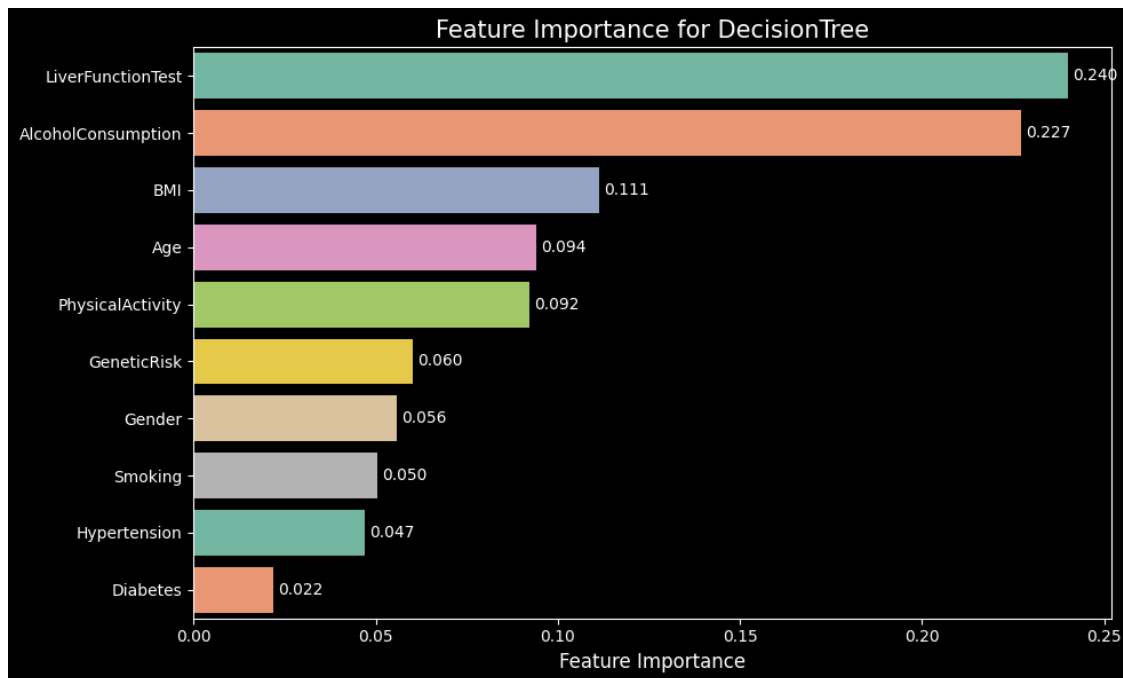
```

DecisionTree - Classification Report:
              precision    recall  f1-score   support

      NO         0.84         0.84         0.84         162
      YES         0.85         0.86         0.86         178

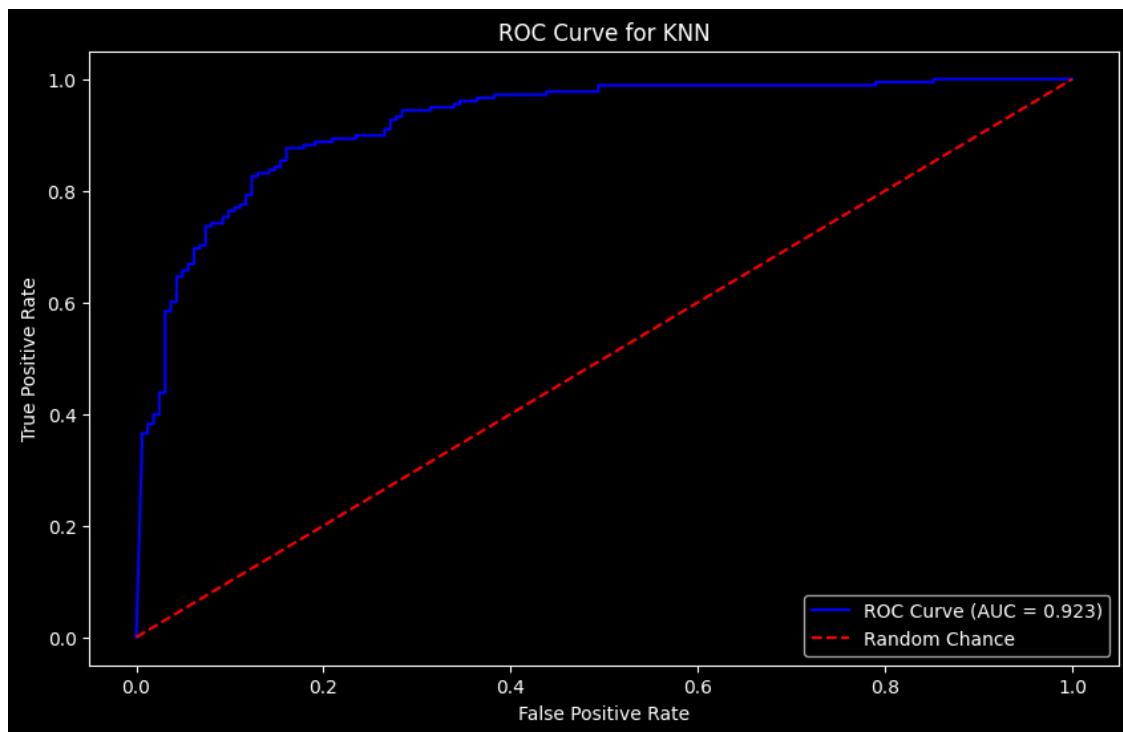
 accuracy              0.85         340
 macro avg         0.85         0.85         0.85         340
 weighted avg         0.85         0.85         0.85         340

```



```
[78]: # KNN
knn_params = best_results_df.loc[best_results_df['Model'] == 'KNN', 'Best_
↳Params'].values[0]
models['KNN'].set_params(**knn_params)
KNN = train_and_evaluate_model(models['KNN'], 'KNN', X_train, y_train, X_test,
↳y_test)
```

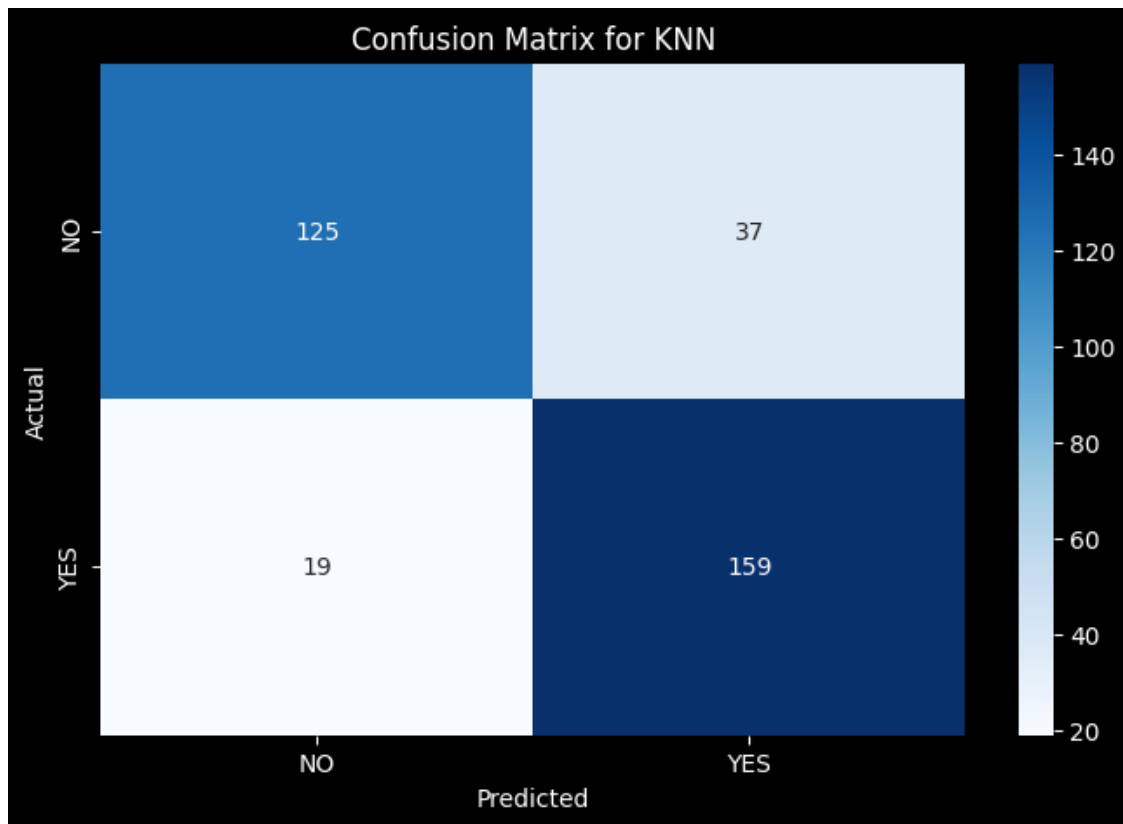
```
KNN - Training Accuracy: 1.000
KNN - Test Accuracy: 0.835
KNN - Training Precision: 1.000
KNN - Test Precision: 0.811
KNN - Training Recall: 1.000
KNN - Test Recall: 0.893
KNN - Training F1 Score: 1.000
KNN - Test F1 Score: 0.850
KNN - ROC AUC Score: 0.923
```



KNN - Confusion Matrix:

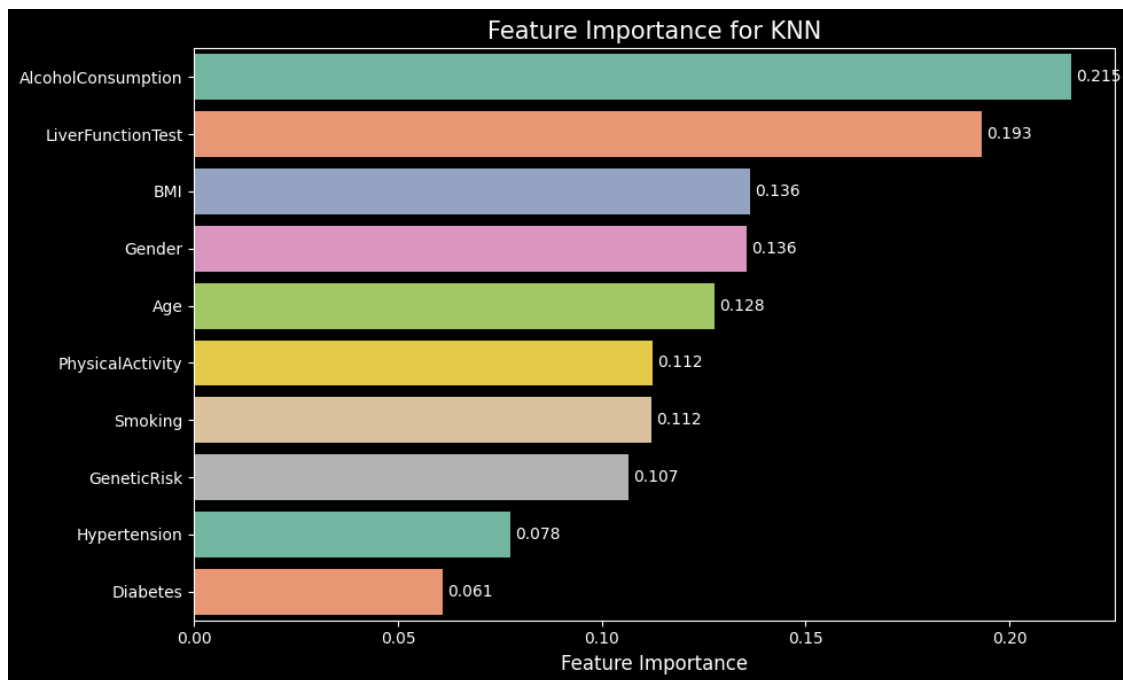
```
[[125  37]
```

```
[ 19 159]]
```



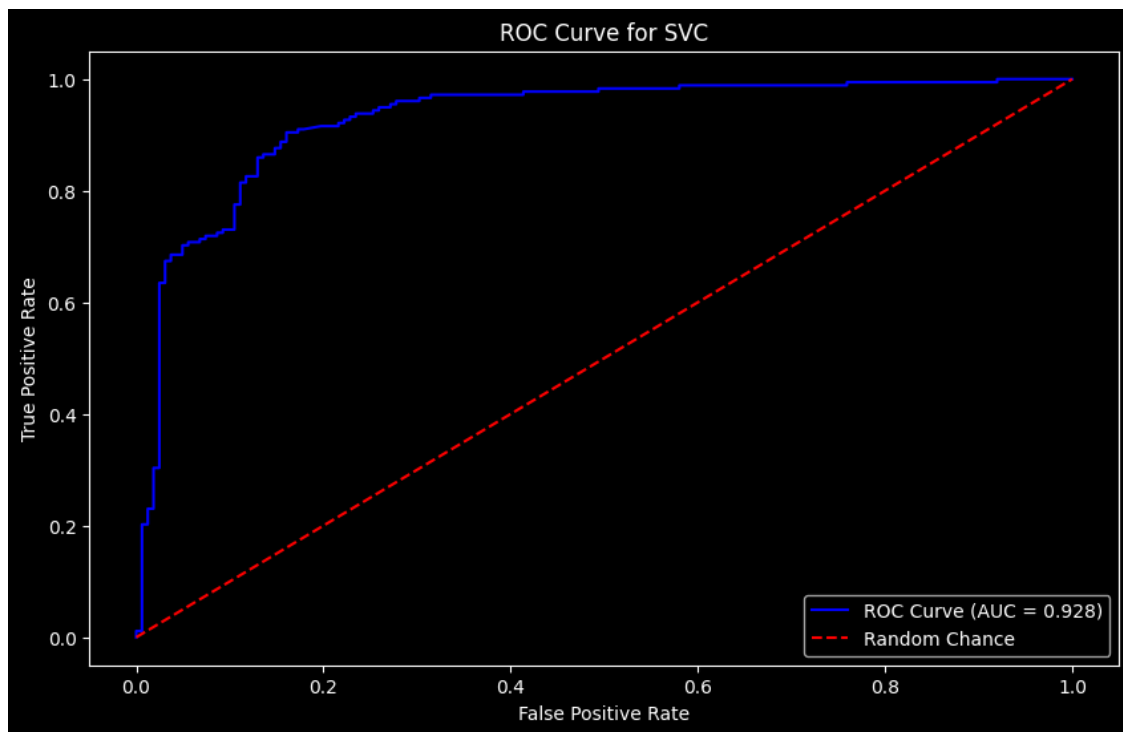
KNN - Classification Report:

	precision	recall	f1-score	support
NO	0.87	0.77	0.82	162
YES	0.81	0.89	0.85	178
accuracy			0.84	340
macro avg	0.84	0.83	0.83	340
weighted avg	0.84	0.84	0.83	340



```
[79]: # SVC
svc_params = best_results_df.loc[best_results_df['Model'] == 'SVC', 'Best_
↳Params'].values[0]
models['SVC'].set_params(**svc_params)
SVC = train_and_evaluate_model(models['SVC'], 'SVC', X_train, y_train, X_test,
↳y_test)
```

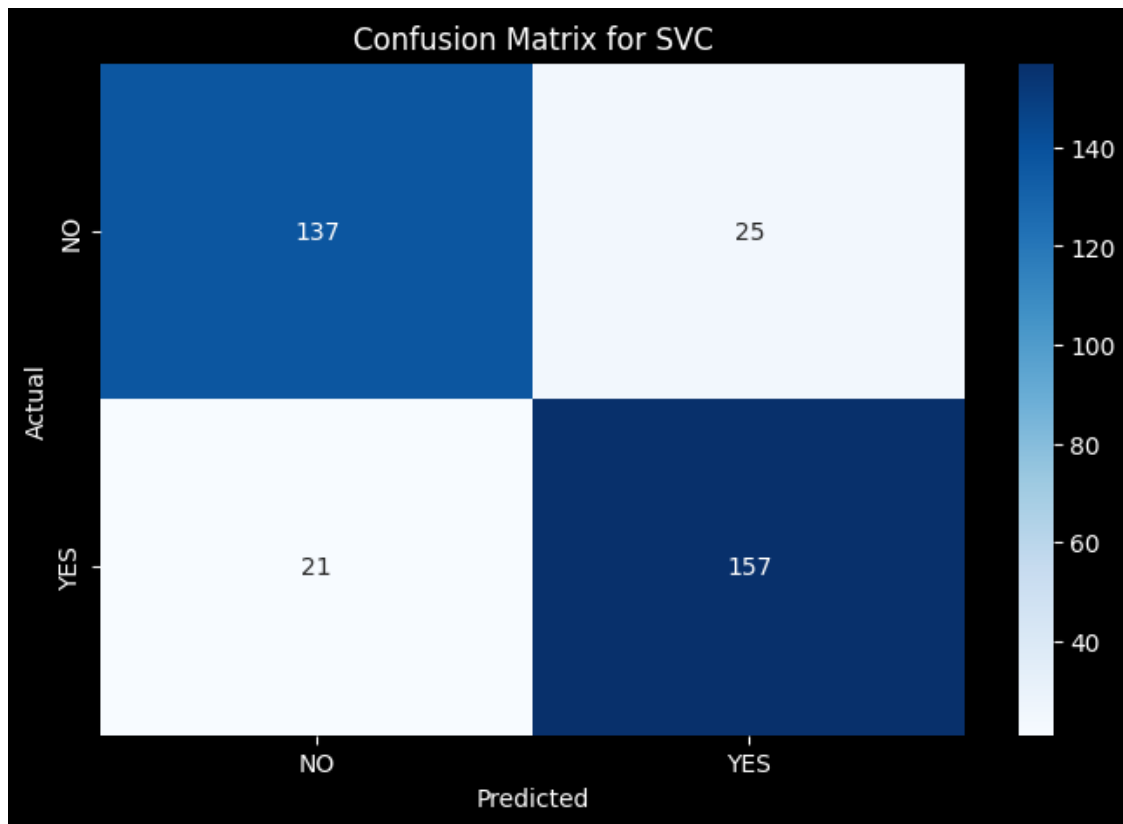
```
SVC - Training Accuracy: 0.872
SVC - Test Accuracy: 0.865
SVC - Training Precision: 0.891
SVC - Test Precision: 0.863
SVC - Training Recall: 0.877
SVC - Test Recall: 0.882
SVC - Training F1 Score: 0.884
SVC - Test F1 Score: 0.872
SVC - ROC AUC Score: 0.928
```



SVC - Confusion Matrix:

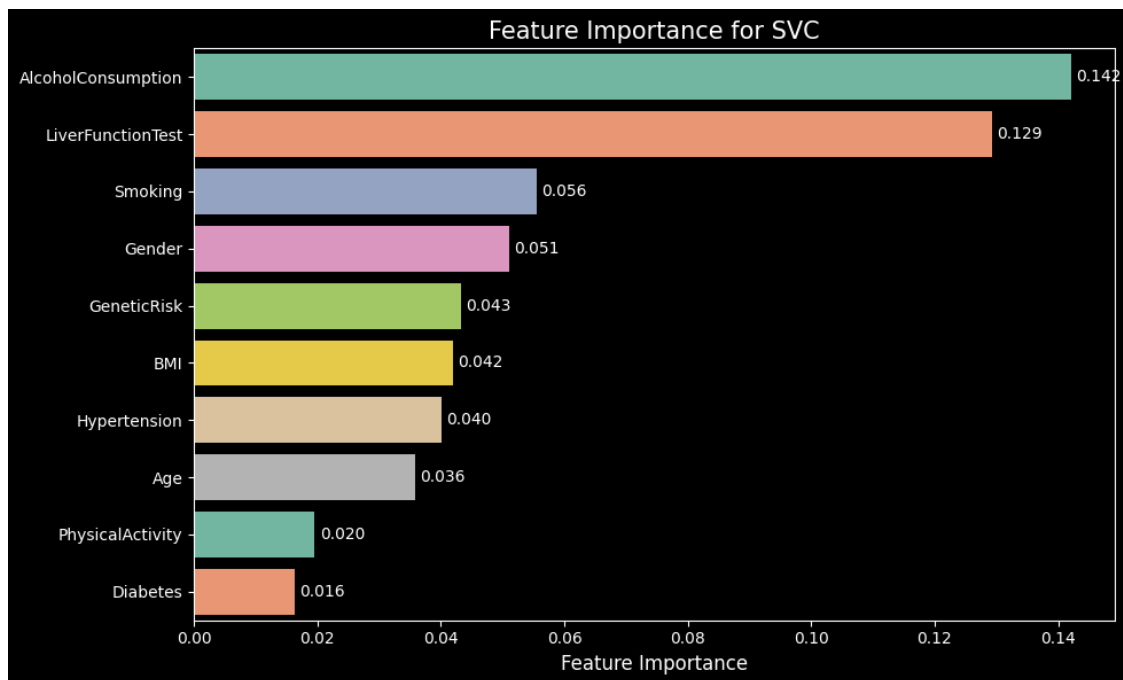
```
[[137  25]
```

```
[ 21 157]]
```

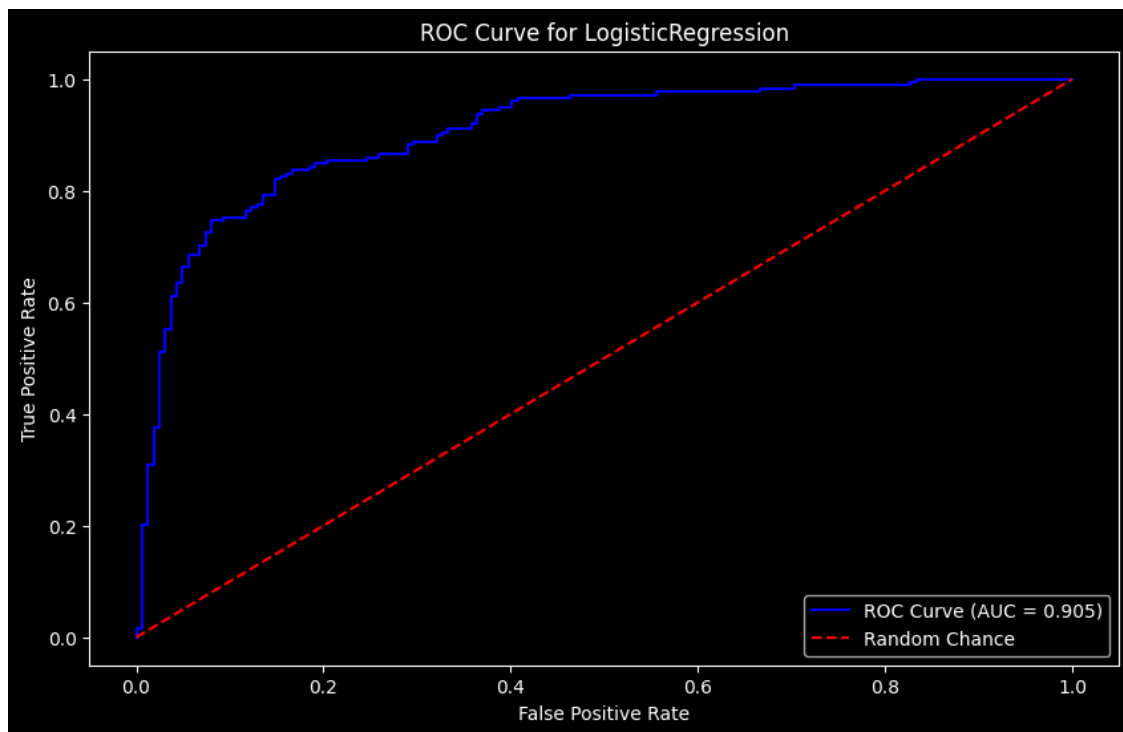
SVC - Classification Report:

	precision	recall	f1-score	support
NO	0.87	0.85	0.86	162
YES	0.86	0.88	0.87	178
accuracy			0.86	340
macro avg	0.86	0.86	0.86	340
weighted avg	0.86	0.86	0.86	340

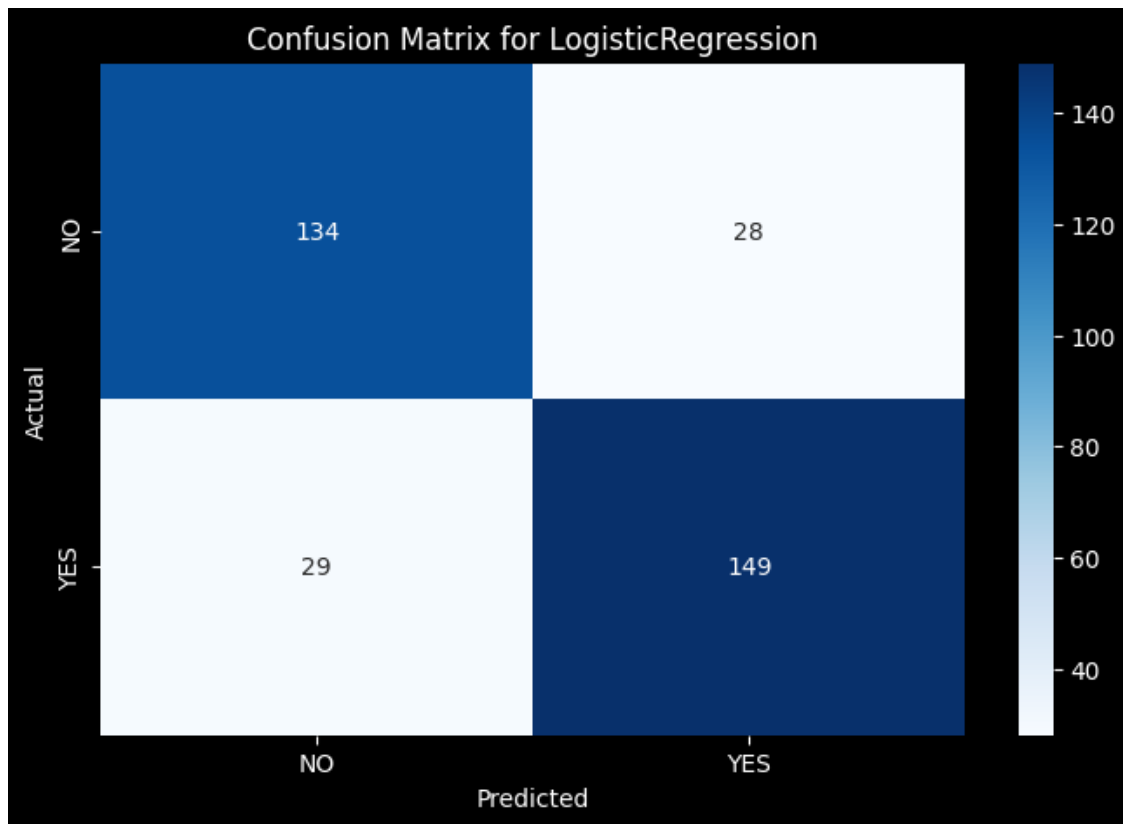


```
[80]: # Logistic Regression
logistic_params = best_results_df.loc[best_results_df['Model'] == 'LogisticRegression', 'Best Params'].values[0]
models['LogisticRegression'].set_params(**logistic_params)
LogisticRegression = train_and_evaluate_model(models['LogisticRegression'], X_train, y_train, X_test, y_test)
```

```
LogisticRegression - Training Accuracy: 0.824
LogisticRegression - Test Accuracy: 0.832
LogisticRegression - Training Precision: 0.857
LogisticRegression - Test Precision: 0.842
LogisticRegression - Training Recall: 0.822
LogisticRegression - Test Recall: 0.837
LogisticRegression - Training F1 Score: 0.839
LogisticRegression - Test F1 Score: 0.839
LogisticRegression - ROC AUC Score: 0.905
```



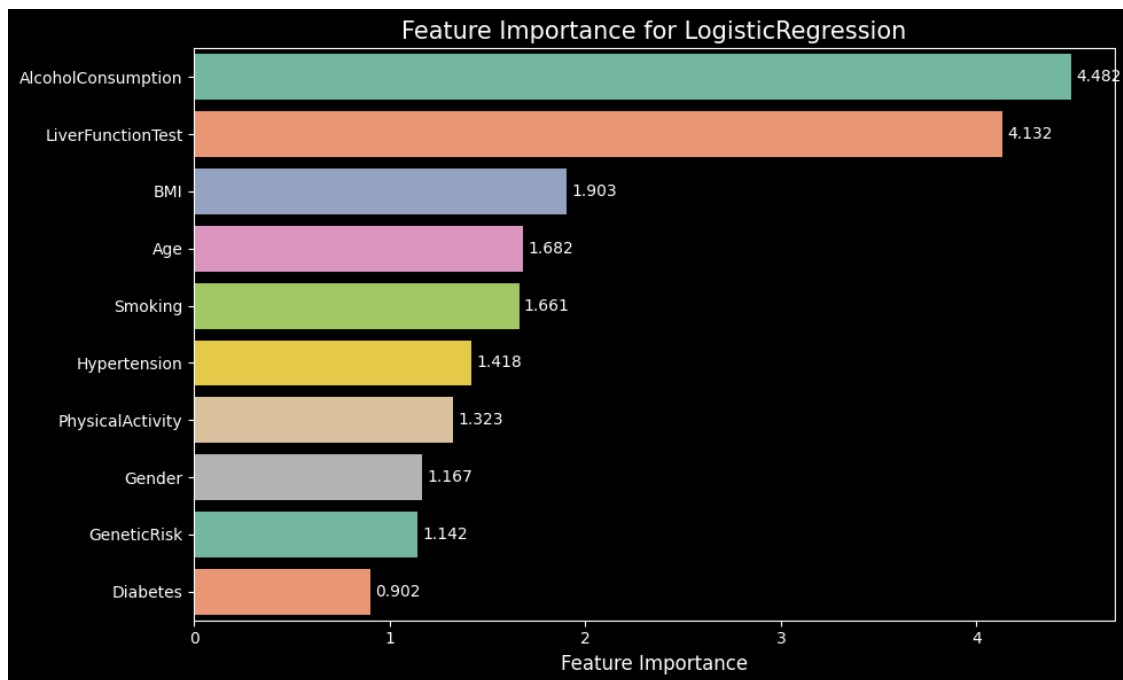
LogisticRegression - Confusion Matrix:
[[134 28]
[29 149]]



```
LogisticRegression - Classification Report:
              precision    recall  f1-score   support

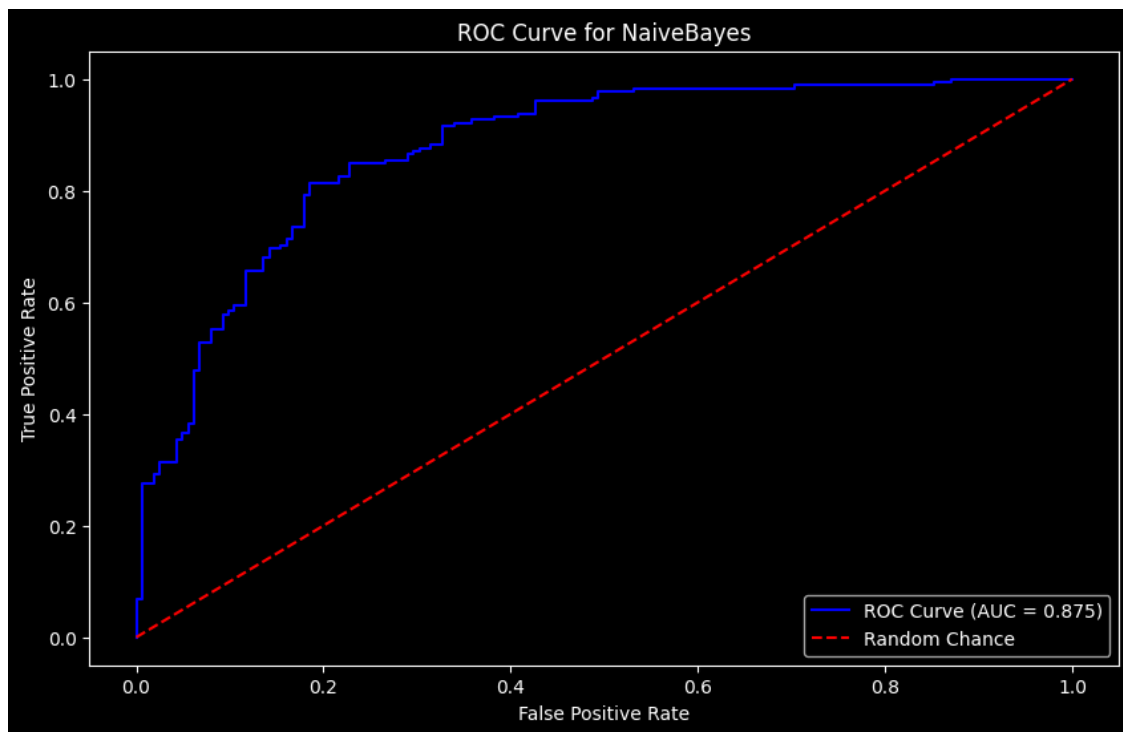
     NO         0.82         0.83         0.82         162
     YES         0.84         0.84         0.84         178

 accuracy              0.83         340
 macro avg         0.83         0.83         0.83         340
 weighted avg         0.83         0.83         0.83         340
```

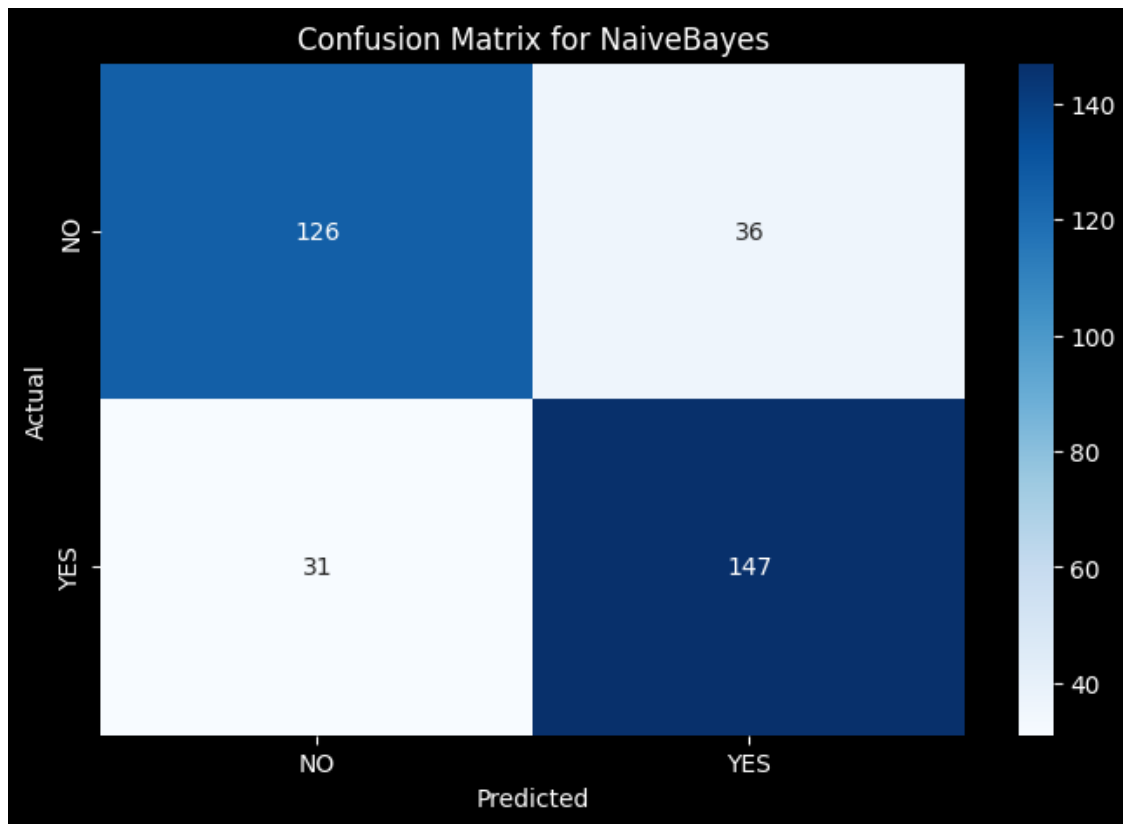


```
[81]: # Naive Bayes
naive_bayes_params = best_results_df.loc[best_results_df['Model'] ==
    ↳ 'NaiveBayes', 'Best Params'].values[0]
models['NaiveBayes'].set_params(**naive_bayes_params)
NaiveBayes = train_and_evaluate_model(models['NaiveBayes'], 'NaiveBayes',
    ↳ X_train, y_train, X_test, y_test)
```

```
NaiveBayes - Training Accuracy: 0.810
NaiveBayes - Test Accuracy: 0.803
NaiveBayes - Training Precision: 0.832
NaiveBayes - Test Precision: 0.803
NaiveBayes - Training Recall: 0.826
NaiveBayes - Test Recall: 0.826
NaiveBayes - Training F1 Score: 0.829
NaiveBayes - Test F1 Score: 0.814
NaiveBayes - ROC AUC Score: 0.875
```

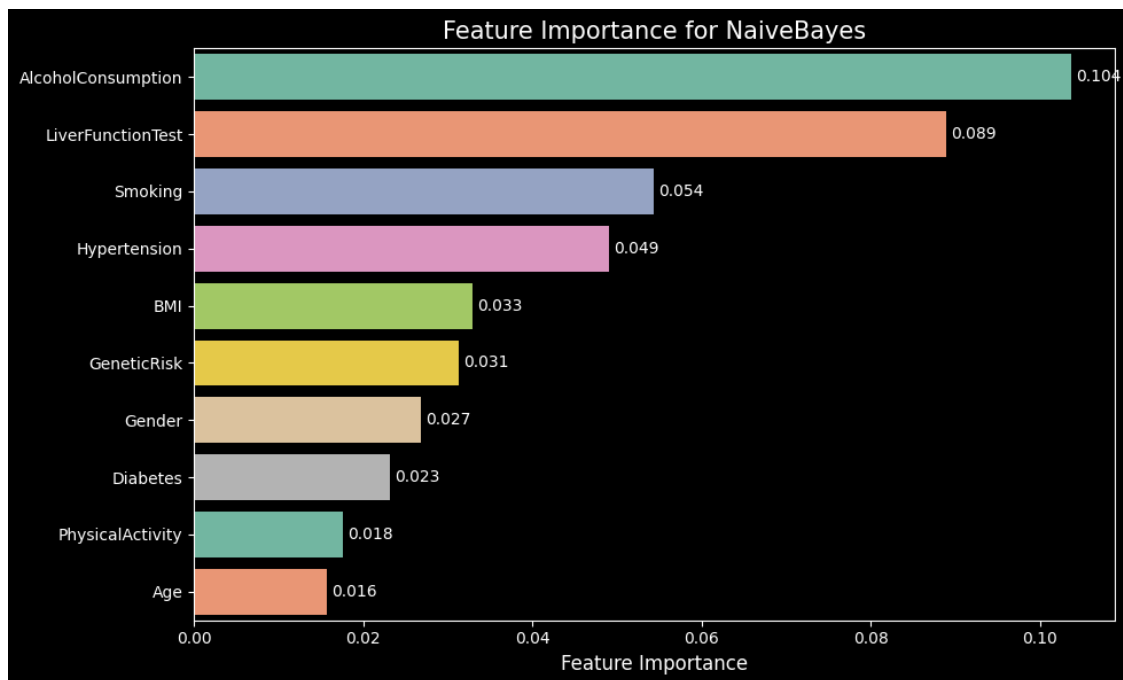


NaiveBayes - Confusion Matrix:
[[126 36]
[31 147]]



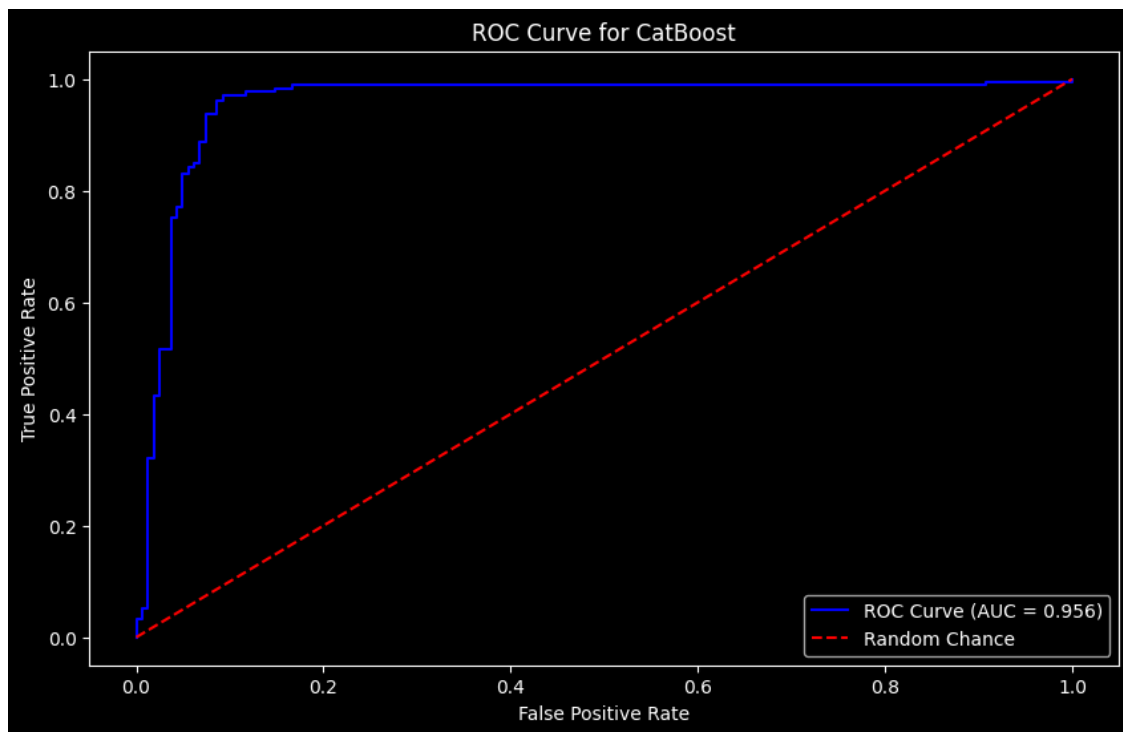
NaiveBayes - Classification Report:

	precision	recall	f1-score	support
NO	0.80	0.78	0.79	162
YES	0.80	0.83	0.81	178
accuracy			0.80	340
macro avg	0.80	0.80	0.80	340
weighted avg	0.80	0.80	0.80	340



```
[82]: # CatBoost
catboost_params = best_results_df.loc[best_results_df['Model'] == 'CatBoost',
↳ 'Best Params'].values[0]
models['CatBoost'].set_params(**catboost_params)
CatBoost = train_and_evaluate_model(models['CatBoost'], 'CatBoost', X_train,
↳ y_train, X_test, y_test)
```

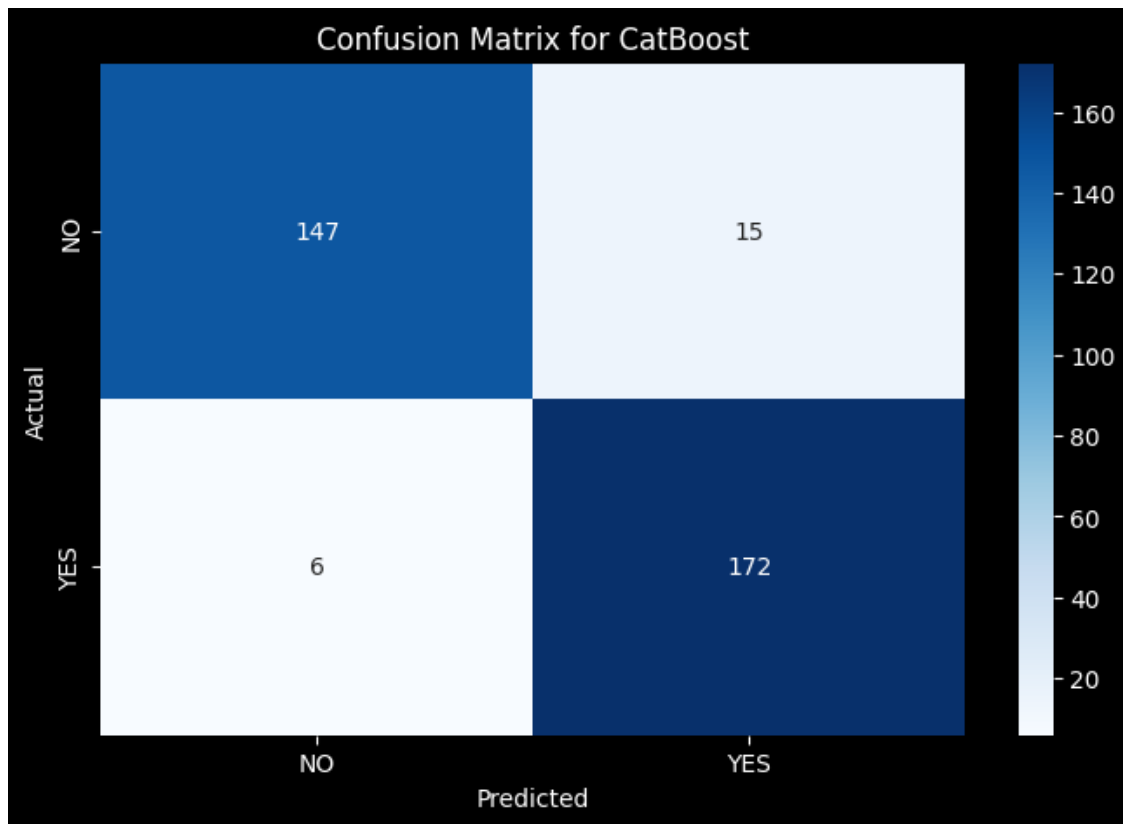
```
CatBoost - Training Accuracy: 0.973
CatBoost - Test Accuracy: 0.938
CatBoost - Training Precision: 0.976
CatBoost - Test Precision: 0.920
CatBoost - Training Recall: 0.975
CatBoost - Test Recall: 0.966
CatBoost - Training F1 Score: 0.976
CatBoost - Test F1 Score: 0.942
CatBoost - ROC AUC Score: 0.956
```

CatBoost - Confusion Matrix:

```
[[147  15]
```

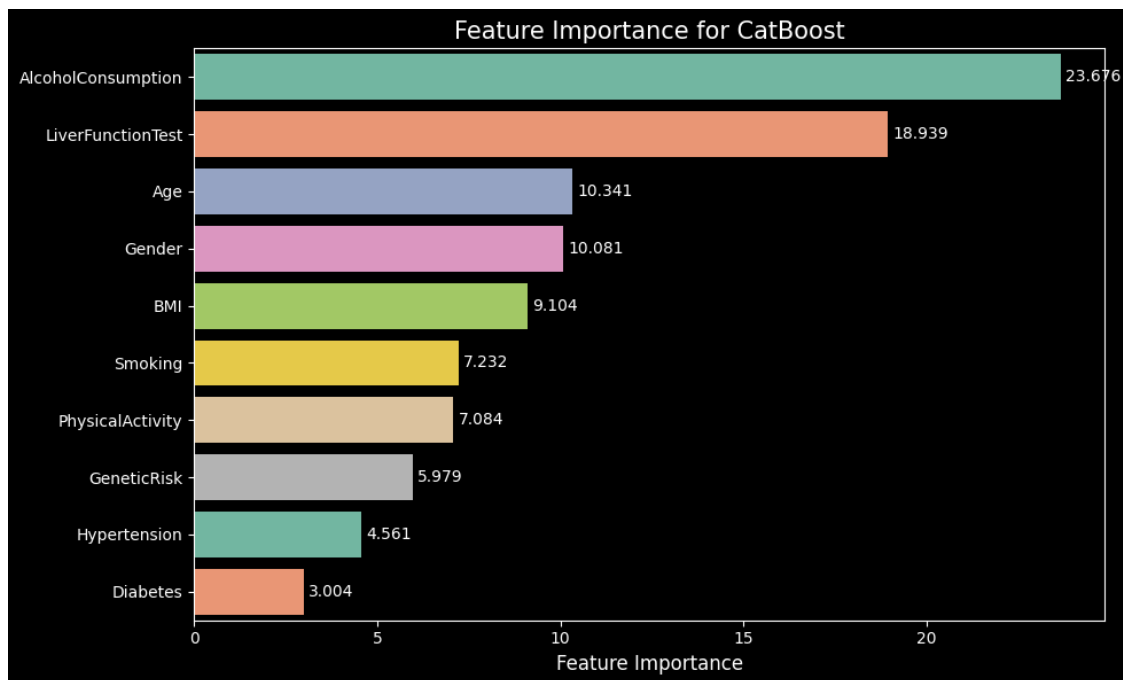
```
[  6 172]]
```



```
CatBoost - Classification Report:
              precision    recall  f1-score   support

     NO         0.96         0.91         0.93         162
     YES         0.92         0.97         0.94         178

 accuracy              0.94         340
 macro avg         0.94         0.94         0.94         340
 weighted avg         0.94         0.94         0.94         340
```



```
[83]: metrics_df = pd.DataFrame(metrics_dict).T
      metrics_df = metrics_df.sort_values(by=['test_f1', 'test_accuracy'],
      ↪ascending=False)
      metrics_df
```

```
[83]:
```

	train_accuracy	train_precision	train_recall	train_f1	\
CatBoost	0.972794	0.976222	0.974934	0.975578	
XGBoost	0.954412	0.953125	0.965699	0.959371	
AdaBoost	0.925000	0.928198	0.937995	0.933071	
LGBM	0.985294	0.989390	0.984169	0.986772	
RandomForest	0.991912	0.988235	0.997361	0.992777	
GradientBoosting	1.000000	1.000000	1.000000	1.000000	
SVC	0.872059	0.891421	0.877309	0.884309	
DecisionTree	0.960294	0.978261	0.949868	0.963855	
KNN	1.000000	1.000000	1.000000	1.000000	
LogisticRegression	0.824265	0.856946	0.821900	0.839057	
NaiveBayes	0.810294	0.832447	0.825858	0.829139	

	test_accuracy	test_precision	test_recall	test_f1	\
CatBoost	0.938235	0.919786	0.966292	0.942466	
XGBoost	0.914706	0.894180	0.949438	0.920981	
AdaBoost	0.905882	0.888298	0.938202	0.912568	
LGBM	0.900000	0.882979	0.932584	0.907104	
RandomForest	0.897059	0.874346	0.938202	0.905149	
GradientBoosting	0.882353	0.870968	0.910112	0.890110	

SVC	0.864706	0.862637	0.882022	0.872222
DecisionTree	0.850000	0.854749	0.859551	0.857143
KNN	0.835294	0.811224	0.893258	0.850267
LogisticRegression	0.832353	0.841808	0.837079	0.839437
NaiveBayes	0.802941	0.803279	0.825843	0.814404

	roc_auc
CatBoost	0.956374
XGBoost	0.956235
AdaBoost	0.948779
LGBM	0.945554
RandomForest	0.949195
GradientBoosting	0.943473
SVC	0.928405
DecisionTree	0.880289
KNN	0.922840
LogisticRegression	0.904876
NaiveBayes	0.874567

```
[84]: # Assuming metrics_df contains the evaluation metrics and is sorted by
      ↪ performance
      # Get the name of the best model
      # Get the metrics for the best model
      best_model_name = metrics_df.index[0]
      best_model_metrics = metrics_df.loc[best_model_name]
      print(f"The best model is: {best_model_name}")
      print(f"Metrics for {best_model_name}:")
      print(f"  Training Accuracy: {best_model_metrics['train_accuracy']:.2f}")
      print(f"  Training Precision: {best_model_metrics['train_precision']:.2f}")
      print(f"  Training Recall: {best_model_metrics['train_recall']:.2f}")
      print(f"  Training F1 Score: {best_model_metrics['train_f1']:.2f}")
      print(f"  Test Accuracy: {best_model_metrics['test_accuracy']:.2f}")
      print(f"  Test Precision: {best_model_metrics['test_precision']:.2f}")
      print(f"  Test Recall: {best_model_metrics['test_recall']:.2f}")
      print(f"  Test F1 Score: {best_model_metrics['test_f1']:.2f}")
```

The best model is: CatBoost

Metrics for CatBoost:

```
  Training Accuracy: 0.97
  Training Precision: 0.98
  Training Recall: 0.97
  Training F1 Score: 0.98
  Test Accuracy: 0.94
  Test Precision: 0.92
  Test Recall: 0.97
  Test F1 Score: 0.94
```

```
[85]: # Plot Metrics
fig, axes = plt.subplots(5, 2, figsize=(25, 35))

# Define font properties for x-axis labels
font_properties = {'fontsize': 12, 'fontweight': 'bold', 'family': 'serif'}

# Plot Train Accuracy and Test Accuracy
sns.barplot(data=metrics_df, x=metrics_df.index, y='train_accuracy', ax=axes[0,0],
            palette='Set1')
axes[0, 0].set_title('Train Accuracy for All Models', fontsize=15,
                    weight='bold')
axes[0, 0].set_ylabel('Accuracy', fontsize=12)
for container in axes[0, 0].containers:
    axes[0, 0].bar_label(container, labels=[f'{v.get_height():.3f}' for v in
    container], padding=3)
axes[0, 0].tick_params(axis='x', rotation=-45)
axes[0, 0].set_xticklabels(metrics_df.index, fontdict=font_properties)

sns.barplot(data=metrics_df, x=metrics_df.index, y='test_accuracy', ax=axes[0,1],
            palette='Set2')
axes[0, 1].set_title('Test Accuracy for All Models', fontsize=15, weight='bold')
axes[0, 1].set_ylabel('Accuracy', fontsize=12)
for container in axes[0, 1].containers:
    axes[0, 1].bar_label(container, labels=[f'{v.get_height():.3f}' for v in
    container], padding=3)
axes[0, 1].tick_params(axis='x', rotation=-45)
axes[0, 1].set_xticklabels(metrics_df.index, fontdict=font_properties)

# Plot Train Precision and Test Precision
sns.barplot(data=metrics_df, x=metrics_df.index, y='train_precision',
            ax=axes[1, 0], palette='Set1_r')
axes[1, 0].set_title('Train Precision for All Models', fontsize=15,
                    weight='bold')
axes[1, 0].set_ylabel('Precision', fontsize=12)
for container in axes[1, 0].containers:
    axes[1, 0].bar_label(container, labels=[f'{v.get_height():.3f}' for v in
    container], padding=3)
axes[1, 0].tick_params(axis='x', rotation=-45)
axes[1, 0].set_xticklabels(metrics_df.index, fontdict=font_properties)

sns.barplot(data=metrics_df, x=metrics_df.index, y='test_precision', ax=axes[1,1],
            palette='Set2_r')
axes[1, 1].set_title('Test Precision for All Models', fontsize=15,
                    weight='bold')
axes[1, 1].set_ylabel('Precision', fontsize=12)
for container in axes[1, 1].containers:
```

```

        axes[1, 1].bar_label(container, labels=[f'{v.get_height():.3f}' for v in
        ↪container], padding=3)
axes[1, 1].tick_params(axis='x', rotation=-45)
axes[1, 1].set_xticklabels(metrics_df.index, fontdict=font_properties)

# Plot Train Recall and Test Recall
sns.barplot(data=metrics_df, x=metrics_df.index, y='train_recall', ax=axes[2,
    ↪0], palette='Set1')
axes[2, 0].set_title('Train Recall for All Models', fontsize=15, weight='bold')
axes[2, 0].set_ylabel('Recall', fontsize=12)
for container in axes[2, 0].containers:
    axes[2, 0].bar_label(container, labels=[f'{v.get_height():.3f}' for v in
    ↪container], padding=3)
axes[2, 0].tick_params(axis='x', rotation=-45)
axes[2, 0].set_xticklabels(metrics_df.index, fontdict=font_properties)

sns.barplot(data=metrics_df, x=metrics_df.index, y='test_recall', ax=axes[2,
    ↪1], palette='Set2')
axes[2, 1].set_title('Test Recall for All Models', fontsize=15, weight='bold')
axes[2, 1].set_ylabel('Recall', fontsize=12)
for container in axes[2, 1].containers:
    axes[2, 1].bar_label(container, labels=[f'{v.get_height():.3f}' for v in
    ↪container], padding=3)
axes[2, 1].tick_params(axis='x', rotation=-45)
axes[2, 1].set_xticklabels(metrics_df.index, fontdict=font_properties)

# Plot Train F1 Score and Test F1 Score
sns.barplot(data=metrics_df, x=metrics_df.index, y='train_f1', ax=axes[3, 0],
    ↪palette='Set1_r')
axes[3, 0].set_title('Train F1 Score for All Models', fontsize=15,
    ↪weight='bold')
axes[3, 0].set_ylabel('F1 Score', fontsize=12)
for container in axes[3, 0].containers:
    axes[3, 0].bar_label(container, labels=[f'{v.get_height():.3f}' for v in
    ↪container], padding=3)
axes[3, 0].tick_params(axis='x', rotation=-45)
axes[3, 0].set_xticklabels(metrics_df.index, fontdict=font_properties)

sns.barplot(data=metrics_df, x=metrics_df.index, y='test_f1', ax=axes[3, 1],
    ↪palette='Set2_r')
axes[3, 1].set_title('Test F1 Score for All Models', fontsize=15, weight='bold')
axes[3, 1].set_ylabel('F1 Score', fontsize=12)
for container in axes[3, 1].containers:
    axes[3, 1].bar_label(container, labels=[f'{v.get_height():.3f}' for v in
    ↪container], padding=3)
axes[3, 1].tick_params(axis='x', rotation=-45)

```

```

axes[3, 1].set_xticklabels(metrics_df.index, fontdict=font_properties)

# Final AUC plot and hide the empty subplot
sns.barplot(data=metrics_df, x=metrics_df.index, y='roc_auc', ax=axes[4, 0],
            palette='Set2_r')
axes[4, 0].set_title('Test ROC AUC for All Models', fontsize=15, weight='bold')
axes[4, 0].set_ylabel('AUC', fontsize=12)
for container in axes[4, 0].containers:
    axes[4, 0].bar_label(container, labels=[f'{v.get_height():.3f}' for v in
    container], padding=3)
axes[4, 0].tick_params(axis='x', rotation=-45)
axes[4, 0].set_xticklabels(metrics_df.index, fontdict=font_properties)

# Hide the empty subplot at [4, 1]
axes[4, 1].axis('off')

plt.tight_layout()
plt.show()

```




```
[86]: ### Save Best Model
with open(f'{best_model_name}_model.pkl', 'wb') as file:
    pickle.dump(best_model_name, file)
with open(f'{best_model_name}_model.pkl', 'rb') as file:
    loaded_model = pickle.load(file)
print(f"The model '{best_model_name}' has been loaded successfully.")
```

The model 'CatBoost' has been loaded successfully.

DL Models

Table of Contents

```
[87]: seed_value = 44
random.seed(seed_value)
np.random.seed(seed_value)
tf.random.set_seed(seed_value)
model = keras.models.Sequential([
    layers.Dense(256, activation='relu', input_shape=(X_train.shape[1],)),
    layers.Dropout(0.5),
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(128, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 256)	2,816
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 512)	131,584
dropout_1 (Dropout)	(None, 512)	0
dense_2 (Dense)	(None, 128)	65,664
dense_3 (Dense)	(None, 1)	129

Total params: 200,193 (782.00 KB)

Trainable params: 200,193 (782.00 KB)

Non-trainable params: 0 (0.00 B)

```
[88]: early_stopping = EarlyStopping(monitor='val_loss',patience=50,restore_best_weights=True)
      model_checkpoint = ModelCheckpoint('keras_model.
      keras',monitor='val_loss',save_best_only=True,mode='min',verbose=1)
      model.compile(optimizer='adam', loss='binary_crossentropy',
      metrics=['accuracy'])
      history = model.fit(X_train, y_train, epochs=100, batch_size=64,
      validation_split=0.2,
                        callbacks=[early_stopping,model_checkpoint])
```

Epoch 1/100

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR

I0000 00:00:1734112962.278164 527 service.cc:145] XLA service 0x7a8fac00dbf0 initialized for platform CUDA (this does not guarantee that XLA will be used).

Devices:

I0000 00:00:1734112962.278223 527 service.cc:153] StreamExecutor device (0): Tesla P100-PCIE-16GB, Compute Capability 6.0

1/17 1:17 5s/step -
accuracy: 0.5000 - loss: 0.7016

I0000 00:00:1734112966.004012 527 device_compiler.h:188] Compiled cluster using XLA! This line is logged at most once for the lifetime of the process.

Epoch 1: val_loss improved from inf to 0.61538, saving model to keras_model.keras

17/17 5s 37ms/step -
accuracy: 0.5596 - loss: 0.6664 - val_accuracy: 0.6581 - val_loss: 0.6154

Epoch 2/100

1/17 0s 20ms/step -
accuracy: 0.5781 - loss: 0.6362

Epoch 2: val_loss improved from 0.61538 to 0.52622, saving model to keras_model.keras

17/17 0s 6ms/step -
accuracy: 0.6456 - loss: 0.6142 - val_accuracy: 0.7647 - val_loss: 0.5262

Epoch 3/100

1/17 0s 20ms/step -
accuracy: 0.6875 - loss: 0.6178

Epoch 3: val_loss improved from 0.52622 to 0.46253, saving model to keras_model.keras

17/17 0s 5ms/step -

```

accuracy: 0.7252 - loss: 0.5544 - val_accuracy: 0.7941 - val_loss: 0.4625
Epoch 4/100
  1/17          0s 20ms/step -
accuracy: 0.6719 - loss: 0.5603
Epoch 4: val_loss improved from 0.46253 to 0.43389, saving model to
keras_model.keras
 17/17          0s 6ms/step -
accuracy: 0.7510 - loss: 0.5137 - val_accuracy: 0.8199 - val_loss: 0.4339
Epoch 5/100
  1/17          0s 21ms/step -
accuracy: 0.7344 - loss: 0.5717
Epoch 5: val_loss improved from 0.43389 to 0.42168, saving model to
keras_model.keras
 17/17          0s 6ms/step -
accuracy: 0.7516 - loss: 0.5009 - val_accuracy: 0.8346 - val_loss: 0.4217
Epoch 6/100
  1/17          0s 21ms/step -
accuracy: 0.7812 - loss: 0.5341
Epoch 6: val_loss improved from 0.42168 to 0.40077, saving model to
keras_model.keras
 17/17          0s 5ms/step -
accuracy: 0.7640 - loss: 0.4916 - val_accuracy: 0.8309 - val_loss: 0.4008
Epoch 7/100
  1/17          0s 20ms/step -
accuracy: 0.6562 - loss: 0.5610
Epoch 7: val_loss improved from 0.40077 to 0.38838, saving model to
keras_model.keras
 17/17          0s 6ms/step -
accuracy: 0.7524 - loss: 0.4768 - val_accuracy: 0.8456 - val_loss: 0.3884
Epoch 8/100
  1/17          0s 21ms/step -
accuracy: 0.7500 - loss: 0.5332
Epoch 8: val_loss improved from 0.38838 to 0.37972, saving model to
keras_model.keras
 17/17          0s 6ms/step -
accuracy: 0.7878 - loss: 0.4635 - val_accuracy: 0.8456 - val_loss: 0.3797
Epoch 9/100
  1/17          0s 21ms/step -
accuracy: 0.7812 - loss: 0.5155
Epoch 9: val_loss improved from 0.37972 to 0.37549, saving model to
keras_model.keras
 17/17          0s 6ms/step -
accuracy: 0.7979 - loss: 0.4519 - val_accuracy: 0.8640 - val_loss: 0.3755
Epoch 10/100
  1/17          0s 19ms/step -
accuracy: 0.7969 - loss: 0.4995
Epoch 10: val_loss improved from 0.37549 to 0.36700, saving model to
keras_model.keras

```

```

17/17          0s 5ms/step -
accuracy: 0.8078 - loss: 0.4414 - val_accuracy: 0.8419 - val_loss: 0.3670
Epoch 11/100
  1/17          0s 21ms/step -
accuracy: 0.7188 - loss: 0.5317
Epoch 11: val_loss improved from 0.36700 to 0.36544, saving model to
keras_model.keras
17/17          0s 5ms/step -
accuracy: 0.7929 - loss: 0.4532 - val_accuracy: 0.8493 - val_loss: 0.3654
Epoch 12/100
  1/17          0s 19ms/step -
accuracy: 0.7188 - loss: 0.5427
Epoch 12: val_loss did not improve from 0.36544
17/17          0s 3ms/step -
accuracy: 0.7820 - loss: 0.4562 - val_accuracy: 0.8346 - val_loss: 0.3686
Epoch 13/100
  1/17          0s 19ms/step -
accuracy: 0.7344 - loss: 0.5286
Epoch 13: val_loss improved from 0.36544 to 0.36428, saving model to
keras_model.keras
17/17          0s 5ms/step -
accuracy: 0.7899 - loss: 0.4402 - val_accuracy: 0.8456 - val_loss: 0.3643
Epoch 14/100
  1/17          0s 20ms/step -
accuracy: 0.7812 - loss: 0.5091
Epoch 14: val_loss improved from 0.36428 to 0.35813, saving model to
keras_model.keras
17/17          0s 5ms/step -
accuracy: 0.8127 - loss: 0.4292 - val_accuracy: 0.8419 - val_loss: 0.3581
Epoch 15/100
  1/17          0s 20ms/step -
accuracy: 0.7188 - loss: 0.5249
Epoch 15: val_loss improved from 0.35813 to 0.35788, saving model to
keras_model.keras
17/17          0s 5ms/step -
accuracy: 0.8161 - loss: 0.4168 - val_accuracy: 0.8419 - val_loss: 0.3579
Epoch 16/100
  1/17          0s 19ms/step -
accuracy: 0.7344 - loss: 0.5081
Epoch 16: val_loss improved from 0.35788 to 0.35551, saving model to
keras_model.keras
17/17          0s 6ms/step -
accuracy: 0.8079 - loss: 0.4198 - val_accuracy: 0.8456 - val_loss: 0.3555
Epoch 17/100
  1/17          0s 20ms/step -
accuracy: 0.7969 - loss: 0.4688
Epoch 17: val_loss did not improve from 0.35551
17/17          0s 3ms/step -

```

```

accuracy: 0.8162 - loss: 0.4129 - val_accuracy: 0.8382 - val_loss: 0.3601
Epoch 18/100
  1/17          0s 18ms/step -
accuracy: 0.7188 - loss: 0.5254
Epoch 18: val_loss improved from 0.35551 to 0.34962, saving model to
keras_model.keras
 17/17          0s 5ms/step -
accuracy: 0.8086 - loss: 0.4177 - val_accuracy: 0.8493 - val_loss: 0.3496
Epoch 19/100
  1/17          0s 19ms/step -
accuracy: 0.7500 - loss: 0.5207
Epoch 19: val_loss did not improve from 0.34962
 17/17          0s 3ms/step -
accuracy: 0.8259 - loss: 0.4096 - val_accuracy: 0.8529 - val_loss: 0.3593
Epoch 20/100
  1/17          0s 19ms/step -
accuracy: 0.7969 - loss: 0.4775
Epoch 20: val_loss did not improve from 0.34962
 17/17          0s 3ms/step -
accuracy: 0.8247 - loss: 0.4130 - val_accuracy: 0.8456 - val_loss: 0.3629
Epoch 21/100
  1/17          0s 18ms/step -
accuracy: 0.7500 - loss: 0.4399
Epoch 21: val_loss improved from 0.34962 to 0.34910, saving model to
keras_model.keras
 17/17          0s 5ms/step -
accuracy: 0.8050 - loss: 0.3953 - val_accuracy: 0.8382 - val_loss: 0.3491
Epoch 22/100
  1/17          0s 20ms/step -
accuracy: 0.7812 - loss: 0.4983
Epoch 22: val_loss did not improve from 0.34910
 17/17          0s 3ms/step -
accuracy: 0.8141 - loss: 0.4064 - val_accuracy: 0.8493 - val_loss: 0.3531
Epoch 23/100
  1/17          0s 18ms/step -
accuracy: 0.7656 - loss: 0.4853
Epoch 23: val_loss did not improve from 0.34910
 17/17          0s 3ms/step -
accuracy: 0.8144 - loss: 0.3983 - val_accuracy: 0.8346 - val_loss: 0.3605
Epoch 24/100
  1/17          0s 19ms/step -
accuracy: 0.7656 - loss: 0.5049
Epoch 24: val_loss did not improve from 0.34910
 17/17          0s 3ms/step -
accuracy: 0.8193 - loss: 0.4066 - val_accuracy: 0.8346 - val_loss: 0.3571
Epoch 25/100
  1/17          0s 18ms/step -
accuracy: 0.7656 - loss: 0.4790

```

Epoch 25: val_loss did not improve from 0.34910
17/17 0s 3ms/step -
accuracy: 0.8169 - loss: 0.4035 - val_accuracy: 0.8419 - val_loss: 0.3506
Epoch 26/100
1/17 0s 19ms/step -
accuracy: 0.7344 - loss: 0.4817
Epoch 26: val_loss improved from 0.34910 to 0.34579, saving model to
keras_model.keras
17/17 0s 5ms/step -
accuracy: 0.8088 - loss: 0.3963 - val_accuracy: 0.8640 - val_loss: 0.3458
Epoch 27/100
1/17 0s 19ms/step -
accuracy: 0.7812 - loss: 0.4934
Epoch 27: val_loss did not improve from 0.34579
17/17 0s 3ms/step -
accuracy: 0.8225 - loss: 0.3880 - val_accuracy: 0.8493 - val_loss: 0.3480
Epoch 28/100
1/17 0s 20ms/step -
accuracy: 0.7812 - loss: 0.4559
Epoch 28: val_loss did not improve from 0.34579
17/17 0s 4ms/step -
accuracy: 0.8319 - loss: 0.3817 - val_accuracy: 0.8493 - val_loss: 0.3502
Epoch 29/100
1/17 0s 19ms/step -
accuracy: 0.7188 - loss: 0.4953
Epoch 29: val_loss did not improve from 0.34579
17/17 0s 3ms/step -
accuracy: 0.8232 - loss: 0.3827 - val_accuracy: 0.8456 - val_loss: 0.3477
Epoch 30/100
1/17 0s 19ms/step -
accuracy: 0.7969 - loss: 0.4655
Epoch 30: val_loss improved from 0.34579 to 0.34379, saving model to
keras_model.keras
17/17 0s 5ms/step -
accuracy: 0.8332 - loss: 0.3822 - val_accuracy: 0.8493 - val_loss: 0.3438
Epoch 31/100
1/17 0s 20ms/step -
accuracy: 0.7812 - loss: 0.4974
Epoch 31: val_loss did not improve from 0.34379
17/17 0s 3ms/step -
accuracy: 0.8279 - loss: 0.3820 - val_accuracy: 0.8529 - val_loss: 0.3474
Epoch 32/100
1/17 0s 19ms/step -
accuracy: 0.7812 - loss: 0.4918
Epoch 32: val_loss improved from 0.34379 to 0.33823, saving model to
keras_model.keras
17/17 0s 5ms/step -
accuracy: 0.8257 - loss: 0.3846 - val_accuracy: 0.8676 - val_loss: 0.3382

Epoch 33/100
 1/17 0s 19ms/step -
 accuracy: 0.7500 - loss: 0.4816
 Epoch 33: val_loss did not improve from 0.33823
 17/17 0s 3ms/step -
 accuracy: 0.8342 - loss: 0.3745 - val_accuracy: 0.8566 - val_loss: 0.3412
 Epoch 34/100
 1/17 0s 18ms/step -
 accuracy: 0.8125 - loss: 0.4498
 Epoch 34: val_loss did not improve from 0.33823
 17/17 0s 3ms/step -
 accuracy: 0.8417 - loss: 0.3690 - val_accuracy: 0.8529 - val_loss: 0.3439
 Epoch 35/100
 1/17 0s 20ms/step -
 accuracy: 0.7812 - loss: 0.4636
 Epoch 35: val_loss did not improve from 0.33823
 17/17 0s 3ms/step -
 accuracy: 0.8367 - loss: 0.3671 - val_accuracy: 0.8640 - val_loss: 0.3445
 Epoch 36/100
 1/17 0s 19ms/step -
 accuracy: 0.8281 - loss: 0.4195
 Epoch 36: val_loss did not improve from 0.33823
 17/17 0s 3ms/step -
 accuracy: 0.8484 - loss: 0.3650 - val_accuracy: 0.8566 - val_loss: 0.3402
 Epoch 37/100
 1/17 0s 18ms/step -
 accuracy: 0.7969 - loss: 0.4408
 Epoch 37: val_loss did not improve from 0.33823
 17/17 0s 3ms/step -
 accuracy: 0.8362 - loss: 0.3674 - val_accuracy: 0.8640 - val_loss: 0.3432
 Epoch 38/100
 1/17 0s 19ms/step -
 accuracy: 0.7344 - loss: 0.5444
 Epoch 38: val_loss did not improve from 0.33823
 17/17 0s 3ms/step -
 accuracy: 0.8272 - loss: 0.3938 - val_accuracy: 0.8529 - val_loss: 0.3495
 Epoch 39/100
 1/17 0s 18ms/step -
 accuracy: 0.7969 - loss: 0.4148
 Epoch 39: val_loss did not improve from 0.33823
 17/17 0s 3ms/step -
 accuracy: 0.8300 - loss: 0.3641 - val_accuracy: 0.8456 - val_loss: 0.3506
 Epoch 40/100
 1/17 0s 18ms/step -
 accuracy: 0.8125 - loss: 0.5021
 Epoch 40: val_loss did not improve from 0.33823
 17/17 0s 3ms/step -
 accuracy: 0.8594 - loss: 0.3722 - val_accuracy: 0.8419 - val_loss: 0.3486

Epoch 41/100
 1/17 0s 18ms/step -
 accuracy: 0.7656 - loss: 0.4248
 Epoch 41: val_loss did not improve from 0.33823
 17/17 0s 3ms/step -
 accuracy: 0.8334 - loss: 0.3633 - val_accuracy: 0.8529 - val_loss: 0.3413
 Epoch 42/100
 1/17 0s 18ms/step -
 accuracy: 0.7969 - loss: 0.4986
 Epoch 42: val_loss did not improve from 0.33823
 17/17 0s 3ms/step -
 accuracy: 0.8473 - loss: 0.3754 - val_accuracy: 0.8419 - val_loss: 0.3450
 Epoch 43/100
 1/17 0s 20ms/step -
 accuracy: 0.7031 - loss: 0.5487
 Epoch 43: val_loss did not improve from 0.33823
 17/17 0s 3ms/step -
 accuracy: 0.8213 - loss: 0.3774 - val_accuracy: 0.8419 - val_loss: 0.3470
 Epoch 44/100
 1/17 0s 18ms/step -
 accuracy: 0.8125 - loss: 0.4458
 Epoch 44: val_loss did not improve from 0.33823
 17/17 0s 3ms/step -
 accuracy: 0.8562 - loss: 0.3553 - val_accuracy: 0.8493 - val_loss: 0.3450
 Epoch 45/100
 1/17 0s 19ms/step -
 accuracy: 0.8125 - loss: 0.4167
 Epoch 45: val_loss did not improve from 0.33823
 17/17 0s 3ms/step -
 accuracy: 0.8399 - loss: 0.3484 - val_accuracy: 0.8493 - val_loss: 0.3397
 Epoch 46/100
 1/17 0s 18ms/step -
 accuracy: 0.8281 - loss: 0.4143
 Epoch 46: val_loss did not improve from 0.33823
 17/17 0s 3ms/step -
 accuracy: 0.8547 - loss: 0.3511 - val_accuracy: 0.8566 - val_loss: 0.3434
 Epoch 47/100
 1/17 0s 18ms/step -
 accuracy: 0.8125 - loss: 0.3955
 Epoch 47: val_loss did not improve from 0.33823
 17/17 0s 3ms/step -
 accuracy: 0.8491 - loss: 0.3396 - val_accuracy: 0.8566 - val_loss: 0.3464
 Epoch 48/100
 1/17 0s 18ms/step -
 accuracy: 0.7656 - loss: 0.4068
 Epoch 48: val_loss did not improve from 0.33823
 17/17 0s 3ms/step -
 accuracy: 0.8357 - loss: 0.3396 - val_accuracy: 0.8603 - val_loss: 0.3393

Epoch 49/100
 1/17 0s 19ms/step -
 accuracy: 0.8125 - loss: 0.3483
 Epoch 49: val_loss did not improve from 0.33823
 17/17 0s 3ms/step -
 accuracy: 0.8535 - loss: 0.3272 - val_accuracy: 0.8456 - val_loss: 0.3504
 Epoch 50/100
 1/17 0s 18ms/step -
 accuracy: 0.8281 - loss: 0.4269
 Epoch 50: val_loss did not improve from 0.33823
 17/17 0s 3ms/step -
 accuracy: 0.8399 - loss: 0.3512 - val_accuracy: 0.8640 - val_loss: 0.3520
 Epoch 51/100
 1/17 0s 19ms/step -
 accuracy: 0.7812 - loss: 0.5229
 Epoch 51: val_loss did not improve from 0.33823
 17/17 0s 3ms/step -
 accuracy: 0.8385 - loss: 0.3750 - val_accuracy: 0.8713 - val_loss: 0.3438
 Epoch 52/100
 1/17 0s 17ms/step -
 accuracy: 0.7812 - loss: 0.5360
 Epoch 52: val_loss did not improve from 0.33823
 17/17 0s 3ms/step -
 accuracy: 0.8391 - loss: 0.3688 - val_accuracy: 0.8676 - val_loss: 0.3451
 Epoch 53/100
 1/17 0s 19ms/step -
 accuracy: 0.8125 - loss: 0.4177
 Epoch 53: val_loss improved from 0.33823 to 0.33609, saving model to
 keras_model.keras
 17/17 0s 5ms/step -
 accuracy: 0.8436 - loss: 0.3425 - val_accuracy: 0.8529 - val_loss: 0.3361
 Epoch 54/100
 1/17 0s 18ms/step -
 accuracy: 0.8750 - loss: 0.3405
 Epoch 54: val_loss did not improve from 0.33609
 17/17 0s 3ms/step -
 accuracy: 0.8478 - loss: 0.3248 - val_accuracy: 0.8529 - val_loss: 0.3523
 Epoch 55/100
 1/17 0s 17ms/step -
 accuracy: 0.7812 - loss: 0.4071
 Epoch 55: val_loss did not improve from 0.33609
 17/17 0s 3ms/step -
 accuracy: 0.8361 - loss: 0.3466 - val_accuracy: 0.8603 - val_loss: 0.3434
 Epoch 56/100
 1/17 0s 18ms/step -
 accuracy: 0.8125 - loss: 0.4460
 Epoch 56: val_loss did not improve from 0.33609
 17/17 0s 3ms/step -

```

accuracy: 0.8416 - loss: 0.3450 - val_accuracy: 0.8713 - val_loss: 0.3369
Epoch 57/100
  1/17          0s 19ms/step -
accuracy: 0.8125 - loss: 0.4722
Epoch 57: val_loss did not improve from 0.33609
 17/17          0s 3ms/step -
accuracy: 0.8542 - loss: 0.3524 - val_accuracy: 0.8713 - val_loss: 0.3442
Epoch 58/100
  1/17          0s 18ms/step -
accuracy: 0.8438 - loss: 0.3771
Epoch 58: val_loss did not improve from 0.33609
 17/17          0s 3ms/step -
accuracy: 0.8671 - loss: 0.3175 - val_accuracy: 0.8603 - val_loss: 0.3513
Epoch 59/100
  1/17          0s 19ms/step -
accuracy: 0.8594 - loss: 0.4081
Epoch 59: val_loss did not improve from 0.33609
 17/17          0s 3ms/step -
accuracy: 0.8588 - loss: 0.3355 - val_accuracy: 0.8676 - val_loss: 0.3454
Epoch 60/100
  1/17          0s 18ms/step -
accuracy: 0.8594 - loss: 0.3953
Epoch 60: val_loss did not improve from 0.33609
 17/17          0s 3ms/step -
accuracy: 0.8833 - loss: 0.3145 - val_accuracy: 0.8640 - val_loss: 0.3519
Epoch 61/100
  1/17          0s 18ms/step -
accuracy: 0.8281 - loss: 0.4220
Epoch 61: val_loss did not improve from 0.33609
 17/17          0s 3ms/step -
accuracy: 0.8718 - loss: 0.3173 - val_accuracy: 0.8640 - val_loss: 0.3446
Epoch 62/100
  1/17          0s 18ms/step -
accuracy: 0.8750 - loss: 0.4172
Epoch 62: val_loss did not improve from 0.33609
 17/17          0s 3ms/step -
accuracy: 0.8649 - loss: 0.3423 - val_accuracy: 0.8676 - val_loss: 0.3459
Epoch 63/100
  1/17          0s 18ms/step -
accuracy: 0.8125 - loss: 0.4146
Epoch 63: val_loss did not improve from 0.33609
 17/17          0s 3ms/step -
accuracy: 0.8618 - loss: 0.3274 - val_accuracy: 0.8640 - val_loss: 0.3468
Epoch 64/100
  1/17          0s 19ms/step -
accuracy: 0.8438 - loss: 0.3504
Epoch 64: val_loss did not improve from 0.33609
 17/17          0s 3ms/step -

```

```

accuracy: 0.8771 - loss: 0.2993 - val_accuracy: 0.8640 - val_loss: 0.3576
Epoch 65/100
  1/17          0s 18ms/step -
accuracy: 0.7969 - loss: 0.4414
Epoch 65: val_loss did not improve from 0.33609
 17/17          0s 3ms/step -
accuracy: 0.8546 - loss: 0.3405 - val_accuracy: 0.8676 - val_loss: 0.3401
Epoch 66/100
  1/17          0s 18ms/step -
accuracy: 0.8438 - loss: 0.4129
Epoch 66: val_loss did not improve from 0.33609
 17/17          0s 3ms/step -
accuracy: 0.8573 - loss: 0.3367 - val_accuracy: 0.8750 - val_loss: 0.3409
Epoch 67/100
  1/17          0s 17ms/step -
accuracy: 0.8750 - loss: 0.3518
Epoch 67: val_loss improved from 0.33609 to 0.33556, saving model to
keras_model.keras
 17/17          0s 5ms/step -
accuracy: 0.8732 - loss: 0.3203 - val_accuracy: 0.8750 - val_loss: 0.3356
Epoch 68/100
  1/17          0s 21ms/step -
accuracy: 0.8438 - loss: 0.3786
Epoch 68: val_loss did not improve from 0.33556
 17/17          0s 3ms/step -
accuracy: 0.8641 - loss: 0.3266 - val_accuracy: 0.8676 - val_loss: 0.3390
Epoch 69/100
  1/17          0s 18ms/step -
accuracy: 0.8438 - loss: 0.3661
Epoch 69: val_loss did not improve from 0.33556
 17/17          0s 3ms/step -
accuracy: 0.8715 - loss: 0.3065 - val_accuracy: 0.8603 - val_loss: 0.3508
Epoch 70/100
  1/17          0s 18ms/step -
accuracy: 0.8438 - loss: 0.3886
Epoch 70: val_loss did not improve from 0.33556
 17/17          0s 3ms/step -
accuracy: 0.8728 - loss: 0.3094 - val_accuracy: 0.8640 - val_loss: 0.3502
Epoch 71/100
  1/17          0s 18ms/step -
accuracy: 0.8438 - loss: 0.3580
Epoch 71: val_loss did not improve from 0.33556
 17/17          0s 3ms/step -
accuracy: 0.8816 - loss: 0.2877 - val_accuracy: 0.8640 - val_loss: 0.3477
Epoch 72/100
  1/17          0s 17ms/step -
accuracy: 0.8750 - loss: 0.3712
Epoch 72: val_loss did not improve from 0.33556

```

```

17/17          0s 3ms/step -
accuracy: 0.8758 - loss: 0.3116 - val_accuracy: 0.8787 - val_loss: 0.3477
Epoch 73/100
  1/17          0s 17ms/step -
accuracy: 0.8594 - loss: 0.4152
Epoch 73: val_loss did not improve from 0.33556
17/17          0s 3ms/step -
accuracy: 0.8760 - loss: 0.3059 - val_accuracy: 0.8640 - val_loss: 0.3563
Epoch 74/100
  1/17          0s 18ms/step -
accuracy: 0.8281 - loss: 0.3946
Epoch 74: val_loss did not improve from 0.33556
17/17          0s 3ms/step -
accuracy: 0.8814 - loss: 0.2976 - val_accuracy: 0.8640 - val_loss: 0.3430
Epoch 75/100
  1/17          0s 18ms/step -
accuracy: 0.8281 - loss: 0.4185
Epoch 75: val_loss did not improve from 0.33556
17/17          0s 3ms/step -
accuracy: 0.8675 - loss: 0.3047 - val_accuracy: 0.8566 - val_loss: 0.3543
Epoch 76/100
  1/17          0s 18ms/step -
accuracy: 0.8438 - loss: 0.3731
Epoch 76: val_loss did not improve from 0.33556
17/17          0s 3ms/step -
accuracy: 0.8632 - loss: 0.3047 - val_accuracy: 0.8566 - val_loss: 0.3495
Epoch 77/100
  1/17          0s 18ms/step -
accuracy: 0.8750 - loss: 0.3643
Epoch 77: val_loss did not improve from 0.33556
17/17          0s 4ms/step -
accuracy: 0.8789 - loss: 0.3006 - val_accuracy: 0.8676 - val_loss: 0.3532
Epoch 78/100
  1/17          0s 17ms/step -
accuracy: 0.8125 - loss: 0.3876
Epoch 78: val_loss did not improve from 0.33556
17/17          0s 3ms/step -
accuracy: 0.8689 - loss: 0.3033 - val_accuracy: 0.8603 - val_loss: 0.3616
Epoch 79/100
  1/17          0s 25ms/step -
accuracy: 0.8750 - loss: 0.3301
Epoch 79: val_loss did not improve from 0.33556
17/17          0s 4ms/step -
accuracy: 0.8783 - loss: 0.2884 - val_accuracy: 0.8787 - val_loss: 0.3452
Epoch 80/100
  1/17          0s 18ms/step -
accuracy: 0.8281 - loss: 0.4169
Epoch 80: val_loss did not improve from 0.33556

```

```

17/17          0s 3ms/step -
accuracy: 0.8741 - loss: 0.3026 - val_accuracy: 0.8676 - val_loss: 0.3509
Epoch 81/100
  1/17          0s 19ms/step -
accuracy: 0.7969 - loss: 0.4553
Epoch 81: val_loss did not improve from 0.33556
17/17          0s 3ms/step -
accuracy: 0.8506 - loss: 0.3223 - val_accuracy: 0.8640 - val_loss: 0.3442
Epoch 82/100
  1/17          0s 19ms/step -
accuracy: 0.8438 - loss: 0.3258
Epoch 82: val_loss did not improve from 0.33556
17/17          0s 4ms/step -
accuracy: 0.8737 - loss: 0.2773 - val_accuracy: 0.8676 - val_loss: 0.3532
Epoch 83/100
  1/17          0s 19ms/step -
accuracy: 0.8281 - loss: 0.3877
Epoch 83: val_loss did not improve from 0.33556
17/17          0s 3ms/step -
accuracy: 0.8760 - loss: 0.2917 - val_accuracy: 0.8603 - val_loss: 0.3632
Epoch 84/100
  1/17          0s 19ms/step -
accuracy: 0.8281 - loss: 0.3825
Epoch 84: val_loss did not improve from 0.33556
17/17          0s 3ms/step -
accuracy: 0.8642 - loss: 0.3030 - val_accuracy: 0.8750 - val_loss: 0.3461
Epoch 85/100
  1/17          0s 18ms/step -
accuracy: 0.8281 - loss: 0.3597
Epoch 85: val_loss did not improve from 0.33556
17/17          0s 3ms/step -
accuracy: 0.8740 - loss: 0.2884 - val_accuracy: 0.8750 - val_loss: 0.3506
Epoch 86/100
  1/17          0s 17ms/step -
accuracy: 0.8281 - loss: 0.3423
Epoch 86: val_loss did not improve from 0.33556
17/17          0s 3ms/step -
accuracy: 0.8867 - loss: 0.2718 - val_accuracy: 0.8787 - val_loss: 0.3569
Epoch 87/100
  1/17          0s 17ms/step -
accuracy: 0.8906 - loss: 0.3092
Epoch 87: val_loss did not improve from 0.33556
17/17          0s 3ms/step -
accuracy: 0.8828 - loss: 0.2813 - val_accuracy: 0.8676 - val_loss: 0.3631
Epoch 88/100
  1/17          0s 19ms/step -
accuracy: 0.8281 - loss: 0.4047
Epoch 88: val_loss did not improve from 0.33556

```

```

17/17          0s 3ms/step -
accuracy: 0.8809 - loss: 0.2895 - val_accuracy: 0.8640 - val_loss: 0.3648
Epoch 89/100
  1/17          0s 18ms/step -
accuracy: 0.8906 - loss: 0.2732
Epoch 89: val_loss did not improve from 0.33556
17/17          0s 4ms/step -
accuracy: 0.8923 - loss: 0.2686 - val_accuracy: 0.8713 - val_loss: 0.3493
Epoch 90/100
  1/17          0s 19ms/step -
accuracy: 0.8281 - loss: 0.4434
Epoch 90: val_loss did not improve from 0.33556
17/17          0s 3ms/step -
accuracy: 0.8701 - loss: 0.3002 - val_accuracy: 0.8750 - val_loss: 0.3379
Epoch 91/100
  1/17          0s 19ms/step -
accuracy: 0.8594 - loss: 0.3454
Epoch 91: val_loss did not improve from 0.33556
17/17          0s 3ms/step -
accuracy: 0.8897 - loss: 0.2642 - val_accuracy: 0.8824 - val_loss: 0.3466
Epoch 92/100
  1/17          0s 19ms/step -
accuracy: 0.7969 - loss: 0.3892
Epoch 92: val_loss did not improve from 0.33556
17/17          0s 3ms/step -
accuracy: 0.8768 - loss: 0.2826 - val_accuracy: 0.8713 - val_loss: 0.3541
Epoch 93/100
  1/17          0s 19ms/step -
accuracy: 0.8125 - loss: 0.3424
Epoch 93: val_loss did not improve from 0.33556
17/17          0s 3ms/step -
accuracy: 0.8765 - loss: 0.2758 - val_accuracy: 0.8676 - val_loss: 0.3526
Epoch 94/100
  1/17          0s 19ms/step -
accuracy: 0.8750 - loss: 0.3494
Epoch 94: val_loss did not improve from 0.33556
17/17          0s 3ms/step -
accuracy: 0.8827 - loss: 0.2807 - val_accuracy: 0.8676 - val_loss: 0.3571
Epoch 95/100
  1/17          0s 19ms/step -
accuracy: 0.8438 - loss: 0.4554
Epoch 95: val_loss did not improve from 0.33556
17/17          0s 3ms/step -
accuracy: 0.8800 - loss: 0.2944 - val_accuracy: 0.8676 - val_loss: 0.3579
Epoch 96/100
  1/17          0s 17ms/step -
accuracy: 0.8281 - loss: 0.4010
Epoch 96: val_loss did not improve from 0.33556

```

```

17/17          0s 3ms/step -
accuracy: 0.8814 - loss: 0.2836 - val_accuracy: 0.8750 - val_loss: 0.3590
Epoch 97/100
 1/17          0s 18ms/step -
accuracy: 0.9219 - loss: 0.3058
Epoch 97: val_loss did not improve from 0.33556
17/17          0s 3ms/step -
accuracy: 0.9051 - loss: 0.2594 - val_accuracy: 0.8676 - val_loss: 0.3579
Epoch 98/100
 1/17          0s 17ms/step -
accuracy: 0.8438 - loss: 0.3785
Epoch 98: val_loss did not improve from 0.33556
17/17          0s 3ms/step -
accuracy: 0.8815 - loss: 0.2724 - val_accuracy: 0.8750 - val_loss: 0.3680
Epoch 99/100
 1/17          0s 18ms/step -
accuracy: 0.8438 - loss: 0.3478
Epoch 99: val_loss did not improve from 0.33556
17/17          0s 3ms/step -
accuracy: 0.8872 - loss: 0.2626 - val_accuracy: 0.8713 - val_loss: 0.3627
Epoch 100/100
 1/17          0s 18ms/step -
accuracy: 0.8125 - loss: 0.3338
Epoch 100: val_loss did not improve from 0.33556
17/17          0s 3ms/step -
accuracy: 0.8785 - loss: 0.2568 - val_accuracy: 0.8750 - val_loss: 0.3641

```

```

[89]: history_df = pd.DataFrame(history.history)
      history_df

```

```

[89]:   accuracy      loss  val_accuracy  val_loss
0   0.574449  0.648900      0.658088  0.615384
1   0.680147  0.596749      0.764706  0.526221
2   0.731618  0.538160      0.794118  0.462528
3   0.749081  0.505830      0.819853  0.433890
4   0.762868  0.485573      0.834559  0.421683
..      ...      ...      ...      ...
95  0.901654  0.244461      0.875000  0.358968
96  0.903493  0.242797      0.867647  0.357853
97  0.887868  0.255581      0.875000  0.368005
98  0.896140  0.238667      0.871324  0.362672
99  0.897978  0.230410      0.875000  0.364109

[100 rows x 4 columns]

```

```

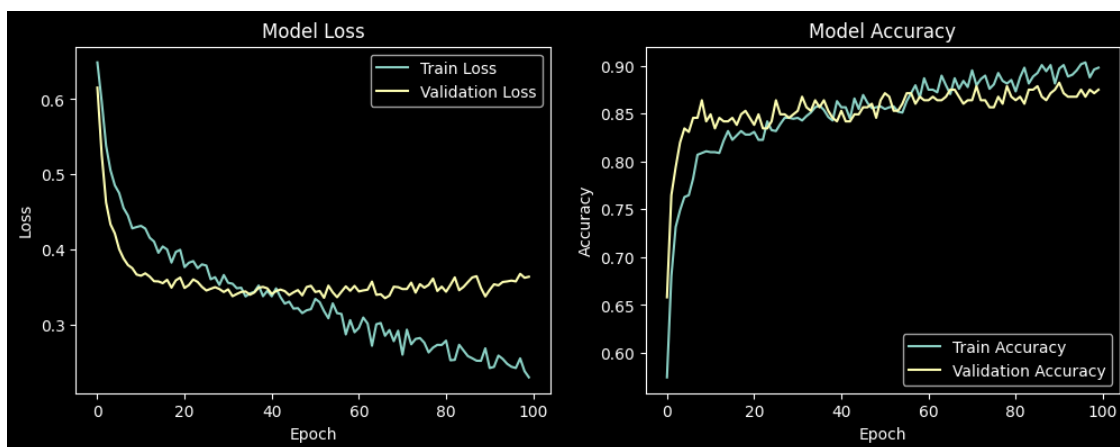
[90]: plt.figure(figsize=(12, 4))
      plt.subplot(1, 2, 1)
      plt.plot(history.history['loss'], label='Train Loss')

```

```

plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()

```



```

[91]: test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=0)
y_pred_prob = model.predict(X_test)
y_pred_classes = (y_pred_prob > 0.5).astype(int)
print(f"Test Loss: {test_loss:.3f}")
print(f"Test Accuracy: {test_accuracy:.3f}")
test_precision = precision_score(y_test, y_pred_classes)
test_recall = recall_score(y_test, y_pred_classes)
test_f1 = f1_score(y_test, y_pred_classes)
print(f"Test Precision: {test_precision:.3f}")
print(f"Test Recall: {test_recall:.3f}")
print(f"Test F1 Score: {test_f1:.3f}")
# Calculate AUC
fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob)
roc_auc = roc_auc_score(y_test, y_pred_prob)
print(f"ROC AUC Score (Test): {roc_auc:.4f}")

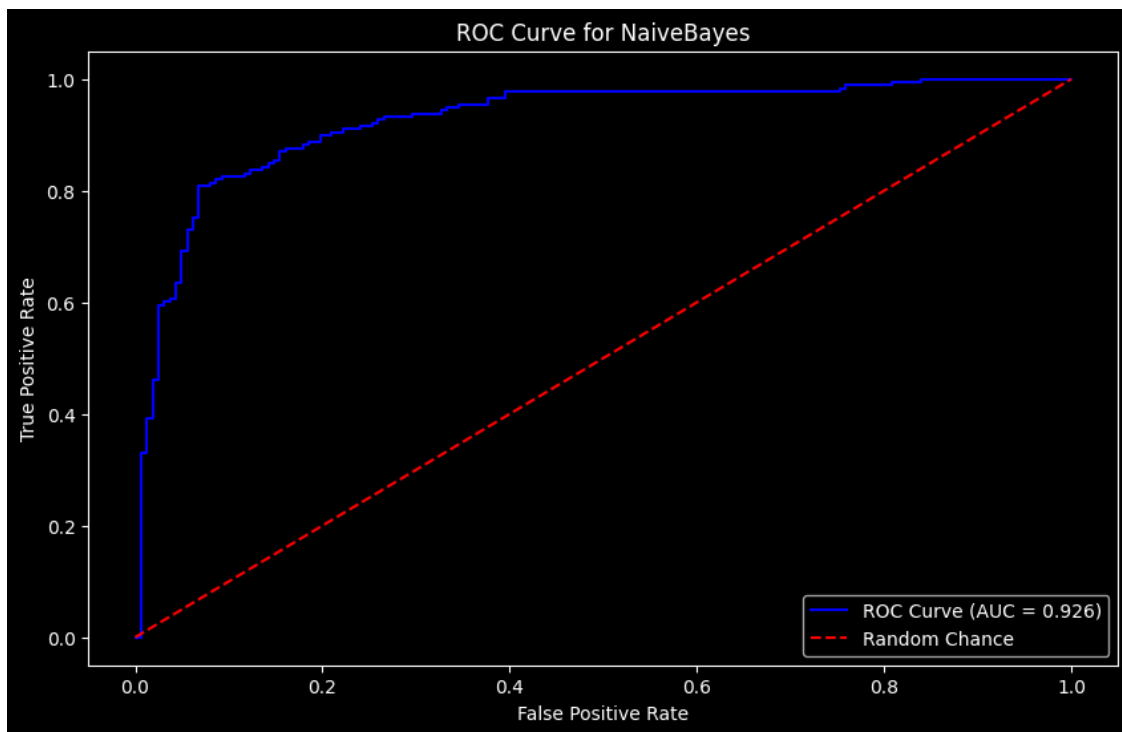
```

11/11 0s 14ms/step

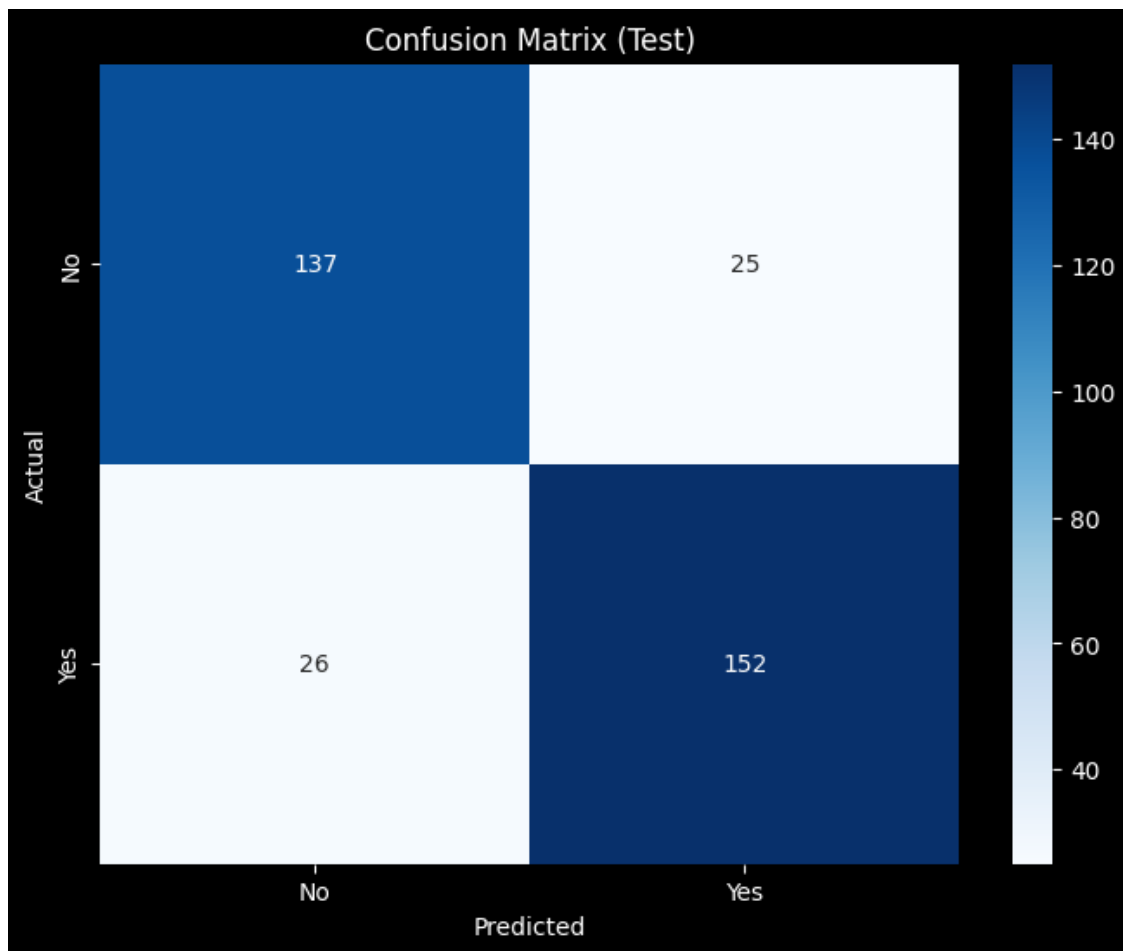
Test Loss: 0.361

Test Accuracy: 0.850
Test Precision: 0.859
Test Recall: 0.854
Test F1 Score: 0.856
ROC AUC Score (Test): 0.9260

```
[92]: plt.figure(figsize=(10, 6))
plt.plot(fpr, tpr, color='blue', label=f'ROC Curve (AUC = {roc_auc:.3f})')
plt.plot([0, 1], [0, 1], color='red', linestyle='--', label='Random Chance')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title(f'ROC Curve for {model_name}')
plt.legend(loc='lower right')
plt.show()
```



```
[93]: conf_matrix = confusion_matrix(y_test, y_pred_classes)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=['No', 'Yes'], yticklabels=['No', 'Yes'])
plt.title('Confusion Matrix (Test)')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
conf_matrix
```



```
[93]: array([[137, 25],
            [ 26, 152]])
```

```
[94]: class_report = classification_report(y_test, y_pred_classes,
      ↪target_names=['No', 'Yes'])
print("Classification Report (Test):")
print(class_report)
```

Classification Report (Test):

	precision	recall	f1-score	support
No	0.84	0.85	0.84	162
Yes	0.86	0.85	0.86	178
accuracy			0.85	340
macro avg	0.85	0.85	0.85	340
weighted avg	0.85	0.85	0.85	340